

Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

September 24, 2026

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Problem Statement	5
1.3	Contributions	5
1.4	Thesis Outline	6
2	Regular Expressions	7
2.1	Syntax	7
2.2	Semantics	8
2.3	Rust Representation	9
3	Derivatives and Partial Derivatives	10
3.1	Naive Matching	10
3.2	Brzozowski Derivatives	12
3.3	Antimirov Partial Derivatives	18
4	Matching versus Parsing	23
4.1	Matching Ambiguity	23
4.2	Parse Trees as Matching Evidence	24
4.3	Disambiguation	26
5	Derivative-Based Parser	28
5.1	From Answer to Evidence	28
5.2	The Direct Construction	29
5.3	The Bit-Coded Construction	33
6	Partial-Derivative-Based Parser	39
6.1	From One Expression to a Frontier	39
6.1.1	Where the Choice Is Actually Made	40
6.1.2	A Family of Witnesses	41
6.2	The Bit-Coded Construction: <code>pderiv_bc</code>	41
6.2.1	The Running Example, Strand by Strand	45
6.3	The Standard Construction: <code>pderiv_std</code>	48
6.3.1	What Each Strand Carries	48
6.3.2	One Table, Two Representations	49
6.4	Frontier Growth	54
6.5	The Two Policies Compared	56

7	External versus Internal Regex	58
7.1	Two ASTs, One Lowering Step	58
7.2	Parsing Surface Syntax	59
7.3	Translating <code>ExtPat</code> into <code>Regex</code>	61
7.4	The Cost of Translation	63
8	Evaluation	66
8.1	Compared Engines	66
8.1.1	Rust's <code>regex</code> Crate	66
8.1.2	RE2	67
8.2	Testing Strategy	67
8.3	Dataset Preparation	67
8.3.1	Substring Search: How the Corpus Is Run	69
8.4	Correctness Verification	70
8.4.1	Paper Examples Reproduced	70
8.4.2	Cross-Implementation Equivalence	71
8.5	Performance Analysis	72
8.5.1	Stack Depth: Recursive vs. Iterative Construction	72
8.5.2	Heap Allocation: <code>deriv_std</code> vs. <code>deriv_bc</code>	72
8.5.3	Derivative-Based vs. Partial-Derivative-Based	72
8.5.4	Pathological Inputs	72
8.5.5	Benchmark Results	72
8.6	Comparison with Existing Engines	73
8.6.1	Performance	73
8.6.2	Standardized Benchmarking via <code>rebar</code>	73
8.7	Optimization Challenges and Insights	73
8.8	Discussion of Results	74
9	Related Work and Conclusion	75
9.1	Related Work	75
9.2	Conclusion	75
A	Code	78
A.1	Core Types	78
A.2	Shared Algorithms	79
A.3	Matchers	81
A.4	The <code>deriv_std</code> Parser	81
A.5	The <code>deriv_bc</code> Parser	82
A.6	The <code>pderiv_bc</code> Parser	83
A.7	The <code>pderiv_std</code> Parser	85
A.8	Frontend	87
B	Tests	90
B.1	Shared Fixtures	90
B.2	Paper Examples Reproduced	91
B.3	Cross-Implementation Equivalence	92
B.4	Frontend	93
C	Diagnostics	94

C.1	deriv_std_rec, match	94
C.2	deriv_bc, match	96
C.3	pderiv_std, match	97
C.4	pderiv_bc, match	99
C.5	deriv_std_rec, no match	100
C.6	deriv_bc, no match	101
C.7	pderiv_std, no match	102
C.8	pderiv_bc, no match	104

1 Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the problem is regular expression matching. Given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression $(ab|a)^*$ matches "aba", as it decomposes into "ab" followed by "a", but does not match "abb", since no sequence of subexpressions can produce it.

For many applications, a Boolean answer is enough. An input validator may only need to accept or reject a password or URL. Others require more. For instance, a lexer must determine how input decomposes into tokens. Capturing groups, schema validation, and information extraction require identifying which input parts correspond to parts of the expression. Recognition alone is not enough; the matcher must also recover the *structure*.

Efficiently solving these problems remains an active research area. The choice of algorithm influences performance, memory consumption, and security. Once matching becomes parsing, the way in which match structure is recovered also influences correctness and raises the question of how to handle ambiguity when multiple matches are possible.

Existing approaches include backtracking and finite-automata-based techniques, such as nondeterministic (NFA) and deterministic (DFA) automata. These approaches have different trade-offs in implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely used, but may exhibit exponential running time on constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require preprocessing or suffer from state-space explosion.

Derivative-based algorithms provide an alternative by transforming the regular expression as input symbols are consumed. Each derivative represents the language remaining after an input symbol has been consumed. By maintaining a correspondence between an expression and its successive residuals, derivative-based methods provide an elegant foundation for regular expression matching and, with suitable extensions, structured parsing.

However, derivative-based techniques are less common in practical regular expression tooling. Their use for practical matching, parsing, and diagnostic tools motivates this thesis: "From Theory to Tools".

1.2 Problem Statement

Derivative-based approaches provide a principled foundation for regular expression matching and parsing. Brzozowski introduced regular expression derivatives as a way of transforming a regular expression after consuming an input symbol [Brzozowski, 1964]. Antimirov later introduced partial derivatives, which represent sets of residual expressions and are closely related to nondeterministic automata [Antimirov, 1996].

Building on Brzozowski’s derivatives, Sulzmann and Lu developed a derivative-based algorithm for POSIX regular expression parsing [Sulzmann and Lu, 2014]. Their work also presents a bit-coded variant that represents parse information as a compact bit sequence, which can be decoded after a successful match rather than maintaining an explicit parse tree throughout the computation. A Haskell implementation of the POSIX parser is available in Sulzmann’s *Programming Paradigms* teaching material [Sulzmann, 2026].

Sulzmann and Lu also studied partial derivatives in the context of regular expression sub-matching [Sulzmann and Lu, 2012]. While sub-matching is not the focus of this thesis, their work provides relevant background for the partial-derivative approach. The *Programming Paradigms* teaching material also presents a bit-coded partial-derivative-based parser in Haskell.

These constructions have primarily been presented in a theoretical and functional programming setting, leaving open practical questions about execution time, memory consumption, and data representation.

This thesis investigates these questions through implementations in Rust. As a systems programming language, Rust provides fine-grained control over memory allocation and data representation, allowing the derivative-based and partial-derivative-based parsers, including their bit-coded variants, to be evaluated in practice.

The implementation also provides traced versions of the parsing algorithms that record intermediate derivative computations during matching. These traces show the transformations performed by the algorithms and help explain successful and unsuccessful matches.

1.3 Contributions

The following contributions are being made:

- **Rust implementation of derivative-based parsers.** The thesis implements derivative-based and partial-derivative-based parsing constructions in Rust, including direct and bit-coded variants. The implementations are based on the constructions described in the literature and the corresponding Haskell material, resulting in four parser variants in total.
- **Empirical evaluation of the parser variants.** The four implementations are evaluated with respect to execution time and memory consumption using synthetic benchmarks and a corpus of real-world regular expressions from Snort/Suricata, SpamAssassin, and RegexLib, following the source categories investigated by Mamouras et al. [Mamouras et al., 2024]. They are also compared with Rust’s `regex` crate and Google’s RE2.

1.4 Thesis Outline

The remainder of this thesis follows the parsing constructions from their formal concepts through their Rust implementation to their evaluation. Theory and implementation are presented together.

- **Chapter 2 - Regular Expressions.** This chapter introduces the syntax and semantics of regular expressions and the Rust representation used throughout the implementation.
- **Chapter 3 - Derivatives.** This chapter introduces Brzozowski’s derivatives and Antimirov’s partial derivatives, together with their basic matching algorithms and Rust implementations.
- **Chapter 4 - Matching vs. Parsing.** This chapter distinguishes regular expression matching from parsing and introduces the disambiguation problem.
- **Chapter 5 - Derivative Parser.** This chapter presents Sulzmann and Lu’s derivative-based POSIX parser and its Rust implementation in direct and bit-coded variants.
- **Chapter 6 - Partial-Derivative Parser.** This chapter presents the partial-derivative-based parser and its Rust implementation in direct and bit-coded variants. It also discusses the differences between the two derivative-based approaches.
- **Chapter 7 - External versus Internal Syntax.** This chapter describes the parsing of regular expression strings into an external AST and their translation into the internal representation used by the parsers.
- **Chapter 8 - Evaluation.** This chapter evaluates the four parser variants with respect to execution time, memory consumption, and correctness. The evaluation uses synthetic benchmarks and a corpus of real-world regular expressions, and compares the implementations with Rust’s `regex` crate and Google’s RE2. It also examines optimization challenges and the insights gained from the evaluation, including the derivative traces produced by the implementations.
- **Chapter 9 - Conclusion.** Finally, this chapter summarizes the findings of the thesis and discusses the practical implications of the results and directions for future work.

2 Regular Expressions

2.1 Syntax

Let Σ be an alphabet. Regular expressions over Σ are defined inductively by the grammar:

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \quad (x \in \Sigma). \quad (2.1)$$

In the grammar, r and s range over regular expressions, while x ranges over the alphabet Σ and denotes an arbitrary symbol, such as a , b , or c . The constants ε and $\emptyset$ represent the *empty-word* and *empty-language* expressions. The operators $+$, $\cdot$, and $*$ denote alternation, concatenation, and Kleene star, respectively.

Concatenation is written implicitly, so rs denotes $r \cdot s$. Given this notation, we adopt the precedence and associativity conventions. Unless parentheses are shown explicitly, Kleene star binds most tightly, followed by concatenation, followed by alternation. For example,

$$\begin{aligned} ab^*c &= (a \cdot (b^*)) \cdot c, \\ a + bc^* &= a + (b \cdot (c^*)). \end{aligned}$$

For repeated operators we use left association, so

$$\begin{aligned} a + b + c &= (a + b) + c, \\ abc &= (ab)c. \end{aligned}$$

The grammar above is the core syntax. Most regular expression libraries also offer one-or-more, optional, and bounded repetition (r^+ , $r?$, $r^{\{m,n\}}$), but these add no expressive power since each can be defined by the core constructs: $r^+ = r \cdot r^*$, $r? = r + \varepsilon$, and

$$r^{\{m,n\}} = \underbrace{r \cdot \dots \cdot r}_{m \text{ times}} \cdot \underbrace{(r + \varepsilon) \cdot \dots \cdot (r + \varepsilon)}_{n-m \text{ times}}.$$

In the implementation, expressions using such operators are translated into the core constructs before they reach the parsing algorithms.

As a running example, consider the regular expression over $\Sigma = \{0, 1\}$

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1. \quad (2.2)$$

By the conventions above, its syntax tree is $((0 + 1)^* \cdot 0) \cdot 1$. This tree represents the hierarchical structure of the expression: a Kleene star applied to the alternation $0 + 1$, followed by concatenation with the literals 0 and 1 .

The syntactic structure must be given a semantic interpretation.

2.2 Semantics

A semantic interpretation assigns a language to each regular expression. A *word* over Σ is a finite sequence of symbols from Σ . The set of all words over Σ , including the empty word ε , is denoted by Σ^* . A *language* over Σ is a subset of Σ^* .

For a regular expression r , we write $L(r)$ for the language it denotes, where $L(r) \subseteq \Sigma^*$. The definition follows the structure of the syntax tree by *structural recursion*. Each case corresponds to one production of the grammar in (2.1):

$$\begin{aligned}
 L(x) &= \{x\}, \\
 L(\varepsilon) &= \{\varepsilon\}, \\
 L(\emptyset) &= \emptyset, \\
 L(r + s) &= L(r) \cup L(s), \\
 L(r \cdot s) &= \{vw \mid v \in L(r), w \in L(s)\}, \\
 L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid n \geq 1, w_1, \dots, w_n \in L(r)\}.
 \end{aligned} \tag{2.3}$$

Table 2.1 applies these definitions to small expressions. The literal a denotes the language containing only the word a . The expression ε denotes the language containing only the empty word, while $\emptyset$ denotes the language containing no words. The alternation $a + b$ combines the languages of a and b to give $\{a, b\}$, and concatenation ab combines words from the languages of a and b to give $\{ab\}$. The Kleene star a^* allows zero or more concatenations of words from the language of a , giving $\{\varepsilon, a, aa, aaa, \dots\}$.

Expression r	Language $L(r)$
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\emptyset$
$a + b$	$\{a, b\}$
ab	$\{ab\}$
a^*	$\{\varepsilon, a, aa, aaa, \dots\}$

Table 2.1: Languages denoted by basic expressions.

For example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2), we compute its language step by step:

$$\begin{aligned}
 L(0) &= \{0\}, \\
 L(1) &= \{1\}, \\
 L(0 + 1) &= L(0) \cup L(1) = \{0, 1\}, \\
 L((0 + 1)^*) &= \{0, 1\}^*, \\
 L(r_{01}) &= L((0 + 1)^*) \cdot L(0) \cdot L(1) = \{w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^*\}.
 \end{aligned} \tag{2.4}$$

This is the set of binary words ending in 01. For example, the words 01, 001, and 1101 belong to $L(r_{01})$, whereas ε , 0, and 010 do not.

This is the question of whether a word *matches* a regular expression: w *matches* r if $w \in L(r)$. Determining this is the *membership problem*.

2.3 Rust Representation

The grammar from (2.1) is represented directly as a Rust enum with one variant for each syntactic construct, as shown in Figure 2.1:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi, Eps, Lit(char),
    Seq(Box<Regex>, Box<Regex>), // r . s
    Alt(Box<Regex>, Box<Regex>), // r + s
    Star(Box<Regex>),           // r*
}

impl Regex {
    pub fn seq(r: Regex, s: Regex) -> Regex {
        Regex::Seq(Box::new(r), Box::new(s))
    }
    pub fn alt(r: Regex, s: Regex) -> Regex {
        Regex::Alt(Box::new(r), Box::new(s))
    }
    pub fn star(r: Regex) -> Regex { Regex::Star(Box::new(r)) }
    pub fn lit(c: char) -> Regex { Regex::Lit(c) }
}
```

Figure 2.1: The `Regex` enum and constructors. Derived traits support operations needed by parsing: `Clone` for duplication, `PartialEq`/`Eq` for comparison, `Hash` for collections.

The type carries no precedence or associativity information, as these are determined by the syntax tree. Recursive variants use `Box<Regex>` because a direct definition `Seq(Regex, Regex)` would have infinite size and `Box` stores these recursive values separately, providing finite size [The Rust Project Developers, 2026a,b]. This differs from Haskell’s heap-allocated data types. Rust requires explicit indirection since the enum sizes must be known at compile time.

Constructors encapsulate the boxing and are purely structural: they perform no simplification, and `Regex::seq(Regex::Eps, r)` yields a `Seq` node, not r . They take arguments by value, moving subexpressions into boxes.

The running example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2) is represented by the syntax tree:

$$\text{Seq}(\text{Seq}(\text{Star}(\text{Alt}(0, 1)), 0), 1). \quad (2.5)$$

In Rust, this is represented as shown in Figure 2.2, with the outermost `Seq` containing `Lit('1')` and `Seq(Star(Alt(0,1)), 0)` as its right and left children.

```
Regex::seq(
    Regex::seq(
        Regex::star(Regex::alt(Regex::lit('0'), Regex::lit('1'))),
        Regex::lit('0')),
    Regex::lit('1'))
```

Figure 2.2: Rust representation of r_{01} .

3 Derivatives and Partial Derivatives

Chapter 2 introduced the syntax and semantics of regular expressions, and defined the *membership problem*: Decide whether a word w belongs to $L(r)$. This chapter introduces three algorithms for solving it, each returning a Boolean answer: match or no match.

The first is a naive recursive matcher that follows the structure of the regular-expression semantics. The other two transform the regular expression as input symbols are consumed. Brzowski derivatives transform it into a single residual expression, while Antimirov's partial derivatives represent the possible residual expressions as a set.

3.1 Naive Matching

A direct way to decide whether $w \in L(r)$ is to interpret the regular-expression semantics as recursive matching rules, by following the structure of the expression and searching for possible decompositions of the input. Let $w[i : j]$ denote the substring from position i to j . A recursive procedure $\text{match}(w, i, j, r)$ determines whether $w[i : j] \in L(r)$:

$$\begin{aligned}
 \text{match}(w, i, j, \epsilon) &\iff i = j, \\
 \text{match}(w, i, j, x) &\iff j = i + 1 \wedge w[i] = x, \\
 \text{match}(w, i, j, r + s) &\iff \text{match}(w, i, j, r) \vee \text{match}(w, i, j, s), \\
 \text{match}(w, i, j, r \cdot s) &\iff \exists k \in [i, j]. \text{match}(w, i, k, r) \wedge \text{match}(w, k, j, s), \\
 \text{match}(w, i, j, r^*) &\iff i = j \vee \exists k \in (i, j]. \text{match}(w, i, k, r) \wedge \text{match}(w, k, j, r^*).
 \end{aligned} \tag{3.1}$$

Each rule corresponds directly to the semantics. Concatenation does not specify where to split, so the matcher must consider all possible split points k .

For matching $(a \cdot a) \cdot a$ against **aaa**, the split points $k \in \{0, 1, 2, 3\}$ are:

$$\begin{aligned}
 &\underbrace{\text{match}(w, 0, 0, a \cdot a)}_{a \cdot a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 3, a)}_{a \text{ against } w[0:3]=\mathbf{aaa}} \\
 \vee &\underbrace{\text{match}(w, 0, 1, a \cdot a)}_{a \cdot a \text{ against } w[0:1]=\mathbf{a}} \wedge \underbrace{\text{match}(w, 1, 3, a)}_{a \text{ against } w[1:3]=\mathbf{aa}} \\
 \vee &\underbrace{\text{match}(w, 0, 2, a \cdot a)}_{a \cdot a \text{ against } w[0:2]=\mathbf{aa}} \wedge \underbrace{\text{match}(w, 2, 3, a)}_{a \text{ against } w[2:3]=\mathbf{a}} \\
 \vee &\underbrace{\text{match}(w, 0, 3, a \cdot a)}_{a \cdot a \text{ against } w[0:3]=\mathbf{aaa}} \wedge \underbrace{\text{match}(w, 3, 3, a)}_{a \text{ against } w[3:3]=\epsilon}.
 \end{aligned} \tag{3.2}$$

At $k = 2$, both operands match, triggering a recursive search on $(a \cdot a)$ against **aa** with split points $k \in \{0, 1, 2\}$, where $k = 1$ succeeds:

$$\begin{aligned}
& \underbrace{\text{match}(w, 0, 0, a)}_{a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\mathbf{aa}} \\
\vee & \underbrace{\text{match}(w, 0, 1, a)}_{a \text{ against } w[0:1]=\mathbf{a}} \wedge \underbrace{\text{match}(w, 1, 2, a)}_{a \text{ against } w[1:2]=\mathbf{a}} \\
\vee & \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\mathbf{aa}} \wedge \underbrace{\text{match}(w, 2, 2, a)}_{a \text{ against } w[2:2]=\epsilon}.
\end{aligned} \tag{3.3}$$

This completes the original match through the nested split points $k = 2$ and $k = 1$.

In Rust. The recursive rules translate directly into the corresponding Rust match arms, shown in Figure 3.1. Alternation tries both branches on the same input, while concatenation and star search possible split points. The star arm clones the body to construct the recursive **Star** expression. This is necessary because the recursive call needs to own the newly constructed expression. The range-based search short-circuits at the first successful split. Nested concatenations repeatedly search smaller substrings, exploring multiple paths before succeeding. This leads to exponential backtracking in the worst case.

```

Regex::Alt(r, s) => {
    match_naive_range(word, i, j, r)
    || match_naive_range(word, i, j, s)
}
Regex::Seq(r, s) => {
    (i..=j).any(|k| {
        match_naive_range(word, i, k, r)
        && match_naive_range(word, k, j, s)
    })
}
Regex::Star(r) => {
    i == j || (i + 1..=j).any(|k| {
        match_naive_range(word, i, k, r)
        && match_naive_range(word, k, j,
            &Regex::star(*r.clone()))
    })
}

```

Figure 3.1: Alternation, concatenation, and star arms of the naive matcher.

Although inefficient, this matcher is useful as a reference implementation because it follows the regular-expression semantics directly without any intermediate transformation. The cost is visible in the worked examples: nested concatenations repeatedly search smaller substrings, each exploring all possible split points before finding a match.

The derivative approach takes a different approach. Rather than searching for a decomposition of w while keeping r fixed, it transforms r as each character of w is consumed. The resulting expression describes the language that can still match the remaining suffix.

3.2 Brzowski Derivatives

Brzowski's derivative of r with respect to a symbol x , written $r \backslash x$, is the expression describing exactly what remains of r 's language after consuming the leading symbol x of a word $w \in L(r)$ [Brzowski, 1964]:

$$L(r \backslash x) = \{w \mid xw \in L(r)\}. \quad (3.4)$$

For $w = x_1x_2 \cdots x_n$, consume one symbol at a time by repeatedly taking derivatives:

$$r, \quad r \backslash x_1, \quad (r \backslash x_1) \backslash x_2, \quad \dots$$

Nullability. After all symbols have been consumed, the remaining input is ε . Thus, the resulting expression must accept ε for the original word to belong to $L(r)$. This is the question of whether the expression is *nullable*. Let $n(r)$ be true exactly when $\varepsilon \in L(r)$:

$$\begin{array}{llll} n(\emptyset) = \text{false} & (\mathbf{N}_\emptyset) & n(r + s) = n(r) \vee n(s) & (\mathbf{N}_+) \\ n(\varepsilon) = \text{true} & (\mathbf{N}_\varepsilon) & n(r \cdot s) = n(r) \wedge n(s) & (\mathbf{N}_{\text{seq}}) \\ n(x) = \text{false} & (\mathbf{N}_{\text{lit}}) & n(r^*) = \text{true} & (\mathbf{N}_*) \end{array} \quad (3.5)$$

The empty language $\emptyset$ is not nullable because it contains no words, including ε , while a literal cannot match the empty word. ε and r^* always match ε . An alternation is nullable when either branch is nullable, and a concatenation only when both components are.

In Rust. The definition is transcribed one `Regex` variant at a time, using the enum of Figure 2.1. The case split of (3.5) becomes the arms of a `match`, shown in Figure 3.2.

```
pub fn nullable(r: &Regex) -> bool {
    match r {
        Regex::Phi => false,
        Regex::Eps => true,
        Regex::Lit(_) => false,
        Regex::Alt(r, s) => nullable(r) || nullable(s),
        Regex::Seq(r, s) => nullable(r) && nullable(s),
        Regex::Star(_) => true,
    }
}
```

Figure 3.2: `nullable` implementation and its six cases: $\mathbf{N}_\emptyset$, $\mathbf{N}_\varepsilon$, $\mathbf{N}_{\text{lit}}$, $\mathbf{N}_+$, $\mathbf{N}_{\text{seq}}$ and $\mathbf{N}_*$.

The expression is passed by reference because `nullable` only inspects r and does not modify or consume it. The underscores in $\mathbf{N}_{\text{lit}}$ and $\mathbf{N}_*$ indicate that the corresponding operands are irrelevant. A literal is never nullable, regardless of its symbol, while a Kleene-star expression is always nullable, regardless of its body. The short-circuiting operators `||` and `&&` avoid evaluating the second operand when the result is already determined, while preserving the corresponding logical semantics.

Derivative Rules. The rules specify the expression that remains after consuming x :

$$\begin{array}{llll}
\emptyset \backslash x = \emptyset & (D_{\emptyset}) & (r + s) \backslash x = (r \backslash x) + (s \backslash x) & (D_{+}) \\
\varepsilon \backslash x = \emptyset & (D_{\varepsilon}) & r^* \backslash x = (r \backslash x) \cdot r^* & (D_{*}) \\
y \backslash x = \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases} & (D_{\text{lit}}) & (r \cdot s) \backslash x = \begin{cases} (r \backslash x) \cdot s + s \backslash x & n(r) = \text{true} \\ (r \backslash x) \cdot s & n(r) = \text{false} \end{cases} & \begin{matrix} (D_{\text{seq}}^n) \\ (D_{\text{seq}}) \end{matrix}
\end{array} \tag{3.6}$$

The empty language and ε cannot consume x . A literal y succeeds exactly when $x = y$, leaving ε . Alternation retains both derivatives and for r^* , one occurrence of r consumes x while further occurrences remain available.

Concatenation is the only rule that depends on nullability. If r is not nullable, r must consume the leading x , leaving $r \backslash x$ as the remainder of r while s remains untouched. If r is nullable, it may either consume x giving $(r \backslash x) \cdot s$, or r matches ε and s consumes x leaving $s \backslash x$. Hence the nullable case produces the alternation $(r \backslash x) \cdot s + s \backslash x$.

In Rust. Figure 3.3 implements (3.6) similarly, one variant at a time.

```

pub fn deriv(r: &Regex, x: char) -> Regex {
    match r {
        Regex::Phi => Regex::Phi,
        Regex::Eps => Regex::Phi,
        Regex::Lit(c) => if *c == x { Regex::Eps } else { Regex::Phi },
        Regex::Alt(r1, r2) =>
            Regex::alt(deriv(r1, x), deriv(r2, x)),
        Regex::Seq(r1, r2) => {
            let dr1 = Regex::seq(deriv(r1, x), *r2.clone());
            if nullable(r1) {
                Regex::alt(dr1, deriv(r2, x))
            } else { dr1 }
        }
        Regex::Star(r1) =>
            Regex::seq(deriv(r1, x), Regex::star(*r1.clone())),
    }
}

```

Figure 3.3: `deriv` implementation with six cases.

`deriv` differs from `nullable` in that it borrows its input but returns an owned `Regex`, since a derivative is a new expression. The `if nullable(r1)` implements the case split of (3.6), with branches in order D_{seq}^n then D_{seq} . This is the only place where a derivative rule consults nullability and where the output can be larger than the input, since the nullable branch produces an alternation while the other produces a concatenation.

The cloning in `*r2.clone()` is necessary because s appears untouched in both branches of the concatenation case, and the star arm must preserve r^* 's body both derived and un-derived. This copying mechanism drives expression growth, which justifies simplification after every derivative step.

Example: Growth without simplification. Consider $(x + y)^*$ against the word $\mathbf{xx}$:
Step 1. The expression is a star, so D_* applies with body $r = x + y$:

$$\begin{aligned} (x + y)^* \backslash \mathbf{x} &= ((x + y) \backslash \mathbf{x}) \cdot (x + y)^* & D_*, r = x + y \\ &= (\varepsilon + \emptyset) \cdot (x + y)^* & D_+, D_{\text{lit}} \end{aligned} \quad (3.7)$$

Step 2. The expression is now the concatenation $(\varepsilon + \emptyset) \cdot (x + y)^*$, and deriving it consumes the second $\mathbf{x}$. The result is an alternation because D_{seq}^n applies with $r = \varepsilon + \emptyset$ and $s = (x + y)^*$, its nullable case being selected since $n(r) = n(\varepsilon + \emptyset) = \text{true}$ by N_+ and N_ε :

$$\begin{aligned} &((\varepsilon + \emptyset) \cdot (x + y)^*) \backslash \mathbf{x} \\ &= ((\varepsilon + \emptyset) \backslash \mathbf{x}) \cdot (x + y)^* + (x + y)^* \backslash \mathbf{x} & D_{\text{seq}}^n \\ &= (\emptyset + \emptyset) \cdot (x + y)^* + (x + y)^* \backslash \mathbf{x} & D_\varepsilon \\ &= (\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^* & D_*, D_+, D_{\text{lit}} \end{aligned} \quad (3.8)$$

After both symbols are consumed, the result is checked for nullability:

$$\begin{aligned} &n((\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^*) \\ &= n((\emptyset + \emptyset) \cdot (x + y)^*) \vee n((\varepsilon + \emptyset) \cdot (x + y)^*) & N_+ \\ &= (n(\emptyset + \emptyset) \wedge n((x + y)^*)) \vee (n(\varepsilon + \emptyset) \wedge n((x + y)^*)) & N_{\text{seq}} \\ &= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) & N_+, N_\emptyset, N_\varepsilon, N_* \\ &= \text{true}. \end{aligned} \quad (3.9)$$

Thus $(x + y)^*$ matches $\mathbf{xx}$.

Simplification. Derivatives can grow unboundedly. Repeatedly deriving $(x + y)^*$ by the same symbol produces expressions of increasing size. This growth results from the nullable branch of D_{seq}^n , which introduces an alternation as a redundant structure that is carried into subsequent derivatives. ε and $\emptyset$ accumulate without bound:

$$\begin{array}{ll} (x + y)^* & 4 \\ \xrightarrow{x} (\varepsilon + \emptyset) \cdot (x + y)^* & 8 \\ \xrightarrow{x} (\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^* & 17 \\ \xrightarrow{x} (\emptyset + \emptyset) \cdot (x + y)^* + ((\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^*) & 26 \end{array} \quad (3.10)$$

To counter the growth, expressions are simplified after each derivative step, using identities that preserve the denoted language:

$$\begin{aligned} \emptyset \cdot r &= \emptyset & (S_1) \\ \varepsilon \cdot r &= r & (S_2) \\ \emptyset + r &= r & (S_3) \end{aligned} \quad (3.11)$$

S_1 eliminates dead concatenations with $\emptyset$, S_2 eliminates identity concatenations with ε , and S_3 eliminates $\emptyset$ from alternations. Each holds equally with the unit on the right.

Applying them to (3.8), after each step, returns the expression back to $(x + y)^*$, keeping the size bounded regardless of input length:

$$\begin{aligned}
& (\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^* \\
&= \emptyset \cdot (x + y)^* + \varepsilon \cdot (x + y)^* && S_3, \text{ twice} \\
&= \emptyset + (x + y)^* && S_1, S_2 \\
&= (x + y)^* && S_3
\end{aligned} \tag{3.12}$$

In Rust. The identities are implemented recursively, applying the rules bottom-up over the expression tree. Each arm of the inner match implements an identity from (3.11).

`simplify` takes ownership of its input and returns simplified subexpressions directly, unlike `deriv`, which borrows and constructs. This model is visible where the two compose:

```
current = simplify(deriv(&current, c));
```

`deriv` borrows `current`, constructs a new expression, and `simplify` takes ownership of that result to return it without additional copying. The recursive calls ensure that all dead code is eliminated before an expression is returned, keeping size bounded at each step. Together, derive-then-simplify forms the core loop of the matcher, transforming and cleaning the expression at every input symbol.

```
pub fn simplify(r: Regex) -> Regex {
  match r {
    Regex::Seq(r, s) => {
      let r = simplify(*r);
      let s = simplify(*s);
      match (&r, &s) {
        (Regex::Phi, _) | (_, Regex::Phi) => Regex::Phi,
        (Regex::Eps, _) => s,
        (_, Regex::Eps) => r,
        _ => Regex::seq(r, s),
      }
    }
    Regex::Alt(r, s) => {
      let r = simplify(*r);
      let s = simplify(*s);
      match (&r, &s) {
        (Regex::Phi, _) => s,
        (_, Regex::Phi) => r,
        _ if r == s => r,
        _ => Regex::alt(r, s),
      }
    }
    Regex::Star(r) => match simplify(*r) {
      Regex::Eps | Regex::Phi => Regex::Eps,
      r => Regex::star(r),
    },
    other => other,
  }
}
```

Figure 3.4: Simplification rules: S_1 and S_2 in concatenation, S_3 in both branches.

Running example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$. Recall from (2.2) that it is left-associated as $((0 + 1)^* \cdot 0) \cdot 1$, so the outer operands are $r = (0 + 1)^* \cdot 0$ and $s = 1$. We derive against $w = 101$ one symbol at a time.

1. **Derive by 1.** At the outer node, $r = (0 + 1)^* \cdot 0$ and $s = 1$. This r is not nullable:

$$n((0 + 1)^* \cdot 0) = \text{true} \wedge \text{false} = \text{false},$$

by $\mathbf{N}_{\text{seq}}$, $\mathbf{N}_*$ and $\mathbf{N}_{\text{lit}}$. Thus $\mathbf{D}_{\text{seq}}$ applies without a second summand:

$$r_{01} \setminus 1 = (((0 + 1)^* \cdot 0) \setminus 1) \cdot 1.$$

At the middle node, the left operand $(0 + 1)^*$ is nullable, so $\mathbf{D}_{\text{seq}}^n$ applies. First,

$$\begin{aligned} (0 + 1)^* \setminus 1 &= ((0 + 1) \setminus 1) \cdot (0 + 1)^* && \mathbf{D}_* \\ &= ((0 \setminus 1) + (1 \setminus 1)) \cdot (0 + 1)^* && \mathbf{D}_+ \\ &= (\emptyset + \varepsilon) \cdot (0 + 1)^* && \mathbf{D}_{\text{lit}} \\ &= (0 + 1)^* && \mathbf{S}_3, \mathbf{S}_2. \end{aligned}$$

Hence

$$\begin{aligned} ((0 + 1)^* \cdot 0) \setminus 1 &= (0 + 1)^* \cdot 0 + (0 \setminus 1) && \mathbf{D}_{\text{seq}}^n \\ &= (0 + 1)^* \cdot 0 + \emptyset && \mathbf{D}_{\text{lit}} \\ &= (0 + 1)^* \cdot 0 && \mathbf{S}_3. \end{aligned}$$

Substituting back into the outer node gives

$$r_{01} \setminus 1 = ((0 + 1)^* \cdot 0) \cdot 1 = r_{01}. \quad (3.13)$$

Thus a 1 not yet part of the final 01 is absorbed by the leading star.

2. **Derive by 0.** The expression is unchanged. Again, $(0 + 1)^* \setminus 0$ simplifies to $(0 + 1)^*$, but now the second summand at the middle node survives:

$$\begin{aligned} ((0 + 1)^* \cdot 0) \setminus 0 &= (0 + 1)^* \cdot 0 + (0 \setminus 0) && \mathbf{D}_{\text{seq}}^n \\ &= (0 + 1)^* \cdot 0 + \varepsilon && \mathbf{D}_{\text{lit}}. \end{aligned}$$

No simplification applies. The outer node is still non-nullable, so

$$r_{01} \setminus 1 \setminus 0 = ((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1. \quad (3.14)$$

After 10, either the trailing 01 is still owed in full, represented by $(0 + 1)^* \cdot 0$, or the consumed 0 was already its first symbol, represented by ε .

3. **Derive by 1.** The outer binding has now changed. The left operand is $(0 + 1)^* \cdot 0 + \varepsilon$, which is nullable by $\mathbf{N}_+$ and $\mathbf{N}_\varepsilon$. So $\mathbf{D}_{\text{seq}}^n$ applies and contributes the second summand $1 \setminus 1 = \varepsilon$. For the first summand, $\mathbf{D}_+$ applies, reusing the result from step 1:

$$\begin{aligned} ((0 + 1)^* \cdot 0 + \varepsilon) \setminus 1 &= ((0 + 1)^* \cdot 0) \setminus 1 + (\varepsilon \setminus 1) && \mathbf{D}_+ \\ &= (0 + 1)^* \cdot 0 + \emptyset && \text{step 1; } \mathbf{D}_\varepsilon \\ &= (0 + 1)^* \cdot 0 && \mathbf{S}_3. \end{aligned}$$

Therefore

$$r_{01} \setminus 1 \setminus 0 \setminus 1 = ((0 + 1)^* \cdot 0) \cdot 1 + \varepsilon = r_{01} + \varepsilon. \quad (3.15)$$

The final expression is nullable by $\mathbf{N}_+$ and $\mathbf{N}_\varepsilon$, so $101 \in L(r_{01})$. The ε summand witnesses that the match is complete.

For comparison, $w = 10$ leaves $((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1$. Neither branch is nullable: $(0 + 1)^* \cdot 0$ still requires a 0, while $\varepsilon \cdot 1$ still requires a 1. Hence $10 \notin L(r_{01})$.

The matcher. The implementation can be summarized by the sequence of residual expressions produced while consuming the input. For $w = x_1x_2 \cdots x_n$, the initial residual is $r_0 = r$, and each subsequent residual is obtained by deriving with respect to the next symbol and simplifying the result:

$$r_0 = r, \quad r_i = \text{simplify}(r_{i-1} \backslash x_i) \quad (1 \leq i \leq n). \quad (3.16)$$

The residual r_i denotes exactly the set of suffixes that complete the consumed prefix $x_1 \cdots x_i$ to a word in $L(r)$. Since simplification preserves the represented language, each r_i is equivalent to the unsimplified derivative after consuming that prefix. After the final symbol has been consumed, the matcher accepts when the remaining residual is nullable:

$$w \in L(r) \iff n(r_n) = \text{true}.$$

So $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ can be viewed as a path through a lazily constructed DFA. Each residual expression represents a state, and $\xrightarrow{x}$ denotes the transition obtained by taking the derivative with respect to x . Unlike an explicitly constructed DFA, only the states reached by the input are constructed. A residual is an accepting state exactly when it is nullable, so acceptance is determined by whether the final residual is nullable.

In Rust. Figure 3.5 implements (3.16) by composing `deriv`, `simplify`, and `nullable`. The implementation follows the mathematical definition directly. The current residual is transformed at each step, and acceptance is determined after the complete input has been consumed.

```
pub fn match_deriv(input: &str, r: &Regex) -> bool {
    let mut current = r.clone();
    for c in input.chars() {
        current = simplify(deriv(&current, c));
    }
    nullable(&current)
}
```

Figure 3.5: `match_deriv`: the core matcher loop. For each input character, derive the current residual expression and simplify it in place. Accept if the final residual is nullable.

The loop implements the residual sequence. At each iteration, `current` contains the current residual, which is then derived and simplified before processing the next input symbol. Only the current residual is retained, since the previous residual is dropped when `current` is reassigned. In contrast to parsing, where intermediate results may be required to reconstruct a parse tree, Boolean matching requires only the final residual.

The derivative-based matcher avoids the exponential backtracking of naive matching by transforming the expression rather than searching for decompositions. Simplification keeps the resulting expressions compact, preventing the unbounded growth illustrated in (3.10).

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 combines alternative residuals into a single expression using $+$. Antimirov’s partial derivatives instead keep them as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set:

$$\begin{aligned}
pd(\emptyset, x) &= \{\} & (P_{\emptyset}) \\
pd(\varepsilon, x) &= \{\} & (P_{\varepsilon}) \\
pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} & (P_{\text{lit}}) \\
pd(r + s, x) &= pd(r, x) \cup pd(s, x) & (P_{+}) \\
pd(r^*, x) &= \{ smartC(r', r^*) \mid r' \in pd(r, x) \} & (P_{*}) \\
pd(r \cdot s, x) &= \begin{cases} \{ smartC(r', s) \mid r' \in pd(r, x) \} \cup pd(s, x) & n(r) = \text{true} \\ \{ smartC(r', s) \mid r' \in pd(r, x) \} & n(r) = \text{false} \end{cases} & \begin{matrix} (P_{\text{seq}}^n) \\ (P_{\text{seq}}) \end{matrix}
\end{aligned} \tag{3.17}$$

The empty language and ε produce the empty set, since neither has a residual with respect to x : where $D_{\emptyset}$ and D_{ε} returned $\emptyset$ as a node in the result, $P_{\emptyset}$ and P_{ε} return no residual at all. A literal y gives $\{\varepsilon\}$ when $x = y$ and $\{\}$ otherwise. Alternation unions the two residual sets, and for r^* one iteration of r consumes x , with each resulting residual placed in front of r^* so the star may continue. None of these alternatives are combined with $+$; each stays a separate element of the set.

Concatenation depends on nullability for the same reason as D_{seq} . If r is not nullable it must consume x , and each of its residuals is concatenated with the untouched s . If r is nullable, r may instead match ε and let s consume x , in which case $pd(s, x)$ joins the set. Where D_{seq}^n combined the two cases with $+$ inside one expression, P_{seq}^n keeps them as two elements of one set.

In Rust. Figure 3.6 presents the implementation of (3.17), with each `match` arm corresponding to one of its cases. The `Phi`, `Eps`, and `Lit` variants implement $P_{\emptyset}$, P_{ε} , and P_{lit} directly, returning `Vec::new()` where the equation gives $\{\}$. `Alt` concatenates the two child vectors, implementing the union of P_{+} . `Star` maps `smart_seq` over r ’s own residuals, matching P_{*} ; `Regex::star(*r.clone())` clones because `Regex::star` needs an owned `Regex` and `r` is only borrowed, once per residual since the closure runs once for each.

`Seq` implements the case split directly. The `map` applies `smart_seq` to each residual of the left operand, matching P_{seq} , and the guarded `out.extend(pderiv(s, x))` adds $pd(s, x)$ for P_{seq}^n . Although the mathematical definition uses sets, the implementation uses `Vec<Regex>` throughout the recursion and defers deduplication to a single `HashSet` insertion at the top, rather than deduplicating at every `Alt` or `Seq` node on the way down.

Every union in `pderiv` is therefore a vector `extend`, not a set operation. Duplicate residuals reached by different strands are only merged once inserted into the `HashSet` used by the matcher below to keep the residual generation separate from deduplication.

```

fn pderiv(r: &Regex, x: char) -> Vec<Regex> {
  match r {
    Regex::Phi => Vec::new(),
    Regex::Eps => Vec::new(),
    Regex::Lit(c) => {
      if *c == x { vec![Regex::Eps] } else { Vec::new() }
    }
    Regex::Alt(r, s) => {
      let mut out = pderiv(r, x);
      out.extend(pderiv(s, x));
      out
    }
    Regex::Seq(r, s) => {
      let mut out: Vec<Regex> = pderiv(r, x)
        .into_iter()
        .map(|r_prime| smart_seq(r_prime, s))
        .collect();
      if nullable(r) {
        out.extend(pderiv(s, x));
      }
      out
    }
    Regex::Star(r) => {
      pderiv(r, x)
        .into_iter()
        .map(|r_prime| smart_seq(r_prime, &Regex::star(*r.clone())))
        .collect()
    }
  }
}

```

Figure 3.6: $pd(r, x)$ in full, corresponding arm for arm to Equation (3.17).

The smart constructor. The concatenation in P_{seq} is constructed using the auxiliary constructor *smartC*, denoted by SC:

$$smartC(r', s) = \begin{cases} s & r' = \varepsilon \\ r' \cdot s & \text{otherwise} \end{cases} \quad (3.18)$$

If $r' = \varepsilon$, the consumed symbol completed the left operand and there is nothing left to concatenate with s , so the residual is s alone. Otherwise r' stays in front of s . *smartC* therefore states the concatenation rule without ever constructing $\varepsilon \cdot s$.

A set element is one possible residual, not one branch of a sum. When several alternatives can consume the same symbol, they stay as separate elements, and *smartC* is applied to each independently. One strand may finish the left operand while another still carries a non-empty residual in front of s . The constructor is therefore applied independently to each residual produced by $pd(r, x)$, preserving the distinct continuations of the concatenation without introducing redundant $\varepsilon \cdot s$ expressions.

In Rust. Figure 3.7 is *smartC*.

```
pub fn smart_seq(r: Regex, s: &Regex) -> Regex {
    match r {
        Regex::Eps => s.clone(),
        r => Regex::seq(r, s.clone()),
    }
}
```

Figure 3.7: `smart_seq`, the smart constructor *smartC*

`smart_seq` takes `r` by value and `s` by reference, matching (3.18). The residual r' comes from the recursive derivative and is consumed, while s must survive into the result. The $r' = \varepsilon$ branch clones only `s` and drops ε , so $\varepsilon \cdot s$ is never built.

Example: No growth under *pd*. Consider $(x + y)^*$ against `xx` under *pd*.

Step 1. P_* applies with star body $r = x + y$:

$$\begin{aligned} pd((x + y)^*, \mathbf{x}) &= \{ smartC(r', (x + y)^*) \mid r' \in pd(x + y, \mathbf{x}) \} & P_* \\ &= \{ smartC(r', (x + y)^*) \mid r' \in \{\varepsilon\} \} & P_+, P_{lit} \text{ twice} \\ &= \{(x + y)^*\} & SC, r' = \varepsilon. \end{aligned} \quad (3.19)$$

$pd(x + y, \mathbf{x}) = \{\varepsilon\} \cup \{\} = \{\varepsilon\}$ by P_+ , since `x` matches the left literal and misses the right, and *smartC* absorbs the ε straight back into $(x + y)^*$.

Step 2. The set is $\{(x + y)^*\}$, so deriving by the second `x` repeats the computation:

$$pd((x + y)^*, \mathbf{x}, \mathbf{x}) = \{(x + y)^*\}.$$

Since $(x + y)^*$ is nullable by N_* , the final partial derivative accepts ε , so `xx` $\in L((x + y)^*)$.

For comparison, applying the Brzozowski derivative to the same expression gives the intermediate alternation in (3.8), $(\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^*$. Only after simplification in (3.12) does this reduce to $(x + y)^*$. Under *pd*, the residual set is $\{(x + y)^*\}$ after both steps, since $P_\emptyset$ and P_ε produce no residual at all. So redundant terms are avoided at construction time rather than built and simplified afterwards.

The next example shows where more than one residual survives at once.

Running example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$. Derive r_{01} against $w = 101$, tracking the residual set. The concatenations bind as in Section 3.2, with $(r, s) = ((0 + 1)^* \cdot 0, 1)$ at the outer node and $(r, s) = ((0 + 1)^*, 0)$ at the inner node; the star body is $0 + 1$. The derivation proceeds from the inside outward.

1. **Derive by 1.** At the star body, P_+ distributes the derivative over two alternatives:

$$\begin{aligned} pd(0+1, 1) &= pd(0, 1) \cup pd(1, 1) & P_+, r = 0, s = 1 \\ &= \{\} \cup \{\varepsilon\} & P_{lit}, \text{ both branches} \\ &= \{\varepsilon\}. \end{aligned} \quad (3.20)$$

The star rule then applies *smartC* to the resulting residual:

$$\begin{aligned} pd((0+1)^*, 1) &= \{ smartC(r', (0+1)^*) \mid r' \in \{\varepsilon\} \} & P_*, r = 0+1 \\ &= \{(0+1)^*\} & SC, r' = \varepsilon. \end{aligned} \quad (3.21)$$

At the middle concatenation, the left operand is nullable by N_* , so the nullable case contributes both the derivatives through the left and right operand:

$$\begin{aligned} pd((0+1)^* \cdot 0, 1) &= \{ smartC(r', 0) \mid r' \in pd((0+1)^*, 1) \} \cup pd(0, 1) & P_{seq}^n \\ &= \{ smartC((0+1)^*, 0) \} \cup \{ \} & \text{above; } P_{lit} \\ &= \{(0+1)^* \cdot 0\} & SC, r' \neq \varepsilon. \end{aligned} \quad (3.22)$$

At the outer node, the left operand is not nullable, so only one branch contributes:

$$\begin{aligned} pd(r_{01}, 1) &= \{ smartC(r', 1) \mid r' \in \{(0+1)^* \cdot 0\} \} & P_{seq}, r = (0+1)^* \cdot 0, s = 1 \\ &= \{(0+1)^* \cdot 0 \cdot 1\} = \{r_{01}\} & SC. \end{aligned} \quad (3.23)$$

The first symbol leaves $\{r_{01}\}$, returning to the starting expression since the consumed 1 belongs to the leading star.

2. **Derive by 0.** The current set is still $\{r_{01}\}$. As before, $pd((0+1)^*, 0) = \{(0+1)^*\}$. At the middle concatenation, however, the right operand is now the matching literal 0, so P_{lit} contributes $\{\varepsilon\}$:

$$\begin{aligned} pd((0+1)^* \cdot 0, 0) &= \{ smartC((0+1)^*, 0) \} \cup pd(0, 0) & P_{seq}^n, r = (0+1)^*, s = 0 \\ &= \{(0+1)^* \cdot 0\} \cup \{\varepsilon\} & P_{lit}, \text{ since } 0 = 0 \\ &= \{(0+1)^* \cdot 0, \varepsilon\}. \end{aligned} \quad (3.24)$$

The outer concatenation applies *smartC* separately to both residuals:

$$\begin{aligned} pd(r_{01}, 1, 0) &= \{ smartC(r', 1) \mid r' \in \{(0+1)^* \cdot 0, \varepsilon\} \} & P_{seq}, s = 1 \\ &= \{(0+1)^* \cdot 0 \cdot 1, 1\} & SC \\ &= \{r_{01}, 1\}. \end{aligned} \quad (3.25)$$

The two residuals are the possible strands after 10: r_{01} keeps both symbols in the leading star, while 1 means the 0 began the final 01. Since $smartC(\varepsilon, 1) = 1$, the latter is represented directly as 1.

3. **Derive by 1.** The two residuals are derived independently and their results are unioned:

$$\begin{aligned} pd(r_{01}, 1, 0, 1) &= pd(r_{01}, 1) \cup pd(1, 1) & \text{union over the set} \\ &= \{r_{01}\} \cup \{\varepsilon\} & \text{step 1; } P_{lit} \\ &= \{r_{01}, \varepsilon\}. \end{aligned} \quad (3.26)$$

The final set contains ε , so one surviving residual is nullable and $101 \in L(r_{01})$. This agrees with the conclusion reached by the Brzozowski derivative in (3.15).

For $w = 10$, the first two steps leave $\{r_{01}, 1\}$, which represents the same alternatives as $((0+1)^* \cdot 0 + \varepsilon) \cdot 1$ in (3.14). Either the trailing 01 remains to be matched, or the consumed 0 was its first symbol. Neither residual is nullable, so $10 \notin L(r_{01})$.

Finiteness. For a fixed regular expression r , the set of reachable partial derivatives is finite and bounded by $\|r\| + 1$, where $\|r\|$ denotes the number of literal occurrences in r [Antimirov, 1996, Theorem 3.4, p. 309]. For r_{01} , $\|r_{01}\| = 4$ (the four literals: 0, 1, 0, 1), so the bound is $4 + 1 = 5$ distinct residuals across all possible inputs. The example with $w = 101$ reaches only three of them: $\{r_{01}\}$, $\{r_{01}, 1\}$, and $\{r_{01}, \varepsilon\}$. Other residuals in the theoretical set are reachable only by different input sequences.

For ordinary derivatives, finiteness requires identifying equivalent expressions [Brzozowski, 1964, Theorem 4.3], yielding an exponential bound $d_R \leq 2^{d_P} - 1$. Simplification rules eliminate only units and redundant $\emptyset$ expressions, not all equivalent forms. The linear bound for partial derivatives applies directly without simplification.

The matcher. A *frontier* of residuals is updated one symbol at a time by taking pd of every element and unioning the results. The frontier F_i represents all residuals reachable after consuming the first i input symbols—every possible state the computation could be in. For frontier F_i after i symbols,

$$F_0 = \{r\}, \quad F_i = \bigcup_{r' \in F_{i-1}} pd(r', x_i) \quad (1 \leq i \leq n). \quad (3.27)$$

The matcher accepts when some residual in the final frontier is nullable. Where Brzozowski's $r_0 \rightarrow r_1 \rightarrow \dots \rightarrow r_n$ traces one path through a lazily constructed DFA, $F_0 \rightarrow F_1 \rightarrow \dots \rightarrow F_n$ traces all states an NFA explores simultaneously, holding each state at once rather than committing to a single residual expression.

In Rust. Figure 3.8 implements (3.27) using a `HashSet<Regex>` frontier, where `Regex`'s `Hash` and `Eq` implementations allow residuals from different paths to collapse. Where the Brzozowski matcher holds one `Regex`, this holds a set. The early return when the frontier empties follows from construction: no later derivative produces residuals from an empty set. The final `any(nullable)` checks whether some residual is nullable. The matcher trades a single growing expression for a set of small ones that never need simplification.

```
pub fn match_pderiv(input: &str, r: &Regex) -> bool {
    let mut states: HashSet<Regex> = HashSet::new();
    states.insert(r.clone());
    for c in input.chars() {
        let mut next: HashSet<Regex> = HashSet::new();
        for state in &states {
            next.extend(pderiv(state, c));
        }
        if next.is_empty() { return false; }
        states = next;
    }
    states.iter().any(nullable)
}
```

Figure 3.8: The partial-derivative matcher.

4 Matching versus Parsing

Chapter 3 established matching with a Boolean answer: whether $w \in L(r)$. A matcher therefore answers only whether the input is accepted by the regular expression. Parsing goes beyond this membership question by identifying the derivation for the match.

This distinction is important because a regular expression can admit more than one way of matching the same word. A parser must therefore represent the choices made during a match. These choices can be represented by *parse trees*, which provide structured witnesses for membership in the language.

4.1 Matching Ambiguity

Consider the regular expression

$$r = (a + ab)(b + \varepsilon)$$

and the input word

$$w = \mathbf{ab}.$$

The matcher can establish that $w \in L(r)$ using the matching procedures of Chapter 3. There are, however, two ways in which the match can be obtained. In the first, the left subexpression matches **a** and the right subexpression matches **b**:

$$\underbrace{a}_{a+ab} \quad \underbrace{b}_{b+\varepsilon}.$$

In the second, the left subexpression matches the entire input and the right subexpression matches the empty word:

$$\underbrace{\mathbf{ab}}_{a+ab} \quad \underbrace{\varepsilon}_{b+\varepsilon},$$

Both choices therefore establish the same membership result,

$$\mathbf{ab} \in L((a + ab)(b + \varepsilon)),$$

but assign the input differently to the subexpressions of r . The Boolean result of matching does not distinguish these cases.

Parsing makes this additional structure explicit. A parser must represent which alternatives were chosen and which parts of the input were consumed by the corresponding subexpressions. A *parse tree* provides a structured witness for such a derivation, recording the choices made by the expression and the corresponding consumption of the input.

4.2 Parse Trees as Matching Evidence

Sulzmann and Lu view regular expressions as types and parse trees as their values [Sulzmann and Lu, 2014]. The judgment $\vdash v : r$ states that v is a valid parse tree for r and records the structural choices made when matching r :

$$\begin{array}{c}
 \vdash () : \varepsilon \quad (\mathsf{T}_\varepsilon) \\
 \frac{l \in \Sigma}{\vdash l : l} \quad (\mathsf{T}_{\text{lit}}) \\
 \frac{\vdash v_1 : r_1 \quad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2} \quad (\mathsf{T}_{\text{seq}}) \\
 \frac{\vdash v_1 : r_1}{\vdash \text{Left } v_1 : r_1 + r_2} \quad (\mathsf{T}_+^L) \\
 \frac{\vdash v_2 : r_2}{\vdash \text{Right } v_2 : r_1 + r_2} \quad (\mathsf{T}_+^R) \\
 \frac{}{\vdash [] : r^*} \quad (\mathsf{T}_*^0) \\
 \frac{\vdash v : r \quad \vdash vs : r^*}{\vdash (v : vs) : r^*} \quad (\mathsf{T}_*)
 \end{array} \tag{4.1}$$

The rules correspond to the constructors of regular expressions. A concatenation uses a pair of parse trees, an alternation records the chosen branch with a tagged injection, and a Kleene star records its iterations as a list.

In Rust. The definitions above translate directly to Rust. Figure 4.1 shows the parse-tree type, with one variant for each constructor in (4.1). The two star rules share a single **Star** variant, represented by a **Vec**.

```

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseTree {
    Empty, // ()
    Char(char), // l
    Pair(Box<ParseTree>, Box<ParseTree>), // (v1, v2)
    Left(Box<ParseTree>), // Left v
    Right(Box<ParseTree>), // Right v
    Star(Vec<ParseTree>), // [v1, ..., vn]
}

```

Figure 4.1: The parse-tree type: The variants correspond to the constructors. The two star rules share the **Star** variant.

The correspondence is one to one except for the star case: T_ε , T_{lit} , T_{seq} , T_+^L , and T_+^R correspond to **Empty**, **Char**, **Pair**, **Left**, and **Right**, respectively. The empty and non-empty star rules are both represented by **Star**, with $[]$ represented by **Star(vec![])**.

The recursive variants use **Box<ParseTree>** because Rust requires recursive types to have a statically known size. **Star** instead uses **Vec<ParseTree>**, which already provides indication through a fixed-size vector handle.

Derived traits. `PartialEq` and `Eq` allow direct comparison of parse trees, while `Hash` is omitted because parse trees are not used as keys for deduplication.

The typing judgment describes the structure of a match, but a parse tree must also be related to the word that it matches. For this purpose, they define a *flattening* function $|v|$ that maps a parse tree to the word represented by its structure:

$$\begin{aligned} |()| &= \varepsilon, & |(v_1, v_2)| &= |v_1||v_2|, \\ |l| &= l, & |Left\ v| &= |v|, \\ |[]| &= \varepsilon, & |Right\ v| &= |v|, \\ & & |v : vs| &= |v||vs|. \end{aligned} \tag{4.2}$$

The flattening function connects parse trees to the language of a regular expression:

$$L(r) = \{ |v| \mid \vdash v : r \}. \tag{4.3}$$

Thus, a parse tree v provides a structured *witness* for membership in $L(r)$, recording both the match and the structural choices by which r produces the word.

In Rust. The function takes `&ParseTree` rather than consuming the tree, so it can be inspected, printed, and compared after flattening. The star case similarly borrows its elements through `vs.iter()`. Keeping flattening separate from tree construction also provides an independent parse validation. A tree for w must satisfy $|v| = w$, as required by (4.5). Flattening is implemented directly from (4.2):

```
pub fn flatten(v: &ParseTree) -> String {
    match v {
        ParseTree::Empty      => String::new(),
        ParseTree::Char(c)    => c.to_string(),
        ParseTree::Pair(v1, v2) =>
            format!("{}", flatten(v1), flatten(v2)),
        ParseTree::Left(v)    => flatten(v),
        ParseTree::Right(v)   => flatten(v),
        ParseTree::Star(vs)   => vs.iter().map(flatten).collect(),
    }
}
```

Figure 4.2: The `flatten` function: extract the matched word from a parse tree.

Unambiguous example. Consider $r = ab$ and $w = \mathbf{ab}$. The sequence rule requires a parse tree for a followed by a parse tree for b . Since each subexpression is a literal, the literal rule gives the only possible derivation:

$$\frac{\frac{a \in \Sigma}{\vdash a : a} (\mathsf{T}_{\text{lit}}) \quad \frac{b \in \Sigma}{\vdash b : b} (\mathsf{T}_{\text{lit}})}{\vdash (a, b) : ab} (\mathsf{T}_{\text{seq}}) \tag{4.4}$$

The parse tree is $v = (a, b)$ and flattening recovers the input word: $|(a, b)| = |a| |b| = \mathbf{ab}$.

4.3 Disambiguation

The typing judgment identifies all valid parse trees for a given regular expression, while the flattening function determines which word each tree represents. So for a regular expression r and word w , a parse result must satisfy:

$$\vdash v : r \quad \text{and} \quad |v| = w. \quad (4.5)$$

When several parse trees satisfy these conditions, the typing judgment does not determine which one should be returned. A *disambiguation policy* specifies how this choice is made. Two policies commonly considered in the literature are *Greedy* and *POSIX*:

- *Greedy* resolves choices by the order in which the expression offers them, preferring the earlier alternative and preferring a further iteration to stopping, whatever each consumes [Frisch and Cardelli, 2004].
- *POSIX* resolves choices by subpattern match length from left to right, with ties resolved by the expression order [IEEE and The Open Group, 2017].

The precise definitions of these policies are given in the corresponding literature; see Frisch and Cardelli [2004] and Sulzmann and Lu [2014] for formal treatments. Instead, the examples below show how the policies distinguish between valid parse trees.

Ambiguity in concatenation. For $r_1 = (a + ab)(b + \varepsilon)$ and $w = \mathbf{ab}$, there are two possible matches, corresponding to two distinct parse trees.

The first consumes $\mathbf{ab}$ with the second alternative of $a + ab$ and then chooses ε :

$$\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : a + ab} \quad \frac{\vdash () : \varepsilon}{\vdash \text{Right}() : b + \varepsilon} \quad \Rightarrow \quad \vdash (\text{Right}(a, b), \text{Right}()) : (a + ab)(b + \varepsilon). \quad (4.6)$$

Thus,

$$v_1 = (\text{Right}(a, b), \text{Right}()), \quad |v_1| = \mathbf{ab}. \quad (4.7)$$

The second matches $\mathbf{a}$ and $\mathbf{b}$ using the first alternatives of $a + ab$ and $b + \varepsilon$:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + ab} \quad \frac{\vdash b : b}{\vdash \text{Left } b : b + \varepsilon} \quad \Rightarrow \quad \vdash (\text{Left } a, \text{Left } b) : (a + ab)(b + \varepsilon). \quad (4.8)$$

Thus,

$$v'_1 = (\text{Left } a, \text{Left } b), \quad |v'_1| = \mathbf{ab}. \quad (4.9)$$

Hence $v_1 \neq v'_1$ are valid parse trees for the same expression and word. POSIX selects v_1 because the leftmost subpattern matches the longer string $\mathbf{ab}$, while Greedy selects v'_1 because its left alternative comes first.

Ambiguity in repetition. A similar ambiguity arises from repetition. Consider $r_2 = (a + b + ab)^*$ against $w = \mathbf{ab}$, with the alternation nested as $a + (b + ab)$. The word can either be consumed by one iteration using the ab alternative or by two iterations consuming $\mathbf{a}$ and $\mathbf{b}$ separately.

The first match is

$$\frac{\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : b + ab}}{\vdash \text{Right Right}(a, b) : a + (b + ab)} \Rightarrow \vdash [\text{Right Right}(a, b)] : (a + (b + ab))^*. \quad (4.10)$$

Thus,

$$v_2 = [\text{Right Right}(a, b)], \quad |v_2| = \mathbf{ab}. \quad (4.11)$$

The second match uses two iterations:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + (b + ab)} \quad \frac{\frac{\vdash b : b}{\vdash \text{Left } b : b + ab}}{\vdash \text{Right Left } b : a + (b + ab)} \quad (4.12)$$

$$\Rightarrow \vdash [\text{Left } a, \text{Right Left } b] : (a + (b + ab))^*.$$

Thus,

$$v'_2 = [\text{Left } a, \text{Right Left } b], \quad |v'_2| = \mathbf{ab}. \quad (4.13)$$

Here POSIX selects the single-iteration tree v_2 , since the star's first iteration can match both characters through the ab alternative. Greedy selects the two-iteration tree v'_2 , since the alternative a comes first and consuming only $\mathbf{a}$ leaves $\mathbf{b}$ for a second iteration.

The examples show that the same regular expression and input can admit multiple valid parse trees, with the selected tree depending on the disambiguation policy. The next two chapters apply these ideas to derivative-based parsing and compare the resulting parser implementations.

5 Derivative-Based Parser

Chapter 3 built a matcher on the Brzozowski derivative. It returns a Boolean. Chapter 4 showed that a Boolean is not enough: one expression and one word can admit several parse trees, and a parser has to return exactly one of them.

This chapter closes that gap twice, following Sulzmann and Lu [Sulzmann and Lu, 2014]. The *direct* construction runs the derivative sequence forward and rebuilds a tree from it backward. The *bit-coded* construction builds the tree during the forward pass and needs no backward pass at all. Both return the POSIX tree, and both return the same tree. Each rule is followed immediately by the Rust that transcribes it, and each construction closes with a worked example checked against the running program's own diagnostic output.

5.1 From Answer to Evidence

Take the smallest expression that has a tree worth rebuilding: $r = a \cdot b$ against $w = \mathbf{ab}$, chosen because it has exactly *one* parse tree, so the question is only whether a tree can be recovered at all and not yet which one. Deriving $a \cdot b$ by $\mathbf{a}$ and then $\mathbf{b}$ gives $\varepsilon \cdot b$ and then $\phi \cdot b + \varepsilon$, which is nullable: the word matches. That is where Chapter 3 stopped, and what it discarded on the way is the problem. The first step turned a into ε ; the second turned that ε into ϕ and b into ε . No consumed character survives in the final expression, so the tree (a, b) cannot be read out of it.

It can be recovered by walking the same sequence backward. Figure 5.1 is the whole idea of the direct construction.

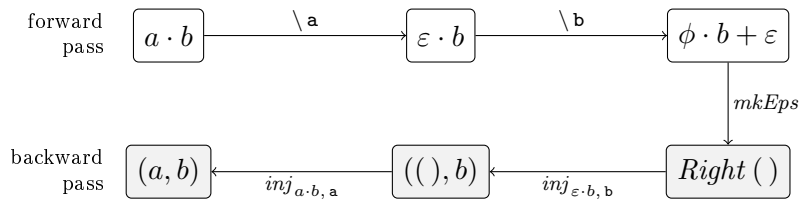


Figure 5.1: The direct construction on $a \cdot b$ against "ab". The top row is the derivative sequence of Section 3.2. *mkEps* builds a tree for the empty word at the end; each *inj* step then undoes one derivative and puts the consumed character back. Every top-row expression must stay in memory until the step below it has run.

mkEps builds a tree for the empty word, all the final expression can justify. Each *inj* step then reinserts one character, moving right to left: after the first the tree is $((), b)$, with $()$ holding open the position where a belongs; after the second that placeholder is a and

the tree is complete. The forward row is unchanged from Chapter 3; this chapter adds only the bottom row.

Which tree. The $a \cdot b$ example has one tree, so it does not say which tree a parser should return when there are several. The chapter’s running example does: $r_1 = (a + ab)(b + \varepsilon)$ against $w = ab$, from Section 4.3, with candidates $v_1 = (Right(a, b), Right())$ and $v'_1 = (Left\ a, Left\ b)$. Both parsers in this chapter return v_1 : POSIX requires each subpattern, left to right, to match the longest possible string [IEEE and The Open Group, 2017, Section 9.1]. The leftmost subpattern $a + ab$ can only reach two characters through its right alternative, so it takes ab whole, leaving $b + \varepsilon$ to take ε , also its right alternative. Two right choices give v_1 . Greedy, the other policy of Section 4.3, prefers the earlier alternative at each choice and selects v'_1 instead. Neither construction below ever builds both candidates and compares them; each makes one local decision, and the two sections that follow say exactly where.

5.2 The Direct Construction

Two functions build the tree, *mkEps* and *inj*, and a third assembles them into the parser; *nullable* and *deriv* are unchanged from Chapter 3. *simplify* is deliberately absent: the direct parser does not import it, so no forward step can simplify by accident, unlike the two matchers of Chapter 3. The paragraph on no-simplification below explains why that omission is required.

The empty-word tree. *mkEps* takes a nullable expression and returns a parse tree for the empty word, recursing on its structure:

$$\begin{aligned}
 mkEps_\varepsilon &= () & (E_\varepsilon) \\
 mkEps_{r^*} &= [] & (E_*) \\
 mkEps_{r_1 r_2} &= (mkEps_{r_1}, mkEps_{r_2}) & (E_{seq}) \\
 mkEps_{r_1 + r_2} &= \begin{cases} Left\ mkEps_{r_1} & \varepsilon \in L(r_1) \\ Right\ mkEps_{r_2} & \text{otherwise} \end{cases} & \begin{matrix} (E_+^L) \\ (E_+^R) \end{matrix}
 \end{aligned} \tag{5.1}$$

Three of the four rules are forced: ε has one tree, a star matching empty took zero iterations, and a concatenation matches empty only when both factors do. There is no rule for $\emptyset$ or a literal, since neither is ever nullable. The fourth rule is the one decision this function makes: when both disjuncts of $r_1 + r_2$ are nullable, "longest possible string" cannot separate them, so POSIX falls back on position and prefers the left, the $\varepsilon \in L(r_1)$ guard. Sulzmann and Lu prove that deciding this way yields the POSIX tree [Sulzmann and Lu, 2014].

In Rust: `mk_eps`. Figure 5.2 is Equation (5.1) one variant at a time.

```

pub fn mk_eps(r: &Regex) -> ParseTree {
  match r {
    Regex::Eps => ParseTree::Empty,
    Regex::Star(_) => ParseTree::Star(Vec::new()),
    Regex::Seq(r1, r2) => ParseTree::Pair(
      Box::new(mk_eps(r1)), Box::new(mk_eps(r2))),
    Regex::Alt(r1, r2) => if nullable(r1) {
      ParseTree::Left(Box::new(mk_eps(r1)))
    } else {
      ParseTree::Right(Box::new(mk_eps(r2)))
    },
    Regex::Lit(c) => panic!("mk_eps called on Lit('{}')", c),
    Regex::Phi => panic!("mk_eps called on Phi"),
  }
}

```

Figure 5.2: `mk_eps`, transcribing Equation (5.1).

r is borrowed, not consumed, because `inject` reads it again later in the backward pass. The `Alt` arm is the whole POSIX decision and is one `if`: r_2 is touched only when `nullable(r1)` is false, with no appeal to lengths, since at the empty word there are none to compare. The last two arms encode a precondition Equation (5.1) leaves implicit, that neither ϕ nor a literal is ever nullable; `panic!` states that precondition rather than inventing a tree that would be wrong and well formed at once, which the agreement tests of Chapter 8 cannot detect on their own.

Reinserting a character. *inj* inverts one derivative step: given a tree for $r \setminus x$, it returns a tree for r with x put back.

$$\begin{aligned}
inj_{x \setminus x}() &= x & (I_{\text{lit}}) \\
inj_{(r_1+r_2) \setminus x}(Left\ v_1) &= Left\ (inj_{r_1 \setminus x}\ v_1) & (I_+^L) \\
inj_{(r_1+r_2) \setminus x}(Right\ v_2) &= Right\ (inj_{r_2 \setminus x}\ v_2) & (I_+^R) \\
inj_{r^* \setminus x}(v, vs) &= (inj_{r \setminus x}\ v) : vs & (I_*) \\
inj_{(r_1 r_2) \setminus x}(v_1, v_2) &= (inj_{r_1 \setminus x}\ v_1, v_2) & (I_{\text{seq}}),\ n(r_1) = \text{false} \\
inj_{(r_1 r_2) \setminus x}(Left(v_1, v_2)) &= (inj_{r_1 \setminus x}\ v_1, v_2) & (I_{\text{seq}}^L),\ n(r_1) = \text{true} \\
inj_{(r_1 r_2) \setminus x}(Right\ v_2) &= (mkEps_{r_1},\ inj_{r_2 \setminus x}\ v_2) & (I_{\text{seq}}^R),\ n(r_1) = \text{true}
\end{aligned} \tag{5.2}$$

The first four rules are direct: a literal was derived to ε and the character goes back; an alternation's incoming tag says which side to descend into; a star's incoming tree is a pair whose first component is the iteration in progress. The three concatenation rules invert the two cases of D_{seq} : when r_1 is not nullable no fresh $+$ was introduced and the tree is a plain pair (I_{seq}); when it is, the fresh $+$'s tag says whether the derivative continued inside r_1 (I_{seq}^L) or skipped it entirely, in which case r_1 's half of the pair was never derived and has to be synthesized by $mkEps_{r_1}$ (I_{seq}^R). That second $mkEps$ call, in the middle of the backward pass, is what Section 5.3 removes.

In Rust: `inject`. `inject(r, l, v)` carries three arguments because *inj* is indexed by an expression and a character, not by the tree alone. Figure 5.3 is the whole function.

```
pub fn inject(r: &Regex, l: char, v: ParseTree) -> ParseTree {
  match r {
    Regex::Lit(c) => { assert!(*c == l);
      ParseTree::Char(l) }
    Regex::Alt(r1, r2) => match v {
      ParseTree::Left(v1) =>
        ParseTree::Left(Box::new(inject(r1, l, *v1))),
      ParseTree::Right(v2) =>
        ParseTree::Right(Box::new(inject(r2, l, *v2))),
      _ => panic!("inject/Alt: want Left|Right"),
    },
    Regex::Star(r1) => match v {
      ParseTree::Pair(v1, vs) => {
        let mut it = vec![inject(r1, l, *v1)];
        if let ParseTree::Star(rest) = *vs { it.extend(rest); }
        ParseTree::Star(it)
      }
      _ => panic!("inject/Star: want Pair"),
    },
    Regex::Seq(r1, r2) => if !nullable(r1) {
      match v {
        ParseTree::Pair(v1, v2) => ParseTree::Pair(
          Box::new(inject(r1, l, *v1)), v2),
        _ => panic!("inject/Seq: want Pair"),
      }
    } else {
      match v {
        ParseTree::Left(vp) => match *vp {
          ParseTree::Pair(v1, v2) => ParseTree::Pair(
            Box::new(inject(r1, l, *v1)), v2),
          o => panic!("inject/SeqL: want Pair, got {:?}", o),
        },
        ParseTree::Right(v2) => ParseTree::Pair(
          Box::new(mk_eps(r1)),
          Box::new(inject(r2, l, *v2))),
        _ => panic!("inject/Seq: want Left|Right"),
      }
    },
    Regex::Eps | Regex::Phi => panic!("inject called on Eps/Phi"),
  }
}
```

Figure 5.3: `inject`, transcribing Equation (5.2) arm for arm; the `Seq` arm’s inner `if` is the side condition $n(r_1)$, and its inner `match` then reads the tag the derivative left behind, in the order of the table above.

The function matches on `r` first and `v` inside each arm, because Equation (5.2) indexes every rule by the expression first; a `Left` tag means one thing under an alternation and

another under a nullable concatenation, so v cannot be matched first. Ownership follows the data flow: r is borrowed and only read, v is taken by value and taken apart. That is what lets $v2$ move straight into a new `Pair` with no copy in the two matching `Seq` arms, where Equation (5.2) leaves v_2 unchanged. The `Right` arm of `Seq` is the only one that builds something new: the derivative skipped r_1 entirely, so nothing exists to reuse for its half of the pair, and `mk_eps(r1)` synthesizes it, the same second `mk_eps` call l_{seq}^R requires. Calling `nullable(r1)` here recomputes what the forward pass already decided when it chose D_{seq}^n ; caching it would mean carrying a second structure alongside the expression sequence, the bookkeeping Section 5.3 avoids by not having a backward pass at all.

The parser, its driver, and why it cannot simplify. The two functions compose into one recursion, applying `mkEps` once at the end of the input and `inj` once per character on the way back:

$$\text{parse}(r, \varepsilon) = \text{mkEps}_r \text{ (if } \varepsilon \in L(r)), \quad \text{parse}(r, xw) = \text{inj}_{r \setminus x}(\text{parse}(r \setminus x, w)).$$

If the expression reached at the end is not nullable, parsing fails. Figure 5.4 materializes the top row of Figure 5.1 as a `Vec<Regex>` and walks it backward; a purely recursive driver is possible too, with the call stack standing in for that `Vec`, but a thread's stack is a fixed allocation where a `Vec` grows on the heap, and the recursive form overflows on input the loop form handles without difficulty (Section 8.5.1).

```
pub fn parse_deriv_std_loop(input: &str, r: &Regex)
  -> Option<ParseTree>
{
  let chars: Vec<char> = input.chars().collect();
  let mut expressions = Vec::with_capacity(chars.len() + 1);
  expressions.push(r.clone());
  for &c in &chars {
    expressions.push(deriv(expressions.last().unwrap(), c));
  }
  if !nullable(expressions.last().unwrap()) { return None; }
  let mut tree = mk_eps(expressions.last().unwrap());
  for i in (0..chars.len()).rev() {
    tree = inject(&expressions[i], chars[i], tree);
  }
  Some(tree)
}
```

Figure 5.4: The loop driver: forward pass into `expressions`, then one `inject` per character walking it back.

One rule constrains the forward pass: it must not simplify. Section 3.2 simplified after every step because a Boolean answer depends only on language equivalence, but $\text{inj}_{r \setminus x}$'s case split reads r 's shape, and $S_2 (\varepsilon \cdot r \rightarrow r)$ deletes exactly the node the backward pass would need. So the direct construction pays for the tree in expression size; `simplify` is not imported by this module, and the two matchers of Chapter 3 are the only callers that do import it. Both drivers return an `Option<ParseTree>`, the one type every parser in this thesis produces, letting agreement tests compare outputs directly; `flatten` is kept separate from construction as an independent check that a successful parse of w still reads back as w .

Worked example. Run $r_1 = (a + ab)(b + \varepsilon)$ against `ab`, checked against `-parser deriv_std_rec -diag 3` (Appendix C.1). Two derivative steps consume the input:

$$r_1 \xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) \xrightarrow{b} \underbrace{(\phi + (\phi b + \varepsilon))(b + \varepsilon)}_X + \underbrace{(\varepsilon + \phi)}_Y. \quad (5.3)$$

The first step takes D_{seq} , since $a + ab$ is not nullable; the second takes D_{seq}^n , since $\varepsilon + \varepsilon b$ now is, which is what introduces the summand Y that makes the whole expression nullable. Note that the `Eps` inside εb , printed by the tool as `Seq(Eps, Lit('b'))`, is exactly the node S_2 would delete; it survives because Section 5.2 forbids simplification here, and the backward pass needs it below.

Both disjuncts X, Y of the final expression are nullable, so $mkEps$ takes its one decision, E_+^L , and works inward through E_{seq} and two applications of E_+^R (neither ϕ nor b is nullable) to reach $v_2 = \text{Left}(\text{Right} \text{ Right } (), \text{Right } ())$, confirmed by the report as `mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))`. Injecting back undoes the two steps in reverse. Undoing `b` applies I_{seq}^L then I_+^R then I_{seq}^R , synthesizing ε 's half with $mkEps$ since the derivative had skipped it, giving $v_1 = (\text{Right}((), b), \text{Right}())$. Undoing `a` applies I_{seq} , I_+^R and I_{lit} to replace the placeholder $()$ with a , reaching

$$v_0 = (\text{Right}(a, b), \text{Right}()) = v_1,$$

the report's own final injection, and the POSIX tree of Section 5.1. The header records what this cost: **Expressions stored:** 3 (`r0 -> r1 -> r2`), one per input character plus the original, each held live until the backward pass consumes it. The next section removes that requirement.

5.3 The Bit-Coded Construction

The direct construction cannot start building the tree until the input is exhausted, because *inj* needs each r_{i-1} to reconstruct against. Sulzmann and Lu's bit-coded construction (their Section 3.2) [Sulzmann and Lu, 2014] removes that dependency by building the tree during the forward pass. Figure 5.5 shows the result on a^* against "aaa", the bit-coded analogue of Figure 5.1: no bottom row, and the expression stays two nodes wide while a bit-prefix carries the tree.

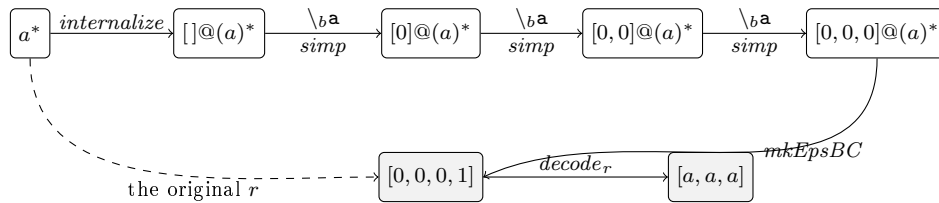


Figure 5.5: The bit-coded construction on a^* against "aaa". Every arrow points forward; no r_i is needed once its successor exists. The dashed arrow is the one backward dependency left: *decode* needs the original r to know what shape the bits describe.

Bits, and annotated expressions. A parse tree becomes a flat bit-sequence: at an alternation, 0 marks *Left* and 1 marks *Right*; inside a star, 0 marks "one more iteration"

and 1 marks "stop", so `aaa` under a^* encodes as $[0, 0, 0, 1]$ and ε as $[1]$. The bits alone are not a tree, only which branch was taken, so reading them back needs r . The construction gets there by annotating the derivative's *input*: an annotated expression ri extends r with a bit-list bs at every node except $\emptyset$, which has no parse tree to annotate,

$$ri ::= \emptyset \mid (bs@ \varepsilon) \mid (bs@ x) \mid (bs@ ri_1 \oplus ri_2) \mid (bs@ ri_1 ri_2) \mid (bs@ ri^*),$$

so that by the time a subexpression has become ε , its accumulated annotation already is the relevant slice of the bit-string. *internalize* converts r into ri with every bit-list empty except at an alternation, where the left operand is tagged with a leading 0 and the right with a leading 1; *fuse*(bs, ri) prepends a bit-list onto whatever annotation ri 's top node already carries, and every other operation here is built from it.

In Rust. Figure 5.6 is a second enum alongside `Regex`.

```
pub enum ARegex {
    Phi, // no annotation slot
    Eps(Vec<bool>), Lit(Vec<bool>, char),
    Alt(Vec<bool>, Box<ARegex>, Box<ARegex>),
    Seq(Vec<bool>, Box<ARegex>, Box<ARegex>),
    Star(Vec<bool>, Box<ARegex>),
}
```

Figure 5.6: The annotated-expression type. `Phi` is the one variant with no `Vec<bool>`, matching the grammar's omission.

Comparing this with Figure 2.1 shows the one place a plain `Regex` will not do: threading parse-tree information through the expression needs somewhere on every node to put it. *fuse* and *internalize* (Figure 5.7) implement the two operations above; *fuse* rebuilds a node one arm per variant since no field is common to all of them, with an early return that makes the (very common) empty-prefix case a move and not a rebuild, and *internalize*'s `Alt` arm is the only place in the whole construction that introduces a bit from nothing, tagging each side exactly once.

```

pub fn fuse(bs: &[bool], ri: ARegex) -> ARegex {
  if bs.is_empty() { return ri; }
  match ri {
    ARegex::Phi => ARegex::Phi,
    ARegex::Eps(p) => ARegex::Eps(combine(bs, p)),
    ARegex::Lit(p, c) => ARegex::Lit(combine(bs, p), c),
    ARegex::Alt(p, r1, r2) => ARegex::Alt(combine(bs, p), r1, r2),
    ARegex::Seq(p, r1, r2) => ARegex::Seq(combine(bs, p), r1, r2),
    ARegex::Star(p, r) => ARegex::Star(combine(bs, p), r),
  }
}

pub fn internalize(r: &Regex) -> ARegex {
  match r {
    Regex::Phi => ARegex::Phi,
    Regex::Eps => ARegex::Eps(vec![]),
    Regex::Lit(c) => ARegex::Lit(vec![], *c),
    Regex::Alt(r1, r2) => ARegex::Alt(vec![],
      Box::new(fuse(&[false], internalize(r1))),
      Box::new(fuse(&[true], internalize(r2)))),
    Regex::Seq(r1, r2) => ARegex::Seq(vec![],
      Box::new(internalize(r1)), Box::new(internalize(r2))),
    Regex::Star(r1) => ARegex::Star(vec![], Box::new(internalize(r1))),
  }
}

```

Figure 5.7: `fuse` prepends a bit-prefix; `internalize` converts a `Regex`, tagging the two sides of every alternation. `combine` concatenates two bit-lists.

The annotated derivative, in Rust. $ri \setminus_{bl}$ has the case structure of Equation (3.6), bits threaded through instead of discarded:

$$\begin{aligned}
(bs @ ri_1 \oplus ri_2) \setminus_{bl} &= (bs @ (ri_1 \setminus_{bl}) \oplus (ri_2 \setminus_{bl})) & (B_+) \\
(bs @ ri_1 ri_2) \setminus_{bl} &= (bs @ (ri_1 \setminus_{bl}) ri_2) & (B_{seq}), \varepsilon \notin L(ri_1) \\
(bs @ ri_1 ri_2) \setminus_{bl} &= (bs @ (ri_1 \setminus_{bl}) ri_2) \oplus fuse(mkEpsBC_{ri_1}, ri_2 \setminus_{bl}) & (B_{seq}^n), \varepsilon \in L(ri_1) \\
(bs @ ri^*) \setminus_{bl} &= (bs @ fuse([0], ri \setminus_{bl}) ([] @ ri^*)) & (B_*)
\end{aligned} \tag{5.4}$$

On the branch where the derivative abandons ri_1 and jumps into ri_2 , the bits ri_1 would contribute to an empty match are fused onto ri_2 's derivative: this is the entire optimization, doing during the forward pass what l_{seq}^R postponed into *inj*.

```

ARegex::Seq(bs, r1, r2) => {
  if nullable_bc(&r1) {
    let eps_bits = mk_eps_bc(&r1);
    let d1 = deriv_bc(*r1, 1);
    let right_branch = fuse(&eps_bits, deriv_bc(*r2.clone(), 1));
    ARegex::Alt(bs, Box::new(ARegex::Seq(vec![], Box::new(d1), r2)),
                Box::new(right_branch))
  } else {
    ARegex::Seq(bs, Box::new(deriv_bc(*r1, 1)), r2)
  }
}
ARegex::Star(bs, r) => {
  let fused = fuse(&[false], deriv_bc(*r.clone(), 1));
  ARegex::Seq(bs, Box::new(fused), Box::new(ARegex::Star(vec![], r)))
}

```

Figure 5.8: The concatenation and star arms of `deriv_bc`, transcribing Equation (5.4). `r2.clone()` and `r.clone()` pay for the body appearing both derived and underived in B_{seq}^n and B_* .

Extracting the bits, and simplification. *mkEpsBC* mirrors *mkEps* but returns bits, with the same left-preference guard at an alternation and a closing 1 at a star:

$$\begin{aligned}
mkEpsBC_{(bs @ \varepsilon)} &= bs, & mkEpsBC_{(bs @ ri^*)} &= bs ++ [1], \\
mkEpsBC_{(bs @ ri_1 ri_2)} &= bs ++ mkEpsBC_{ri_1} ++ mkEpsBC_{ri_2}, \\
mkEpsBC_{(bs @ ri_1 \oplus ri_2)} &= bs ++ \begin{cases} mkEpsBC_{ri_1} & \varepsilon \in L(ri_1) \\ mkEpsBC_{ri_2} & \text{otherwise.} \end{cases}
\end{aligned} \tag{5.5}$$

decode_r then turns the bit-string into an ordinary parse tree, using slice patterns on the conventions above: `[false, rest @ ..]` means **Left** at an alternation and one more iteration at a star, `[true, ..]` the opposite. It takes the *original*, un-annotated *r*, since *ri* no longer describes the shape once its own annotations are consumed, the dashed arrow in Figure 5.5. *decode* is the one function this chapter shares with Chapter 6’s *pderiv_bc*, which reaches an identical (*Regex*, *bits*) pair by an entirely different route.

Section 5.2 forbade simplification in the direct construction; the bit-coded one is free to simplify, since the information a backward pass would need is already carried on the expression as bits, and it needs to: Sulzmann and Lu [2014] show a^* doubling in size at every step without it, $a^* \rightarrow \varepsilon a^* \rightarrow \phi a^* + \varepsilon a^* \rightarrow \dots$. A function *simp*, applied after every step, propagates ϕ through *Seq/Alt*, deduplicates nested alternations keeping the first syntactic duplicate, and folds an eliminated ε ’s bits forward with `fuse` rather than dropping them; an explicit *isPhi* test recognizes an empty-language expression built up from these rules, not only the literal $\emptyset$. Figure 5.9 shows where the bound this buys holds and where it does not.

The bits are not redundant duplicates to be pruned on shape alone: each branch’s annotation is a different candidate parse tree, and dropping duplicates that ignore bits would change which tree *mkEpsBC*’s left-preference selects, not merely shrink the expression. Sulzmann and Lu claim, without fully proving, that the construction runs in time linear in the input, on the premise that *simp* keeps the expression small; Figure 5.9 is one concrete

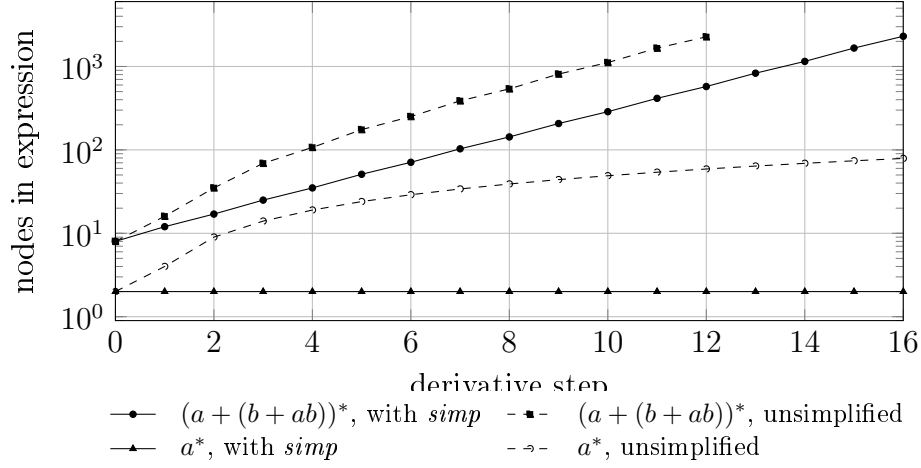


Figure 5.9: Expression size per derivative step, log scale. *simp* holds a^* at two nodes but $(a + (b + ab))^*$ still grows $\Theta(2^{n/2})$: a straight line on this axis is exponential growth, because at every step the surviving alternation branches are distinct only in their bit annotations, so S_4 never collapses them. Regenerate with `cargo run --release --example thesis_figures --growth; tests/test_thesis_figures.rs` pins these values.

case where that premise holds (a^*) and one where it fails ($(a + (b + ab))^*$). Chapter 8 measures the wall-clock consequence.

The driver is four lines of loop and three of finish:

```
pub fn parse_bitcoded_loop(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut ri = internalize(r);
    for l in input.chars() { ri = simp(deriv_bc(ri, l)); }
    if !nullable_bc(&ri) { return None; }
    Some(decode(r, &mk_eps_bc(&ri)))
}
```

Compare with Figure 5.4: no `Vec<Regex>` and no backward loop, `ri` reassigned in place so only one annotated expression is ever live, and `r` itself still borrowed at the last line, the dashed dependency of Figure 5.5 made visible in the signature.

Worked example. The same r_1 against `ab`, checked against `-parser deriv_bc -diag 3` (Appendix C.2). `internalize` tags the two alternatives of $a + ab$ with `[false]/[true]` and likewise for $b + \varepsilon$; those four tags are the only bits this run will ever produce, since `internalize` is the only source of new bits. The first character takes B_{seq} and moves no bit, only turning 'a' to `epsilon`. The second takes B_{seq}^n : two bits move, and both are traceable. A `[false, false]` prefix appears on the right disjunct, which is `fuse(mkEpsBC ri1)`, $a + ab$'s own left-alternative bit followed by $(b + \varepsilon)$'s; and a `[true]` tags the left disjunct, recording that the derivative continued into it, the same tagging *inj* discovered on the way back in Section 5.2 but here committed immediately. Both disjuncts are now nullable, and `mkEpsBC` faces the same choice *mkEps* faced above: EB_+^L then EB_+^R give `[1, 1]`, and the report confirms `decode((a|ab)(b|), [11]): = (Right (a, b), Right ())`, the same v_1 Section 5.2 reached through an explicit backward pass, read here off the forward pass alone.

r_1 has no star; running a^* against "aaa" exercises B_* instead, and is exactly Figure 5.5: the

expression stays `([]@a)*` at every step while the prefix grows by one `false` per character, and `mkEpsBC(ri3) = [0001]` decodes to `[a, a, a]`, three iterations and a stop.

Comparison, and where this leaves the next chapter.

	<code>deriv_std</code>	<code>deriv_bc</code>
Named intermediates	<code>r0 -> r1 -> r2</code>	<code>ri0 -> ri1 -> ri2</code>
Retained at the end	all three	the last one only
Tree-building calls	<code>1 mkEps + 2 inject</code>	<code>1 mkEpsBC + 1 decode</code>
Simplification	none, by necessity	<code>simp</code> after every step

`deriv_std` needs one `inject` per character; `deriv_bc` needs exactly one `mkEpsBC` and one `decode`, however long the input is: the whole content of the optimization. Against `b`, neither parser reaches a tree-building phase; `deriv_std` holds a `Seq` whose left operand has both alternatives dead, while `deriv_bc`'s `simp` collapses the same expression to `phi` outright, the clearest illustration of what *isPhi* adds over plain `simplify`. Both then report the identical failure, character for character (Appendices C.5, C.6), since the diagnostics layer reads its expected set off the expression at the point of failure regardless of which construction produced it. Running both parsers, plus `deriv_std`'s loop driver, over 300 generated expressions against 39 inputs each, 11 700 cases total (Section 8.4.2), all three agree on every one, on membership and on the exact tree, despite the bit-coded and inject routes sharing no code beyond `Regex` and `nullable`.

Both constructions compute the POSIX tree by the same move: consume the input with `r\`*x*, then resolve the one remaining choice, an alternation whose two sides both match the empty word, by preferring the left. *mkEps* makes that choice on the way back; *mkEpsBC* makes it at the end of the way out. Chapter 6 keeps this shape and replaces `r\`*x* with Antimirov's *pd(r, x)*, which returns a *set*. With a set there is no single expression to be nullable and no single alternation to prefer the left of, which changes not only the machinery but the answer.

6 Partial-Derivative-Based Parser

Chapter 5 built two parsers around Brzozowski’s derivative. This chapter repeats the exercise with Antimirov’s partial derivative from Section 3.3 in place of $r \backslash x$ throughout.

The substitution looks local and is not. Both operators consume one character and return what remains to be matched, and either can drive a parser. But $r \backslash x$ returns one expression and $pd(r, x)$ returns a *set*, and a set has no single top-level alternation whose left branch a rule can prefer. That is where Chapter 5’s two constructions made their POSIX choice, in *mkEps* and in *mkEpsBC*, and with the set representation there is nothing there to make it on. The choice moves somewhere else, and it comes out differently. Section 6.1 locates it, and Section 6.1.2 shows the consequence on an expression small enough to check by inspection.

Neither *flops14-extended* [Sulzmann and Lu, 2014] nor Sulzmann’s teaching-page Haskell reference [Sulzmann, 2026] defines a POSIX partial-derivative parser at all. The paper’s bit-coded construction is Brzozowski-only, and the teaching page’s own bit-coded partial-derivative parser, `pDerivBC`, is built around a deduplication rule that settles the matter by itself, without comparing candidate trees. This chapter builds two parsers on that basis: `pderiv_bc` (Section 6.2), following `pDerivBC` directly, and `pderiv_std` (Section 6.3), this project’s own construction, built to give `pderiv_bc` the same kind of matched non-bit-coded counterpart Chapter 5’s `deriv_bc` has in `deriv_std`. As in that chapter, each construction’s rules are followed immediately by the Rust that implements them. Both parsers are verified to agree with each other exactly, and with the two POSIX parsers on membership alone, in Chapter 8 and again in Section 6.5.

Trace excerpts below use the same running example, r_1 against `ab` and against `b`.

6.1 From One Expression to a Frontier

Section 3.3 defined $pd(r, x)$ as a set of residual expressions, not one combined expression, and noted the NFA reading: each element is one state of an implicit NFA that r denotes, alive after reading x . Extending this to parsing means pairing every surviving element with enough information to rebuild a parse tree, should that element turn out to be the winner at the end of the input. That extra payload is either an accumulated bit-string per element (Section 6.2) or, in this project’s own extension, an accumulated tree-injection closure per element (Section 6.3).

Either way the algorithm has the same shape. Maintain a *frontier* of `(residual, extra)` pairs, one per live *strand*; at each character replace it with the union of pd applied to every current strand, each contributing its own possibly-empty set of successors; and

at the end of the input select the first strand in the frontier whose residual is nullable. Figure 6.1 draws this for the running example, with `pderiv_bc`'s bits as the concrete choice of payload; Section 6.3 redraws the same frontier with the other payload.

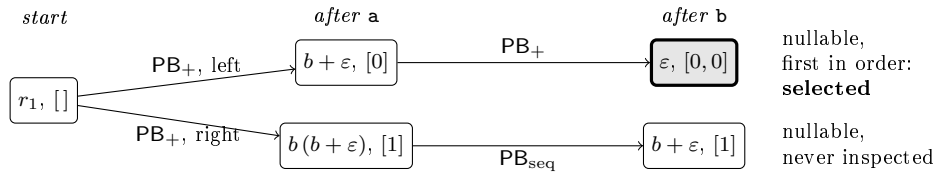


Figure 6.1: The frontier for $r_1 = (a + ab)(b + \varepsilon)$ against "ab", with `pderiv_bc`'s bits as the per-strand extra and the rule producing each strand on its arrow. One character advances every strand independently. At the end *both* strands are nullable, and selection takes the first in list order without comparing them, which is where the construction settles on one tree and not the other.

6.1.1 Where the Choice Is Actually Made

That last step, "the *first* nullable strand in the frontier", is this construction's disambiguation policy. It is decided implicitly, as a side effect of keeping the frontier's growth finite, and no rule anywhere in the construction states a preference between candidate trees.

The reason no such rule appears is worth following, because it is not an oversight. Antimirov's finiteness theorem (Section 3.3) bounds the set of *distinct* partial derivatives, but nothing about the extended, tree-carrying construction keeps strands distinct as they accumulate payload: two different derivation paths can reach the identical residual *shape* by different routes, each carrying its own bits or injection. Kept without pruning, the frontier would multiply copies of an already-explored residual instead of staying inside the bounded syntactic set *pd* guarantees. So pruning is needed. But pruning by comparing the accumulated bits or trees, keeping whichever strand has consumed more in the sense of Section 5.1, would mean comparing incomplete, still-growing state: a strand's tree is finished only once the input is exhausted, and at the moment the pruning must happen there is nothing complete to compare.

The Haskell reference's choice is `nub2`: deduplicate purely on residual *shape*, keeping whichever strand reached that shape *first*. This sidesteps the problem by discarding the comparison altogether. Once a shape has been claimed, every later strand reaching the identical shape is dropped, whatever either one's payload would eventually have produced. "First" means first in the list-ordered traversal *pd*'s own case structure produces, which explores the left operand of $+$ before the right at every step. The construction is therefore left-biased and commits as soon as something succeeds, which is not the longest-wins-with-left-as-tiebreak rule of Section 5.1.

It is worth being careful about what that establishes. It says which strand this construction selects. It does not say that the construction implements any particular named disambiguation policy in general. The trace states the consequence in one line, at the point of decision:

```

SELECTION (first nullable residual, priority/list order)
any nullable residual in final frontier = true -> proceed to

```

mkEpsBC + decode

There is no comparison here, because there is nothing to compare against: the moment a nullable residual is found, scanning stops. That is the shaded node in Figure 6.1, and the strand below it is never examined although it is nullable too.

This reading is this thesis’s own account of where the construction settles the question. Neither primary source states it in these terms, and neither claims the result coincides with a named policy. What can be checked is narrower, and is checked in Chapter 8: the two parsers of this chapter agree with each other on every case tried, they agree with the derivative parsers on membership always, and they disagree with them on the tree wherever the expression is ambiguous, including the `paper_r1` and `paper_r2` examples of Section 4.3.

6.1.2 A Family of Witnesses

The two paper examples are not the only place the divergence shows, and it helps to have a case where the pattern is simple enough that both answers can be read off by inspection. Take $r = (b + bb)^*$ against a run of `bs`. The derivative parsers prefer the longer alternative at each iteration; the partial-derivative parsers take the left alternative as soon as it works. The two therefore partition the same input differently:

Input	Derivative	Partial derivative
"bb"	[Right (b, b)]	[Left b, Left b]
"bbb"	[Right (b, b), Left b]	[Left b, Left b, Left b]
"bbbbbb"	[Right (b, b) × 3]	[Left b] × 6

The derivative parsers use as few iterations as the longest-alternative rule allows, padding with a single `Left b` only when the length is odd. The partial-derivative parsers use one iteration per character throughout, because b is the left alternative and it always matches. Both flatten back to the same input and both are valid parse trees for it: the expression is ambiguous, and the two constructions resolve the ambiguity differently. These rows are the output of `cargo run - parse "(b|bb)*" "bb" -parser all`, which runs all five entry points on one input and prints the trees side by side.

6.2 The Bit-Coded Construction: `pderiv_bc`

Sulzmann’s teaching-page Haskell transcript gives a bit-coded partial-derivative construction, `pDerivBC`, that this thesis’s `pderiv_bc` follows directly [Sulzmann, 2026]. It extends `pd` with per-strand bits the same way Section 5.3’s `\b` extends the Brzozowski derivative, with one visible difference: the bits stay *external* to the expression tree, as a plain list carried alongside a plain, un-annotated r , instead of being folded into an `ARegex`-shaped tree.

Writing D_r for $pDerivBC(x, r)$ throughout, to keep the set comprehensions readable, the base and non-sequence cases are

$$\begin{aligned}
D_\varepsilon &= \{\} & D_\emptyset &= \{\} & (\text{PB}_\varepsilon), (\text{PB}_\emptyset) \\
D_y &= \begin{cases} \{(\varepsilon, [])\} & x = y \\ \{\} & x \neq y \end{cases} & & (\text{PB}_{\text{lit}}) \\
D_{r+s} &= \text{nub2}(\{(r', 0:bs) \mid (r', bs) \in D_r\} \\
&\quad \cup \{(s', 1:bs) \mid (s', bs) \in D_s\}) & (\text{PB}_+) \\
D_{r*} &= \text{nub2}(\{(\text{smartC}(r', r^*), 0:bs) \mid (r', bs) \in D_r\}) & (\text{PB}_*)
\end{aligned} \tag{6.1}$$

and the sequence case splits on the nullability of the left factor exactly as Equation (3.6) does:

$$\begin{aligned}
D_{rs} &= \{(\text{smartC}(r', s), bs) \mid (r', bs) \in D_r\} & (\text{PB}_{\text{seq}}), n(r) = \text{false} \\
D_{rs} &= \text{nub2}(\{(r's, bs) \mid (r', bs) \in D_r\} \\
&\quad \cup \{(s', \text{mkEpsBC}_r ++ bs) \mid (s', bs) \in D_s\}) & (\text{PB}_{\text{seq}}^n), n(r) = \text{true}
\end{aligned} \tag{6.2}$$

The names extend Section 3.3's **P** scheme, one **PB** rule per **P** rule, and the derivation below cites them with the binding of r and s given at each application. Here mkEpsBC_r is the plain- r analogue of Section 5.3's mkEpsBC , identical in structure but walking an unannotated tree; smartC is the smart concatenation constructor of Section 3.3, collapsing $\varepsilon \cdot s$ to s ; and **nub2** deduplicates a list of (**residual**, **bits**) pairs by comparing the residual alone, keeping the first occurrence.

Two details of Equation (6.2) are worth holding onto. PB_{seq}^n 's second set is where a strand inherits mkEpsBC_r , the bits r would have contributed by matching empty, which is the same move B_{seq}^n made with *fuse* in Chapter 5 and the same one I_{seq}^R made with *mkEps*. And the two sequence cases build their residuals differently: PB_{seq} uses *smartC* while PB_{seq}^n builds its first strand with a plain $r's$. That asymmetry is present in the reference itself and is preserved here and not "corrected", since correctness of this construction is judged by literal agreement with that reference (Chapter 8) and not against an independently derived ideal form. Section 6.3 returns to the same question from the other side, where the choice between the two constructors is not free at all.

In Rust. The module `pderiv_bc/pderiv.rs` transcribes *pDerivBC* case for case. Each D_r becomes a vector of (`Regex`, `Vec<bool>`) pairs and not a set, because order is what encodes priority here, and a `HashSet` would discard exactly the information selection depends on. Deduplication is factored into its own function, Figure 6.2, used by every case that can produce more than one strand.

```

fn nub2(pairs: Vec<(Regex, Vec<bool>>>) -> Vec<(Regex, Vec<bool>>> {
    let mut seen: HashSet<Regex> = HashSet::with_capacity(pairs.len());
    let mut out = Vec::with_capacity(pairs.len());
    for (r, bs) in pairs {
        if seen.insert(r.clone()) {
            out.push((r, bs));
        }
    }
    out
}

```

Figure 6.2: `nub2`: stable deduplication on the residual alone. `HashSet::insert` returns `true` the first time a value is seen and `false` on every repeat, so the loop keeps a pair exactly when its residual has not been seen before, in the order the input `Vec` presents them.

This is the one place in the codebase where `Regex`'s derived `Hash` implementation (Section 2.3) is load-bearing; elsewhere equality alone would do, but a hash set is what keeps `nub2` linear in the number of strands instead of quadratic.

Figure 6.3 gives the two arms that produce more than one strand and tag them as they go; `Star` is left out, since it tags its one strand with a single leading `false` the same way `Alt`'s left arm does, and otherwise matches Figure 5.8's star arm in Chapter 5.

```

Regex::Alt(r1, r2) => {
  let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
    .into_iter()
    .map(|(r1_, mut bs)| {
      let mut b = vec![false];
      b.append(&mut bs);
      (r1_, b)
    })
    .collect();
  out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, mut bs)| {
    let mut b = vec![true];
    b.append(&mut bs);
    (r2_, b)
  }));
  nub2(out)
}

Regex::Seq(r1, r2) => {
  if nullable(r1) {
    let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
      .into_iter()
      .map(|(r1_, bs)| (Regex::seq(r1_, *r2.clone()), bs))
      .collect();
    let eps_bits = mk_eps_bits(r1);
    out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, bs)| {
      let mut b = eps_bits.clone();
      b.extend(bs);
      (r2_, b)
    }));
    nub2(out)
  } else {
    pderiv_bc(r1, x)
      .into_iter()
      .map(|(r1_, bs)| (smart_seq(r1_, r2), bs))
      .collect()
  }
}

```

Figure 6.3: The alternation and sequence arms of `pderiv_bc`, transcribing PB_+ , PB_{seq} and PB_{seq}^n .

`Alt` prepends its tag with `vec![false]/vec![true]` and `append`, which is $0:bs$ and $1:bs$ read as a `Vec` operation, then deduplicates the combined vector once at the end, matching `nub2`'s position on the outside of the union in Equation (6.1).

`Seq` is the one place the two constructors genuinely differ, and the code keeps that difference instead of smoothing it away. The nullable branch's first strand is built with plain `Regex::seq(r1_, *r2.clone())`, matching PB_{seq}^n 's plain r 's; its second strand reuses `mk_eps_bits(r1)` exactly as Section 6.2 described. The non-nullable branch, reached only when `nullable(r1)` is false, calls `smart_seq(r1_, r2)` instead, matching PB_{seq} . A reader who only skims the two branches could mistake this for an inconsistency; it is the

literal transcription of the asymmetry Equation (6.2) already has.

6.2.1 The Running Example, Strand by Strand

Run $r_1 = (a + ab)(b + \varepsilon)$ against **ab** through this construction, checked against **cargo run - parse "(a|ab)(b|)" "ab" -parser pderiv_bc -diag 3** (Appendix C.4), so that it can be seen disagreeing with Chapter 5 on the same input, and not merely be asserted to do so. The frontier starts as one strand, the whole expression with no bits, which is the leftmost node of Figure 6.1.

The first character binds $r = a + ab$ and $s = b + \varepsilon$ at the outer node. That r is not nullable, by N_+ over a literal and a concatenation, so PB_{seq} applies and s contributes no strand at all. The strands that do appear come from PB_+ one level down:

$$\begin{aligned}
 D_{a+ab} &= \{(\varepsilon, [0])\} \cup \{(b, [1])\} && PB_+, r = a, s = ab \\
 D_{r_1} &= \{(smartC(\varepsilon, b + \varepsilon), [0]), (smartC(b, b + \varepsilon), [1])\} && PB_{seq}, n(a + ab) = false \\
 &= \{(b + \varepsilon, [0]), (b(b + \varepsilon), [1])\} && SC \text{ twice}
 \end{aligned} \tag{6.3}$$

The two applications of SC take different branches, which is why the two strands look so different. Against the first, $r' = \varepsilon$, so SC absorbs it and leaves $b + \varepsilon$ alone; against the second, $r' = b \neq \varepsilon$, so it keeps the concatenation. Since the outer node's $r = a + ab$ is not nullable here, this is the non-nullable branch of Figure 6.3's Seq arm, and both strands go through smart_seq. The trace agrees:

```

Step 1 (char: 'a', position 1)
  rule: pDerivBC('a', r1.r2) [nullable(r1)=false]
        = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
frontier after (2 residual(s)):
  [0] Alt(Lit('b'), Eps)    bits = [0]
  [1] Seq(Lit('b'), Alt(Lit('b'), Eps))    bits = [1]

```

Both bits were emitted by PB_+ one level down, and PB_{seq} carried them outward unchanged, which is why neither strand's bits grew at this step. Where the plain bit-coded construction of Chapter 5 kept the two alternatives inside one expression, joined by $\oplus$, they are now two separate objects that will be advanced independently.

The second character steps *both* strands, and each is matched against the rule set on its own, so two different rules fire on the same character:

$$\begin{aligned}
 D_{b+\varepsilon} &= \{(\varepsilon, [0])\} \cup \{\} && PB_+, r = b, s = \varepsilon; PB_\varepsilon \\
 D_{b(b+\varepsilon)} &= \{(smartC(\varepsilon, b + \varepsilon), [1])\} = \{(b + \varepsilon, [1])\} && PB_{seq}, n(b) = false
 \end{aligned} \tag{6.4}$$

Strand [0] is an alternation whose left disjunct contributes ε tagged 0 and whose right contributes nothing, since PB_ε returns the empty set. Strand [1] is a concatenation whose left factor is a bare literal, so PB_{seq} applies and only that factor is derived. Each strand's accumulated bits are its own prefix followed by whatever the step produced, so [0] gains a 0 and reaches [0,0] while [1] gains nothing and stays at [1]:

```

Step 2 (char: 'b', position 2)
  from [0]:
    rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++
      tag(1, pDerivBC('b',r2)), nub2
    -> (Eps, [0])
  from [1]:
    rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
      = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
    -> (Alt(Lit('b')), Eps), [])
frontier after (2 residual(s)):
[0] Eps    bits = [00]
[1] Alt(Lit('b')), Eps)    bits = [1]

```

Selection now scans this final frontier in list order and takes the first nullable strand. Strand [0], $(\varepsilon, [0,0])$, is checked and accepted before strand [1] is inspected at all, although [1] is nullable too:

```

selected residual's accumulated bits ++ mkEpsBC(residual) = [00]

```

```

decode((a|ab)(b|), [00]):
  = (Left a, Left b)

```

This is the leftmost-commits rule made mechanical. Strand [0] is the "matched **a**, then **b** separately" reading, the first alternative *pDerivBC* generated back at step one, and it wins purely because it comes first in the frontier, not because any length comparison favours it. The decoded tree is the *shorter* reading, and it is the opposite answer from Chapter 5's (*Right(a,b)*, *Right()*) on the identical expression and input. Nothing about r_1 or "ab" changed between the two traces. Only the policy governing which nullable candidate wins did.

Where nub2 is and is not applied. nub2 appears inside *pDerivBC*, once per case that can produce more than one strand, and nowhere else. In particular the frontier-driving loop concatenates the per-strand results into the next frontier without deduplicating across them, so the guarantee nub2 provides is local to a single strand's expansion and does not extend to the frontier as a whole. Two strands that were distinct at step i can converge on the same residual at step $i + 1$ and both survive. Section 6.4 measures what that costs.

The Driver. `step_frontier` and `select_bits` (`src/parsers/pderiv_bc/parse.rs`) implement the loop of Section 6.1. Figure 6.4 gives both, plus the entry point that calls them.

```

pub(super) fn step_frontier(frontier: &[(Regex, Vec<bool>)] , x: char)
    -> Vec<(Regex, Vec<bool>>>
{
    let mut out = Vec::new();
    for (r, bits) in frontier {
        for (r_next, extra) in pderiv_bc(r, x) {
            let mut combined = bits.clone();
            combined.extend(extra);
            out.push((r_next, combined));
        }
    }
    out
}

pub(super) fn select_bits(frontier: &[(Regex, Vec<bool>)] )
    -> Option<Vec<bool>>
{
    for (r, bits) in frontier {
        if nullable(r) {
            let mut result = bits.clone();
            result.extend(mk_eps_bits(r));
            return Some(result);
        }
    }
    None
}

pub fn parse_pderiv_bc(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier = vec![(r.clone(), Vec::new())];
    for c in input.chars() {
        frontier = step_frontier(&frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    let bits = select_bits(&frontier)?;
    Some(decode(r, &bits))
}

```

Figure 6.4: The frontier loop and its entry point. `step_frontier` replaces every (`residual`, `bits`) pair with the union of `pderiv_bc` applied to it, concatenating each result’s bits onto that strand’s accumulated prefix. `select_bits` scans in list order and returns the first nullable strand’s bits ++ `mk_eps_bits(residual)`.

`parse_pderiv_bc` drives the loop and passes the winning bit-string, together with the *original* `Regex` and never the derived residual, to `decode` (Section 5.3). That is the same shared decoder `deriv_bc` uses, since both constructions arrive at the identical (`Regex`, `bits`) shape by the time decoding is needed, whatever route each took to get there. The early return on an empty frontier is the same short-circuit Figure 3.8’s matcher uses, now

guarding a `Vec<(Regex, Vec<bool>)>` instead of a `HashSet<Regex>`.

6.3 The Standard Construction: `pderiv_std`

`pderiv_std` (`src/parsers/pderiv_std/`) is this project’s own construction, with no counterpart in either reference. Its purpose is to stand to `pderiv_bc` as `deriv_std` stands to `deriv_bc`: the same traversal, the same disambiguation policy and the same answers, but building a `ParseTree` directly instead of a bit-string that has to be decoded afterwards. Having both means this chapter’s answer is computed twice by routes sharing almost no code, which is what makes the agreement check of Section 6.5 worth anything.

Figure 6.5 redraws Figure 6.1 for this construction: same expression, same input, same shape, with injection closures standing in place of bits.

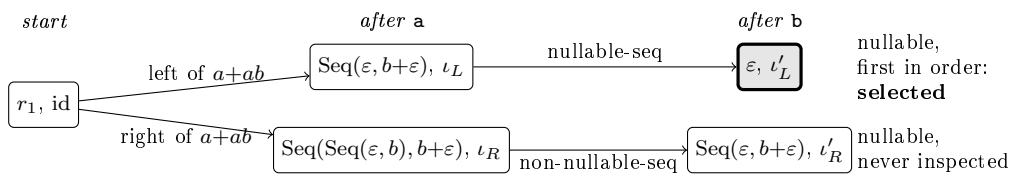


Figure 6.5: The same frontier as Figure 6.1, with each strand’s extra payload an injection closure ι instead of a bit prefix; Figure 6.6 shows what one such closure chain is built from. The winning residual at the end is ε , exactly as in Figure 6.1, but the intermediate residuals differ: where `pderiv_bc` reaches `Alt(Lit('b'), Eps)` after `a`, this construction reaches `Seq(Eps, Alt(Lit('b'), Eps))`, retaining the ε node `smart_seq` would collapse. Section 6.3 explains why.

6.3.1 What Each Strand Carries

Where `pderiv_bc` pairs each residual with a `Vec<bool>`, `pderiv_std` pairs it with an *injection closure*, defined in this module’s `inject.rs`. A strand’s `Inj` is a function that, given the parse tree of that strand’s *residual*, returns the parse tree of the *original* expression. The frontier starts with one strand carrying the identity closure, and each step composes the step’s own injection inside the strand’s accumulated one. Figure 6.6 contrasts the two representations after two steps.

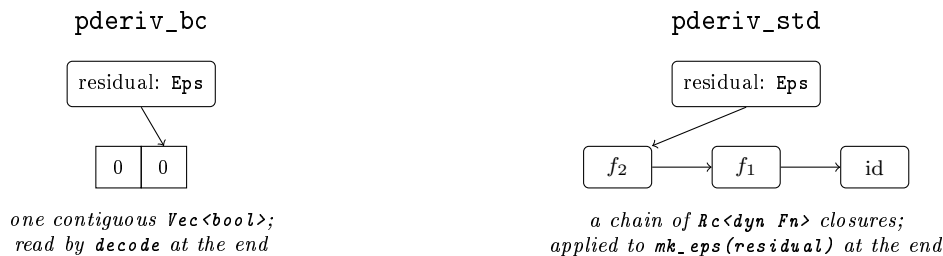


Figure 6.6: What one strand holds after two derivative steps, for the winning strand of Figure 6.1. The bit-coded strand accumulates into a flat buffer; the tree-building strand accumulates a chain of heap-allocated closures, one link per step, each holding the next. Both are consumed exactly once, on the winning strand, at the end of the input.

This is the same idea as Chapter 5’s *inj*, with one difference that matters. `deriv_std` applies `inject` once per character during an explicit backward pass, walking back along

the expression sequence as it unwinds; `pderiv_std` never runs a backward pass at all. It builds the composition forward, one layer per step, and applies the finished composition exactly once at the end. In that sense `pderiv_std` is as single-pass as `pderiv_bc`: the closure chain plays the role the bit-string plays there.

The `Rc` is required and not incidental. A single step can expand one strand into several, and every successor needs the same parent injection, so the parent has to be shared, not cloned or moved. `Rc<dyn Fn(...)>` gives shared ownership of a heap-allocated closure at the cost of a reference-count bump per successor. Section 8.5.2 measures what that costs against `pderiv_bc`'s flat `Vec<bool>`; the structural point here is the one Figure 6.6 draws, that a strand's accumulated state is a chain whose length grows with the input where `pderiv_bc`'s is a contiguous buffer.

In Rust. Figure 6.7 is the whole of `inject.rs`: the type, the identity closure the frontier starts with, and the `Inj`-carrying analogue of `nub2`.

```
pub(crate) type Inj = Rc<dyn Fn(ParseTree) -> ParseTree>;

pub(crate) fn identity_inj() -> Inj {
    Rc::new(|v| v)
}

pub(crate) fn nub_tree(strands: Vec<(Regex, Inj)>) -> Vec<(Regex, Inj)> {
    let mut seen: HashSet<Regex> = HashSet::with_capacity(strands.len());
    let mut out = Vec::with_capacity(strands.len());
    for (r, inj) in strands {
        if seen.insert(r.clone()) {
            out.push((r, inj));
        }
    }
    out
}
```

Figure 6.7: `Inj`, its identity value, and `nub_tree`. `nub_tree` is `nub2` (Figure 6.2) with `Vec<bool>` replaced by `Inj`; the deduplication logic itself, keep the first strand to reach a given residual, is unchanged.

`Rc<dyn Fn(ParseTree) -> ParseTree>` is the payload Section 6.3 named: a boxed, reference-counted closure. Cloning it bumps a counter; it does not copy whatever state the closure captured. `identity_inj` is `id`, the closure every strand in Figure 6.5's start column carries.

6.3.2 One Table, Two Representations

The module `pderiv_std/pderiv_tree.rs` is the `(Regex, Inj)` analogue of `pderiv_bc`, case for case, with each case's bit-tagging replaced by the tree constructor that bit would have decoded to:

Case	pderiv_bc attaches	pderiv_tree attaches
$\varepsilon, \emptyset$	no strands	no strands
x (matching)	<code>[]</code>	<code> _ Char(x)</code>
$r + s$, left	<code>0:bs</code>	<code> v Left(inj1(v))</code>
$r + s$, right	<code>1:bs</code>	<code> v Right(inj2(v))</code>
rs , continue r	<code>bs</code>	<code> Pair(v1,v2) Pair(inj1(v1), v2)</code>
rs , jump to s	<code>mkEpsBC_r ++ bs</code>	<code> v2 Pair(mk_eps(r), inj2(v2))</code>
r^*	<code>0:bs</code>	<code> Pair(v1,vs) Star(inj1(v1) : vs)</code>

Every row is the same decision, expressed once as a bit and once as a constructor. The correspondence is exact enough that the "jump to s " row calls the shared `mk_eps` (Section 5.2) at precisely the point `pderiv_bc` calls `mk_eps_bits`, and for the reason I_{seq}^R gave in Chapter 5: the left factor matched empty, so its contribution to the pair has to be synthesized and not derived.

In Rust: `pderiv_tree`. Figures 6.8 and 6.9 give the two rows of the table above that a bit alone cannot show: what the closure itself does, not just which bit it would have been. The `Alt` arm builds the table's third and fourth rows; the `Seq` arm builds the fifth and sixth, together with the non-nullable case the table left implicit.

```

Regex::Alt(r1, r2) => {
  let mut out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
    .into_iter()
    .map(|(r1_, inj1)| {
      let f: Inj =
        Rc::new(move |v| ParseTree::Left(Box::new(inj1(v))));
      (r1_, f)
    })
    .collect();
  out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
    let f: Inj =
      Rc::new(move |v| ParseTree::Right(Box::new(inj2(v))));
    (r2_, f)
  }));
  nub_tree(out)
}

```

Figure 6.8: The alternation arm of `pderiv_tree`, building the table's third and fourth rows.

```

Regex::Seq(r1, r2) => {
  if nullable(r1) {
    let mut out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
      .into_iter()
      .map(|(r1_, inj1)| {
        let r2c = (**r2).clone();
        let f: Inj = Rc::new(move |v: ParseTree| match v {
          ParseTree::Pair(v1, v2) =>
            ParseTree::Pair(Box::new(inj1(*v1)), v2),
          other => panic!(
            "pderiv_tree: Seq-continue strand expected \
              Pair, got {:?}", other),
        });
        (Regex::seq(r1_, r2c), f)
      })
      .collect();
    let v1_eps = mk_eps(r1);
    out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
      let v1c = v1_eps.clone();
      let f: Inj = Rc::new(move |v2: ParseTree| {
        ParseTree::Pair(Box::new(v1c.clone()), Box::new(inj2(v2)))
      });
      (r2_, f)
    }));
    nub_tree(out)
  } else {
    pderiv_tree(r1, x)
      .into_iter()
      .map(|(r1_, inj1)| {
        let r2c = (**r2).clone();
        let f: Inj = Rc::new(move |v: ParseTree| match v {
          ParseTree::Pair(v1, v2) =>
            ParseTree::Pair(Box::new(inj1(*v1)), v2),
          other => panic!(
            "pderiv_tree: Seq(non-nullable) strand \
              expected Pair, got {:?}", other),
        });
        (Regex::seq(r1_, r2c), f)
      })
      .collect()
  }
}

```

Figure 6.9: The sequence arm of `pderiv_tree`, built from the same case split as Figure 6.3's Seq arm with Inj closures in place of bit prefixes.

Read next to Figure 6.3, the shape is identical arm for arm: the same two-way split of `Alt`, the same `nullable(r1)` branch in `Seq`, the same second strand built from what r_1 contributes at ε . What changes is only what gets attached: a bit becomes a closure that pattern-matches the tree it will eventually receive and reassembles it. The `Seq` arm's two branches build visibly similar closures, the "continue into r_1 " shape appearing in both,

because both are inverting the same derivative step `inject`'s concatenation arm inverts in Chapter 5 (Figure 5.3); the difference is which one also carries a fresh `mk_eps(r1)` for the half of the pair the derivative skipped.

`nub_tree` is called in exactly the positions `nub2` is in Figure 6.3, deduplicating on the residual alone and keeping the first occurrence, so the priority order is preserved identically between the two constructions.

Why the Smart Constructor Cannot Be Used Here. `pderiv_bc` is free to apply *smartC* in its non-nullable `Seq` case and in its `Star` case, collapsing $\varepsilon \cdot s$ to s . `pderiv_tree` is not, and uses plain `Regex::seq` everywhere its sibling uses the smart one, visibly so in both branches of Figure 6.9's `Seq` arm. The reason follows directly from what an `Inj` is applied to.

Selection ends by calling `inj(mk_eps(r))` on the winning strand, so each accumulated closure is applied to a tree whose *shape is determined by its own residual*. The `Seq` strands' closures pattern-match on `ParseTree::Pair`, which is what `mk_eps` produces for a `Seq` residual. Collapsing `Seq(Eps, s)` to `s` would make `mk_eps` produce `s`'s own tree shape instead, and the closure expecting a `Pair` would receive something else. The residual and the injection are two halves of one invariant, and a smart constructor rewrites one half without the other.

This is not a speculative concern. Substituting `smart_seq` for `Regex::seq` in the two `Seq` strands of `pderiv_tree` makes the test suite fail immediately, on the simplest possible sequence pattern, with the panic Figure 6.9's own non-nullable branch raises:

```
---- parsers::pderiv_std::parse::tests::seq_matches stdout ----
thread '...::seq_matches' panicked at
  src/parsers/pderiv_std/pderiv_tree.rs:71:38:
pderiv_tree: Seq(non-nullable) strand expected Pair, got Char('b')
```

The panics in `pderiv_tree` therefore serve the purpose those in `inject` (Section 5.2) and `decode` (Section 5.3) serve: they encode a precondition that holds by construction and convert a violation into an immediate, located failure instead of a silently wrong parse tree.

The Driver. Figure 6.10 is the driver module, `pderiv_std/parse.rs`. It mirrors its `pderiv_bc` counterpart from Figure 6.4 one function at a time.

```

pub(super) fn step_frontier(frontier: Vec<(Regex, Inj)>, x: char)
    -> Vec<(Regex, Inj)>
{
    let mut out = Vec::new();
    for (r, inj_parent) in frontier {
        for (r_next, inj_step) in pderiv_tree(&r, x) {
            let parent = inj_parent.clone();
            let combined: Inj = Rc::new(move |v| parent(inj_step(v)));
            out.push((r_next, combined));
        }
    }
    out
}

pub(super) fn select(frontier: &[(Regex, Inj)]) -> Option<ParseTree> {
    for (r, inj) in frontier {
        if nullable(r) {
            return Some(inj(mk_eps(r)));
        }
    }
    None
}

pub fn parse_pderiv_std(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier: Vec<(Regex, Inj)> =
        vec![(r.clone(), identity_inj())];
    for c in input.chars() {
        frontier = step_frontier(frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    select(&frontier)
}

```

Figure 6.10: `step_frontier` expands every strand and composes injections; `select` scans in list order and returns `inj(mk_eps(r))` for the first nullable residual.

The one line worth pausing on is `step_frontier`'s closure composition, `Rc::new(move |v| parent(inj_step(v)))`: it wraps the *step*'s injection inside the *strand*'s accumulated one, so applying the combined closure runs `inj_step` first and `parent` second, undoing the derivative steps in the same last-taken-first-undone order Chapter 5's explicit backward pass does, except that here the undoing is compiled into a closure once and run only when, and if, that strand wins. `parse_pderiv_std` returns `None` as soon as the frontier becomes empty, since a frontier with no strands can never regain one. As with `pderiv_bc`, no deduplication is applied across the flattened frontier, only within each strand's own expansion.

The Same Example, and What the Trace Cannot Show. Running r_1 against `ab` (`-parser pderiv_std`, Appendix C.3) produces the same two-strand fork at step one and the same winner at step two that Figure 6.5 draws. Two differences from `pderiv_bc`'s trace are visible, and both were predicted by the smart-constructor argument above.

The first is the residuals themselves. Where `pderiv_bc` lists `Alt(Lit('b'), Eps)`, this parser lists `Seq(Eps, Alt(Lit('b'), Eps))`; where that one lists `Seq(Lit('b'), ...)`, this one lists `Seq(Seq(Eps, Lit('b')), ...)`. The ε its sibling discards is exactly the one whose `Pair` shape the closure needs.

The second follows from the first, and is sharper: because the residuals differ, the two parsers do not even take the same rules. At step two, `pderiv_bc`'s strand `[0]` is an `Alt` and fires `PB+`, while `pderiv_std`'s is a `Seq` whose left factor is the retained `Eps`, so it fires the *nullable* sequence rule instead:

```
rule: pderiv_tree('b', r1.r2) [nullable(r1)=true]
    = nub_tree $ [(r1'.r2, inj: continue r1) | ...]
      ++ [(r2', inj: mk_eps(r1) paired with r2's
              result) | ...]
```

The two parsers are therefore not running over the same residuals, and not applying the same rules to them, and they still select the same tree. Because `nub_tree` deduplicates on residual syntax, they can in principle prune at different points as well, which makes their agreement on every tested case a stronger check than it would be if they shared a residual representation.

One thing the trace cannot show is what each strand would build if it won:

```
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not
    printable)
```

A `Vec<bool>` can be printed; an `Rc<dyn Fn>` cannot. The trace reports which residuals are alive and which rule produced each, but not the tree each carries, which is precisely the information `pderiv_bc`'s `bits = [00]` annotations do carry. That is a direct cost of the representation choice in Figure 6.6: a trace can only report state the construction actually keeps, and this one does not keep the bits.

6.4 Frontier Growth

Both parsers inherit Antimirov's finiteness result for the set of distinct partial derivatives (Section 3.3), and neither inherits a bound on the size of the frontier, for the reason Section 6.1 identified: the frontier holds `(residual, extra)` pairs, and only the residual component is subject to *pd*'s bound. This section measures the consequence.

Take $(a + (b + ab))^*$, the `paper_r2` expression, against $(ab)^n$, and record both the number of strands in the frontier after each character and the number of *distinct* residuals among them.

Every strand beyond the second is a duplicate of one already present, differing only in the bits or injection it carries. `nub2` does not remove them because it never sees them together: it runs inside *pDerivBC*, on the successors of one strand, and these duplicates arise *between* strands, at the point `step_frontier` concatenates the per-strand results.

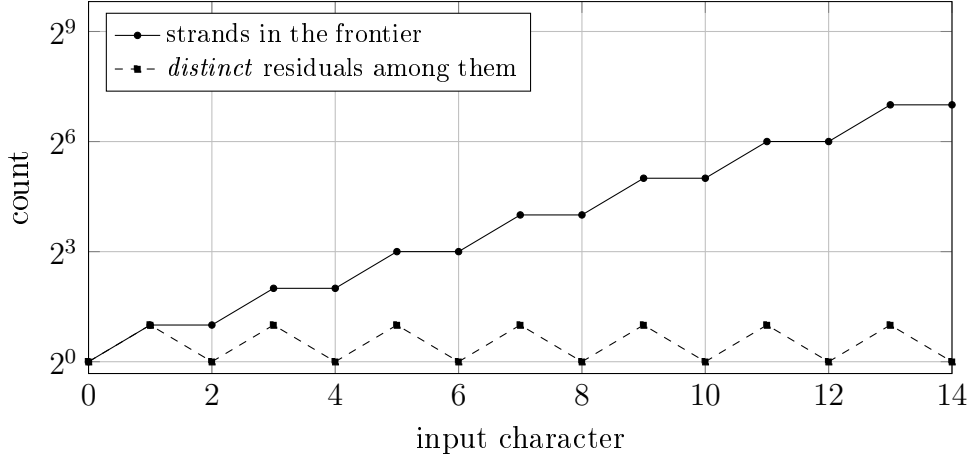


Figure 6.11: Frontier size against distinct residual count for $(a+(b+ab))^*$ on $(ab)^n$, logarithmic vertical scale. The frontier doubles every two characters while the number of distinct residuals never exceeds two: every strand beyond the second is a duplicate carrying different accumulated state. Regenerate with `cargo run --release --example thesis_figures -- frontier; tests/test_thesis_figures.rs` pins these values.

This is the partial-derivative counterpart of Section 5.3’s finding for `deriv_bc`, and the two have one root cause seen from different sides. There, alternation branches that were structurally identical survived deduplication because their bit annotations made them syntactically distinct. Here, residuals that are syntactically identical survive because deduplication is never applied at the level where they meet. In both cases the bounded object is the set of residual *shapes*, and in both cases what actually grows is the set of shapes paired with accumulated parse-tree state.

Deduplicating across the frontier. The obvious question is whether the duplicates can simply be dropped. Moving `nub2` out of `pDerivBC` and applying it across the whole flattened frontier, keeping the first occurrence, is a change of one deduplication site, and it preserves the priority order for the same reason `nub2` does inside a single expansion: the frontier is built in priority order, so the first occurrence of a residual is the highest-priority strand that reaches it.

Whether that argument is enough to preserve the selected *tree* is a different question, and this thesis does not prove it. It was tested instead, by running both variants over the 300 generated expressions and 39 inputs of Section 8.4.2, 11 700 cases in total. The two agree on every case, on membership and on the exact tree returned, and the peak frontier over the whole run falls from 170 strands to 4.

The shipped `pderiv_bc` nonetheless keeps `nub2` where the reference puts it, for the reason given in Section 6.2: this parser’s correctness criterion is literal agreement with `pDerivBC`, and a change verified on 11 700 cases but not proved is a change to the thing being verified against. Section 8.7 records it as an available optimization and not an applied one, with the measurement above as the evidence for its value.

6.5 The Two Policies Compared

Section 4.3 constructed by hand the *second* valid derivation of $r_1 = (a + ab)(b + \varepsilon)$ against **ab** (paper_r1) that Chapter 5 set aside: $v' = (Left\ a, Left\ b)$, matching the *first* alternative of $a + ab$ against **a** alone instead of the second against **ab** in one step. That is this chapter's result, and both of its parsers have now been shown computing it. Figure 6.12 sets the two answers side by side.

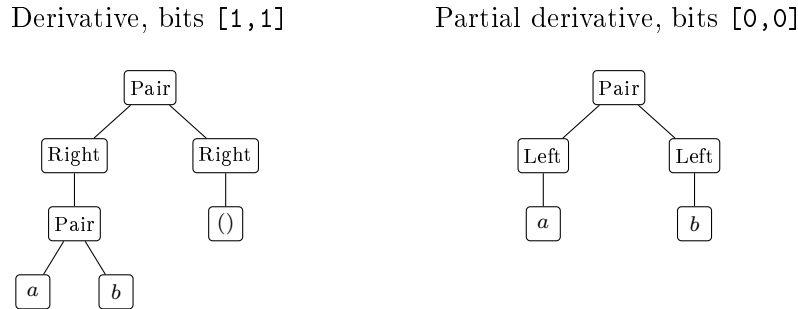


Figure 6.12: The two parse trees for $(a + ab)(b + \varepsilon)$ against "ab". Both flatten to "ab" and both are valid. The derivative parsers take the second alternative of $a + ab$, consuming both characters at once and leaving $b + \varepsilon$ to match empty; the partial-derivative parsers take the first, leaving $b + \varepsilon$ to consume the **b**.

The bit-strings underneath make the comparison exact. Four alternation choices exist in r_1 , two of which are reached on any given parse, and the two parsers disagree on both: $[1,1]$ against $[0,0]$. Nothing else differs. The same expression, the same input, the same membership answer, and the entire distinction between the two chapters compressed into two bits.

Failure behaviour is shared. Against **b** instead of **ab**, the first partial derivative of r_1 produces no residual at all, since neither alternative of $a + ab$ begins with **b**:

```
rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
    = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
-> []
frontier after (0 residual(s)):
no nullable residual in this frontier [FAIL]
```

An empty frontier is the partial-derivative counterpart of `deriv_bc` simplifying to `phi` (Section 5.3), and it is detected the same way but earlier: both parsers return as soon as the frontier empties, without consuming the rest of the input, because a frontier with no strands has no way to acquire one. For a long input whose first character already fails, that is the difference between one step and n .

Both then emit the same error summary as Chapter 5's two, character for character:

```
Failure at position: 1 (1-indexed)
Found:               'b'
Expected:            'a'
```

The two chapters' constructions disagree, deliberately, on which tree a successful match produces, and agree exactly on the Boolean question of membership and on the diagnostics

reported for a non-match. Neither the disambiguation policy nor the choice between bits and closures is ever reached until the underlying walk has already found at least one live residual. The full reports are Appendix C.7 and Appendix C.8.

All five entry points at once. The `-parser all` mode runs every parser on one input and checks the three invariants this and the previous chapter claim, with no diagnostics level selected:

```
$ cargo run -- parse "(a|ab)(b|)" "ab" --parser all
Regex: (a|ab)(b|)
Input: "ab"
```

Parser	Policy	Result
DERIV_REC	POSIX	(Right (a, b), Right ()) -> "ab"
DERIV_LOOP	POSIX	(Right (a, b), Right ()) -> "ab"
DERIV_BC	POSIX	(Right (a, b), Right ()) -> "ab"
PDERIV_STD	NON-POSIX	(Left a, Left b) -> "ab"
PDERIV_BC	NON-POSIX	(Left a, Left b) -> "ab"

```
POSIX parsers agree
PDERIV parsers agree
NON-POSIX agrees with POSIX on membership
```

The three closing lines are the invariants themselves, checked on this input: the three derivative entry points must return identical trees, the two partial-derivative ones must return identical trees, and all five must agree on whether the input matched. Nothing requires the two *policies* to agree, and on this input they do not, which is the entire subject of both chapters. Section 8.4 runs the same check at scale.

What the substitution cost. Replacing $r \setminus x$ with $pd(r, x)$ changed three things, and only the first was intended. The representation changed, from one expression to a frontier of strands. The bound changed: Antimirov's theorem bounds the residual shapes, and Section 6.4 showed the frontier growing anyway, because what the algorithm carries is shapes paired with parse-tree state. And the answer changed, because the left-preference that *mkEps* applied to a single alternation had nowhere to live in a set, and *nub2*'s first-occurrence-wins rule took its place. The last of these is not a consequence of partial derivatives as such. It is a consequence of how this particular construction keeps its frontier finite, and Section 6.1 set out why a construction of this shape has little room to do otherwise.

7 External versus Internal Regex

Every example so far in this thesis has been written directly in `Regex` terms, the six-constructor core algebra of Section 2.3 that `deriv` and `pderiv` operate on. No real regular-expression user writes that way. A Snort rule, a SpamAssassin filter, or an ordinary call to `cargo run - match` arrives as an ordinary PCRE-style pattern *string*, `"[a-c]+\d{2}"` rather than `Seq(Star(...), ...)`. `src/frontend/` is the module that closes this gap: it parses that string into its own surface-syntax AST, then lowers that AST into the `Regex` every parser in Chapters 5 and 6 already understands.

It is modeled throughout on the `ExtPattern`, `Parse` and `Translate` modules of `Text.Regex.PDeriv`, from `luzhuomi/regex-pderiv`, Kenny Lu's own reference implementation of Sulzmann and Lu [2012]'s partial-derivative submatching. That is a second, independent reference alongside Sulzmann and Lu [2014]'s own teaching-page material, used here specifically because the `flops14`-extended paper itself has nothing to say about surface syntax: POSIX parsing is defined directly over the same six-constructor `Regex` algebra this thesis's core inherits, with no PCRE-facing frontend of its own.

Section 8.3's real-world corpus depends on everything described here, surface-syntax parsing, desugaring, and the substring-search padding scheme, to turn raw Suricata, SpamAssassin and `RegexLib` rule text into `Regex` values in the first place. The chapter closes (Section 7.4) with the price that translation charges: how large a `Regex` an ordinary-looking pattern actually becomes, which is what the four parsers then have to walk once per input character.

7.1 Two ASTs, One Lowering Step

`src/frontend/` is built around a deliberate separation into two abstract syntax trees connected by a single function, rather than parsing pattern text directly into `Regex`. Figure 7.1 gives the whole module in one view: a common path that every entry point follows, and two optional stages that only some of them switch on.

`ExtPat` (`ext_pattern.rs`) is the *external*, surface-syntax AST: marking and non-marking groups, `{m,n}` bounds, `\d/\w/\s` shorthands, `^/$` anchors, and a `greedy`: `bool` flag on every quantifier (`a*` against `a*?`). It holds everything a PCRE-style string can express, kept exactly as written, with nothing desugared yet. `Regex` (Section 2.3) is the *internal*, six-constructor core every parser derives or partial-derives directly: `Phi` | `Eps` | `Lit` | `Seq` | `Alt` | `Star`, with no groups, no bounds, no anchors and no laziness flag. Anything `ExtPat` can say that `Regex` cannot represent is desugared away by `translate` (Section 7.3) or, for `^` and `$` specifically, erased into `Eps` and handled one level up instead (Section 8.3.1).

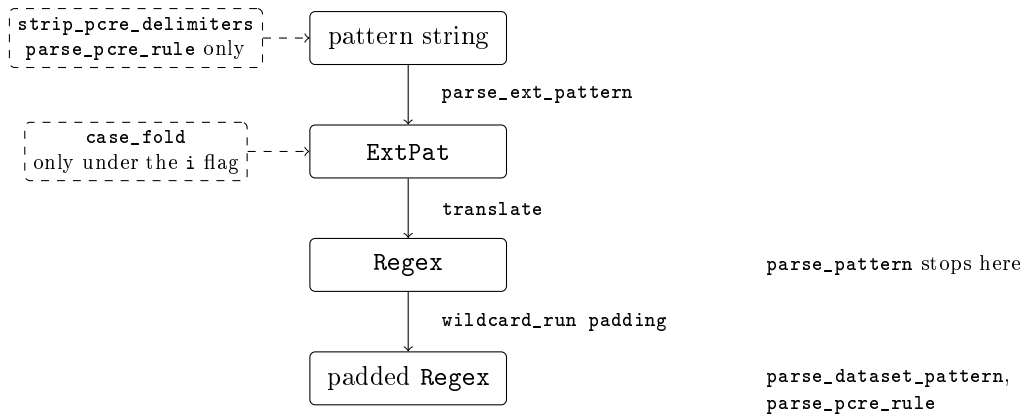


Figure 7.1: The frontend as a pipeline. The solid path is common to all three entry points; the dashed stages apply only where marked. `parse_pattern` stops at `Regex` and means full-string equality, while `parse_dataset_pattern` and `parse_pcre_rule` continue into the padding of Section 8.3.1.

Keeping `ExtPat` as an honest AST in its own right, rather than routing straight from the parser into `Regex` in a single pass, is what makes `translate` independently testable. The unit tests in `translate.rs` construct `ExtPat` values by hand and check their `Regex` output directly, with no dependency on the recursive-descent logic in `parse.rs` at all. It is the same separation-of-concerns argument that Section 5.2 makes for keeping `ParseTree` construction separate from `flatten` and `decode`, applied one layer further out, at the point where a PCRE-style string first becomes structured data at all.

The separation earns its keep a second time at the entry points. `mod.rs` exposes three of them, all sharing one parser and one `translate`, and differing only in what they do to the `ExtPat` in between:

Entry point	Behaviour
<code>parse_pattern</code>	<code>translate</code> alone: full-string match
<code>parse_dataset_pattern</code>	plus anchor-aware padding (Section 8.3.1)
<code>parse_pcre_rule</code>	plus <code>/.../i</code> delimiter stripping and case folding

Every example in Chapters 5 and 6, and every invocation of `cargo run - parse`, goes through `parse_pattern`, which is why `"(a|ab)(b|)"` against `"ab"` means the whole input and not a substring of it. The corpus of Section 8.3 goes through the other two, which is where real rule text is handled.

7.2 Parsing Surface Syntax

`frontend/parse.rs` is an ordinary recursive-descent parser over a `Chars` iterator, structured directly after the grammar of `Text.Regex.PDeriv.Parse`. Its productions correspond one for one, each handling exactly the surface-syntax layer its reference counterpart does:

Reference	This frontend	Layer handled
<code>p_ere</code>	<code>parse_or</code>	alternation
<code>p_branch</code>	<code>parse_concat</code>	concatenation
<code>p_exp</code>	<code>parse_postfixed</code>	postfix quantifiers
<code>p_atom</code>	<code>parse_atom</code>	a single atom
<code>p_group</code>	<code>parse_group</code>	(...) forms
<code>p_charclass</code>	<code>parse_charclass</code>	[...]
<code>p_enum</code>	<code>parse_class_enum</code>	class members
<code>p_bound</code>	<code>parse_bound</code>	{m,n}

The table is a list of names; Figure 7.2 is what those names mean operationally. Each production consumes exactly one syntactic layer and then calls the next one down, so the order of the calls *is* the precedence order, and the single back-edge from `parse_group` is what makes the grammar recursive at all.

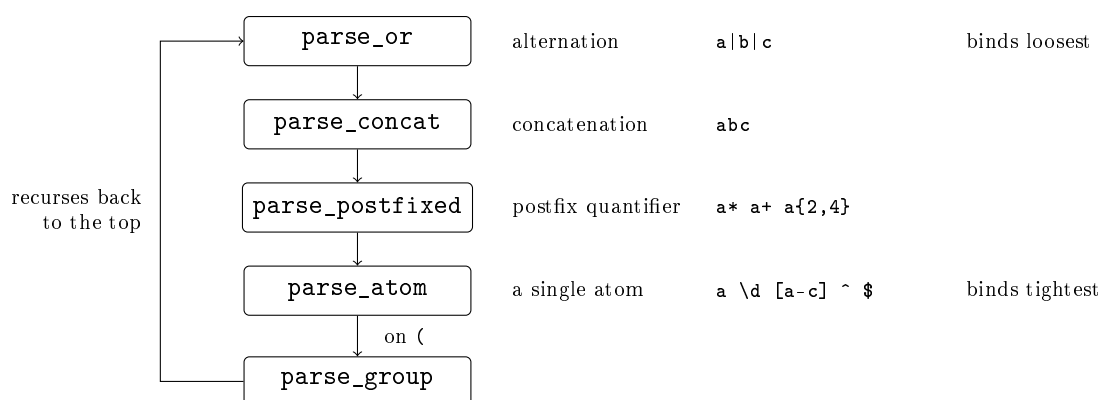


Figure 7.2: The recursive-descent ladder of `frontend/parse.rs`. Each production strips one layer and delegates the rest downward, so precedence falls out of the call order. `parse_group` is the only production that climbs back to the top, which is why parentheses override precedence and why nesting is unbounded.

Four deliberate departures from the reference:

- **Class shorthands are PCRE character classes, not literal escapes.** The reference’s `Translate.hs` reads a bare `EEscape c` as the literal character `c` always; this frontend special-cases `d/D/w/W/s/S` into character classes (Section 7.3’s `alphabet.rs`) instead, since real datasets use them that way throughout.
- **Backreferences and lookahead are rejected outright.** The backreferences `\1` to `\9`, and the four lookahead openers `(?=`, `(?!`, `(?<=` and `(?<!`, all produce a descriptive parse error rather than the reference’s literal-digit or three-digit octal-ASCII escape reading. Both describe languages that are not regular in the formal sense this thesis’s entire construction depends on, so silently mis-parsing them into *some* `Regex` would be a worse outcome than refusing them, and Section 8.3’s extraction pipeline depends on this refusal being loud and categorized rather than silent.
- **Named groups are accepted.** `(?<name>...)` and `(?P<name>...)` parse successfully, and the name is dropped during `translate`. A name carries no meaning once lowered to `Regex`, since this project has no capture-group semantics at all, but rejecting a construct this common in real Suricata and `RegexLib` rules purely be-

cause the reference parser predates it would reject working patterns for no benefit. (`?P=name`), a named *backreference*, is rejected along with the numeric ones.

- **\xHH hex escapes are accepted**, another addition absent from the reference, for the same reason.

Two small extensions exist for this thesis's own sake rather than for the datasets. A literal ε character in the pattern text parses to `ExtPat::Eps`, and a literal $\emptyset$ to `ExtPat::Never`, so that Section 2.1's own notation for ε and $\emptyset$ can be written directly on the command line. PCRE syntax has no way to spell either, and the alternative used throughout Chapters 5 and 6 is an empty alternative: `(b|)` is $b + \varepsilon$, because `parse_concat` returns `ExtPat::Empty` when a branch is empty rather than failing.

Bounds are parsed speculatively. `parse_bound` is the one place where the parser backtracks. A `{` in a pattern is only sometimes the start of a repetition bound, so `parse_bound` clones the iterator, tries to read a complete `{m}`, `{m,}` or `{m,n}` specification from the clone, and commits the clone back only on success. On failure the `{` is left unconsumed, and since `{` is in the parser's `SPECIALS` set, the next `parse_atom` call reports it as unexpected.

This has a consequence worth recording, because it makes one check in `translate` unreachable. An inverted bound such as `a{4,2}` is rejected by `try_parse_bound_spec`, which returns `None` when the upper bound is below the lower one, so the error the user sees comes from the *atom* parser:

```
$ cargo run -- parse "a{4,2}" "aa"
Regex parse error: unexpected special character '{' here
```

rather than from `desugar_bound`'s own "upper bound is less than lower bound" message, which no input reaching it through `parse_pattern` can trigger. The message is less specific than the one that exists for the condition, which is worth knowing when reading extraction-failure categories in Section 8.3.

`parse_bound` additionally accepts a lazy-quantifier marker (`a*?`, `a+?`, `a{2,4}?`) and records it as `ExtPat`'s `greedy`: `bool` flag: correctly read, but, as Section 7.3 explains, not honored by `translate`. Grouping, alternation and concatenation otherwise follow the reference's precedence exactly: alternation binds loosest, then concatenation, then postfix quantifiers, then a single atom, which may be a literal, an escape, a character class, a group or an anchor.

7.3 Translating ExtPat into Regex

`translate` (`frontend/translate.rs`) is a single structural recursion over `ExtPat`, one match arm per constructor, each either a direct one-to-one mapping or an explicit desugaring:

ExtPat	Regex
Empty, Eps	Eps
Never	Phi
Char(c)	Lit(c)
Group/GroupNonMarking(r)	translate(r) (grouping is precedence-only)
Or([r1, ..., rn])	left-folded Alt chain
Concat([r1, ..., rn])	left-folded Seq chain
Opt(r)	Alt(translate(r), Eps)
Plus(r)	Seq(translate(r), Star(translate(r)))
Star(r)	Star(translate(r))
Carat, Dollar	Eps (see Section 8.3.1)
Dot, Any(cs), NoneOf(cs)	sorted, deduplicated Alt of Lit
Escape(c)	class shorthand, or Lit(c)
Bound(r, lo, hi)	unrolled exact/optional repetition, below

The left-folding of `Or` and `Concat` is what makes the associativity convention of Section 2.1 concrete: `abc` becomes `Seq(Seq(a, b), c)`, and it is that tree, not the string, that every derivative in this thesis is taken of.

Character classes are ordinary alternation, not a primitive. `Any`, `NoneOf`, `Dot` and the class-shorthand escapes all route through `alt_of_chars`, which sorts and deduplicates the character set and left-folds it into a plain `Alt` chain of `Lit` nodes. `alphabet.rs` supplies the underlying sets: `alphabet()`, which is 98 characters (printable ASCII plus tab, newline and carriage return), `digit_chars`/`word_chars`/`space_chars` for `\d`/`\w`/`\s`, and `complement` (the alphabet minus a given set) for their negated and uppercase counterparts and for `[^...]`.

Every one of the four parsers only ever needs to understand `Regex`'s original six constructors, at the cost of a `\d` inside a real pattern becoming a ten-way `Alt` that the parser walks one branch at a time on every character. Section 7.4 measures how large that cost actually is.

Two consequences of drawing the alphabet this way are worth stating, since both are departures from PCRE that a rule author would not expect. The alphabet is ASCII only, so a negated class `[^a]` denotes 97 specific characters rather than "every character except `a`", and a non-ASCII input character matches no class at all. And `alphabet()` *includes* newline, so `.` here matches a newline where PCRE's `.` by default does not.

Bounds unroll literally, capped against blowup. `desugar_bound` builds `r` repeated exactly `lo` times, via `repeat_exact`, a left-folded `Seq` chain seeded with `Eps`, then appends either `Star(r)` for an unbounded upper end (`r{2,}`) or `hi - lo` further nested optional copies for a bounded one (`r{2,4}`), each layer of `optional_extra` being `Alt(Eps, Seq(r, ...))`. An exact bound `r{2}` takes a third path, returning `repeat_exact` alone with nothing appended. The `Eps` seed stays in the result rather than being smart-constructed away, so `a{2}` lowers to `Seq(Seq(Eps, Lit('a')), Lit('a'))` rather than `Seq(Lit('a'), Lit('a'))`. That costs one node and one nullable-test per derivative step, and it keeps `repeat_exact` a single unconditional fold with `Eps` as the natural identity for `lo == 0`.

Unlike every other desugaring in this table, this one can make a pattern arbitrarily large from a short piece of source text. `r{1586,}`, which is Section 8.3's own example of a

rejected real-world bound, would otherwise unroll into 1586 nested `Seq` nodes before the `Star` even starts. `MAX_BOUND`, at 1000 repetitions, is this project's own safety limit rather than something carried over from the reference, and `translate` refuses any bound past it with a descriptive error rather than silently building an oversized `Regex`:

```
$ cargo run -- parse "a{1586,}" "aa"
Regex parse error: bound {1586,} exceeds this frontend's cap of
1000 repetitions (literal unrolling would blow up the regex size)
```

Greedy and lazy quantifier syntax is parsed but not honored. Every `Opt`, `Plus`, `Star` and `Bound` match arm above ignores the `greedy` flag that `parse_bound` (Section 7.2) recorded, because `Regex` has no per-operator laziness concept at all. POSIX-versus-Greedy disambiguation in this thesis is a whole-parser policy, `deriv_std/deriv_bc` against `pderiv_std/pderiv_bc` (Section 4.3), rather than a per-quantifier one. `a*` and `a*?` therefore produce identical `Regex` values.

A pattern mixing lazy and greedy quantifiers, expecting the per-operator control PCRE engines provide, will parse successfully but not behave as its author intended. This is a genuine expressiveness gap between the surface syntax this frontend accepts and what `Regex` can distinguish, noted here rather than left to be discovered. The flag is nonetheless carried on `ExtPat` rather than discarded at parse time, so that the information survives to the one layer that could act on it if per-quantifier policy were ever added.

Case folding. `parse_pcre_rule` handles a `/PATTERN/FLAGS` wrapper, the form Suricata and SpamAssassin rules are written in. `strip_pcre_delimiters` splits the body from the flags, passing a pattern through unchanged if it is not `/`-delimited, and the `i` flag triggers `case_fold` on the `ExtPat` before `translate` runs. `case_fold` rewrites each literal letter into a two-way `Any`, so `a` becomes `[aA]`, and leaves digits, punctuation and class shorthands untouched, since those are already case-agnostic. Folding at the `ExtPat` layer rather than after lowering is what keeps this to a single recursion: at the `Regex` layer the same change would have to distinguish literals that came from a class from literals that came from source text, a distinction `translate` has by then already erased.

7.4 The Cost of Translation

Every desugaring in this chapter is exact, and several of them are expensive. Since the object that reaches Chapters 5 and 6 is the lowered `Regex` rather than the pattern string, the size of that object, and not the length of the pattern, is what the parsers' per-character work scales with. Counting nodes in the lowered result for a few patterns makes the translation's real price visible:

The purely structural desugarings stay small, and their shapes are worth seeing written out, since each is exactly what Section 7.3's table prescribes:

Pattern	Lowerts to	Nodes
<code>a*</code>	<code>Star(Lit)</code>	2
<code>a?</code>	<code>Alt(Lit, Eps)</code>	3
<code>a+</code>	<code>Seq(Lit, Star(Lit))</code>	4
<code>a{2}</code>	<code>Seq(Seq(Eps, Lit), Lit)</code>	5
<code>a{2,}</code>	exact copies then <code>Star</code>	8
<code>a{2,4}</code>	exact copies then nested options	15

Character classes and search padding are what change the order of magnitude. A `\d` is a 10-way `Alt` chain at 19 nodes, `\w` a 63-way one at 125, `[^a]` a 97-way one at 193, and `.` a 98-way one at 195. An unanchored search for the three-character literal `abc` reaches 399.

That last figure is the one that matters most for Section 8.3. Searching for a three-character literal builds a 399-node expression, because each `wildcard_run()` is a 195-node alternation under a `Star` and two of them are needed. Every derivative step then walks that structure.

Figure 7.3 puts these on one logarithmic axis, together with two patterns closer to what the corpus actually contains. The span is the point: the same six constructors carry expressions differing by more than two orders of magnitude, and nothing about the pattern’s source length predicts where in that range it lands.

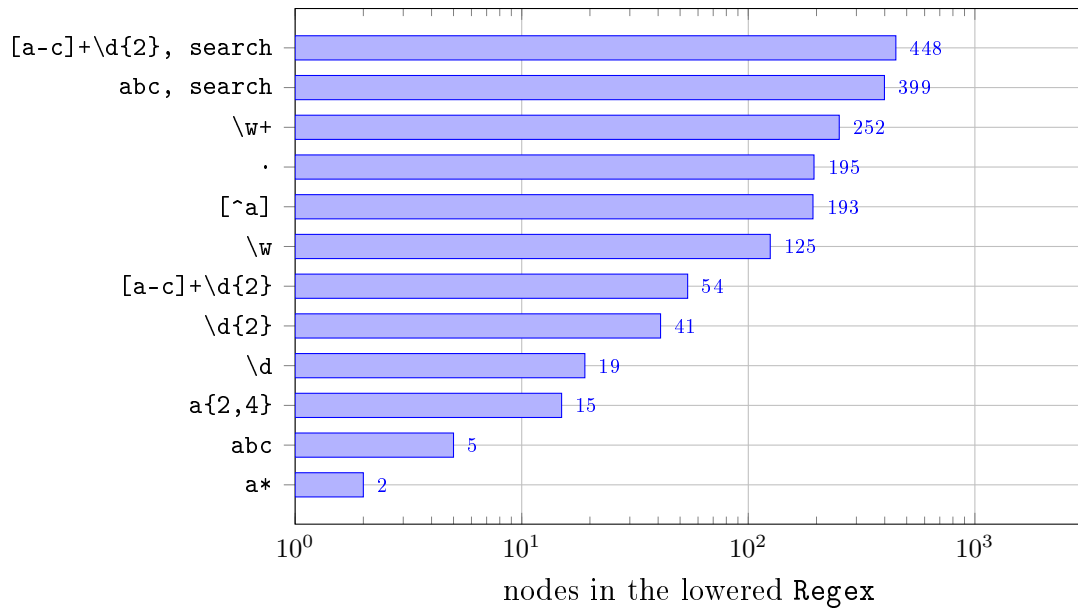


Figure 7.3: Lowered node counts, logarithmic horizontal axis. The three smallest are plain structural desugarings, with no character class involved. Every entry from `\d` upward expands one, and the two `search` entries additionally carry the padding of Figure 8.1. A realistic corpus pattern therefore starts around 450 nodes before any input is read. Regenerate with `cargo run --release --example thesis_figures -- lowered`.

This is the single largest constant factor separating this implementation from an engine with a native character-class representation, and it applies to the corpus patterns uniformly, since essentially all of them are unanchored searches containing at least one class shorthand.

The expansion is also visible in the diagnostics, in a way that is unhelpful rather than merely large. Because the expected-set reported on a failure is read off the first-literal

set of the expression at the point of failure, and a lowered class *is* that set, a single mismatched character against `.` reports all 98 alternatives individually:

```
$ cargo run -- parse "." "ä" --diag 1
Regex:  .
Input:  "ä"
Match:  false
Error:  position 1: found 'ä', expected '\t', '\n', '\r', ' ',
        '!', '"', '#', '$', '%', '&', ... , '}', or '~'
```

(abbreviated here; the tool prints the full list). The message is accurate, and it is exactly what the pattern denotes after lowering, but it no longer resembles the pattern the user wrote. A native class representation would let the diagnostics layer report **expected: any character** instead. It is also an instance of a general limit: once an abstraction has been compiled away, no amount of tracing can recover it, because the information is no longer in the object being traced.

The effect compounds rather than merely adding, which is why the chart climbs faster than the patterns lengthen. `\d{2}` lowers to two copies of the 19-node digit chain, reaching 41 nodes, and `\w+` to a 63-way alternation appearing twice, once for the required copy and once inside the **Star**, reaching 252. Stacking both effects, `[a-c]+\d{2}` is 54 nodes as a full-string pattern and 448 once padded for search, from twelve characters of source text.

Case folding is the one remaining multiplier, and it is comparatively cheap: `case_fold` turns each literal *letter* into a two-way **Alt** and leaves everything else alone, so `/abc/i` lowers to 405 nodes against `/abc/`'s 399, six more for three folded letters. Unlike the class expansion, this one scales with the number of letters in the pattern rather than with the size of the alphabet.

None of this changes which language is denoted, and none of it changes which parse tree any of the four parsers selects. It changes only how much structure each of them traverses per input character, which is exactly the quantity Section 8.5 measures.

8 Evaluation

Chapters 5 and 6 developed four parser combinations – `deriv_std`, `deriv_bc`, `pderiv_std`, `pderiv_bc` – each claimed to compute either the POSIX or the leftmost-first parse tree, by four structurally different constructions. This chapter verifies those claims empirically: Section 8.2 describes how the test suite checks the four parsers, and the three Boolean matchers of Chapter 3, against each other and against the paper’s own worked examples; Section 8.3 describes how a corpus of real-world regular expressions was assembled to test and benchmark against, alongside the synthetic patterns used elsewhere in this thesis; Section 8.4 reports the results of that testing; and Section 8.5 measures where the four parsers’ costs actually diverge from each other, on both the synthetic and real-world workloads.

8.1 Compared Engines

The primary object of comparison in this chapter is the four parser combinations against *each other*: two built on the derivative and two on the partial derivative, one tree-building and one bit-coded within each pair (Chapters 5-6) – since that internal comparison is what this thesis’s own theoretical claims (POSIX-correctness of the derivative parsers, agreement of all four on membership, the tree-versus-bit-coded performance trade-off) are actually about. A second, external comparison against established regular expression engines is motivated in Chapter 1.1 and positions these results relative to the wider field. It is implemented as an optional, feature-gated addition to this codebase (Rust’s `regex` crate as an ordinary optional dependency; Google’s RE2 via a small C++ shim over its own API, since RE2 offers no C API of its own) rather than a default one, specifically so that the default build stays free of any external regular-expression engine dependency – `cargo build/cargo test/cargo bench` never touch either engine unless the `external-engines` feature is explicitly enabled. Section 8.6 reports and discusses the resulting numbers; what follows briefly positions both engines conceptually, since that section refers back to this framing.

8.1.1 Rust’s `regex` Crate

Rust’s `regex` crate compiles a pattern to a Thompson-construction NFA and simulates it without backtracking, guaranteeing linear-time matching in the input length independent of the pattern – the same automaton-based guarantee Chapter 1.1 attributes to finite-automata approaches generally. Its disambiguation policy is leftmost-first, matching Section ??’s Greedy order rather than POSIX leftmost-longest; it offers no POSIX mode. That makes it the natural external comparison for this thesis’s partial-derivative

parsers (`pderiv_std/pderiv_bc`), which are also leftmost-first, on performance. Whether the two agree on the parse tree as well is a question this thesis does not settle: the partial-derivative construction is left-biased and commits on first success (Section 6.1.2), which is not by itself a claim that it implements the same policy the crate does.

8.1.2 RE2

Google’s RE2 is likewise automaton-based (Thompson NFA compiled lazily to a DFA on demand), explicitly designed to avoid the exponential backtracking blowup of backreference-supporting engines [Google LLC, 2024, Cox, 2007]. Unlike Rust’s `regex` crate, RE2 supports an explicit POSIX leftmost-longest mode alongside its default Perl-style leftmost-first mode, making it a comparison point for *both* disambiguation policies studied in this thesis, in one engine, depending on which mode is selected.

8.2 Testing Strategy

The test suite is split, following ordinary Rust convention, into unit tests (`#[cfg(test)] mod tests` blocks inside the `src/` files they exercise, testing private helper functions alongside public ones) and integration tests (`tests/`, exercising only the public API each parser or matcher module exposes). `tests/common/mod.rs` factors out what is shared across the integration suites: the two worked examples from the paper, `paper_r1` and `paper_r2` (Section 4.2), and two assertion helpers, `assert_round_trip` (checking `flatten(parse(r,w)) == w`, i.e. that a produced parse tree is actually evidence for the word it was built from) and `assert_parsers_agree` (comparing two `Option<ParseTree>` results with a labeled failure message identifying which two implementations disagreed).

8.3 Dataset Preparation

The synthetic patterns used throughout this thesis – `paper_r1`, `paper_r2`, the pathological (a^*) -style expressions of Section ?? – are deliberately small and hand-constructed, chosen to isolate one specific algorithmic property at a time. They say little about how any of the four parsers behaves on the regular expressions people actually write. Mamouras et al.’s disambiguation-robustness study [Mamouras et al., 2024] motivates closing that gap: it studies exactly the POSIX-versus-Greedy question this thesis studies, via real Snort/-Suricata intrusion-detection signatures, and finds concrete real-world patterns where the two policies visibly diverge. That paper provides no public dataset artifact of its own, so this thesis’s corpus was independently assembled from three families of real-world regular expression sources chosen to cover different "shapes" of real regex use, not three samples of the same thing:

- **Snort/Suricata** network intrusion-detection signatures (`pcre:"..."` rule options, sourced from the Emerging Threats Open ruleset) – literal-heavy, moderate complexity, the closest of the three to "everyday" regex use.
- **SpamAssassin** email spam-filtering rules (`body/header` Perl-regex rules) – content matching over unstructured text, a different shape than network signatures.
- **RegexLib** crowd-sourced general-purpose patterns (validators, extractors) – the

opposite end, where chained lookahead assertions are idiomatic, stress-testing the frontend’s rejection path hardest of the three.

Extraction. Each source embeds its patterns in a different surrounding syntax, so each needs its own extractor (`examples/extract_dataset.rs`): Suricata’s rule lines carry a `pcre:"/PATTERN/FLAGS"`; option; SpamAssassin’s carry a bare `body/header NAME = /PATTERN/FLAGS` without the Suricata’s trailing-quote convention; RegexLib’s sample file lists plain patterns directly, one (possibly multi-line) entry per blank-line-delimited block. All three converge on the same `/PATTERN/FLAGS` PCRE delimiter convention once extracted, parsed by `frontend::parse_pcre_rule` – including the case-insensitive `i` flag, applied by case-folding every literal character in the translated `ExtPat` to an `Any` of both cases (Chapter 7’s frontend) rather than as a runtime matching option, since `Regex` itself has no case-insensitivity concept.

Filtering. Real rules routinely use constructs no derivative-based engine over the six-constructor `Regex` of Section 2.3 can express at all – backreferences and lookahead assertions describe languages that are not regular in the formal sense this thesis’s entire construction depends on. `parse_pcre_rule` rejects both outright with a descriptive error rather than silently mis-parsing them, and `extract_dataset` tallies every rejection by cause (`backreference`, `lookaround`, or `other` – chiefly bound expressions such as `{1586,}` that would unroll past this frontend’s repetition cap, and a handful of unsupported `(?...)` group syntaxes). Re-running this pipeline against the tracked raw sources gives the current triage:

Format	Extracted	Accepted	Backreference	Lookaround	Other
Suricata	231	209 (90.5%)	5	11	6
SpamAssassin	82	69 (84.1%)	1	11	1
RegexLib	499	407 (81.6%)	8	55	29
Total	812	685 (84.4%)	14	77	36

Lookaround dominates the rejections and is concentrated almost entirely in RegexLib, matching that family’s idiomatic use of chained assertions; Suricata and SpamAssassin reject mostly on the same handful of negative-lookahead idioms (guarding against one specific following word). The accepted 685 patterns, deduplicated across all three formats into `data/corpus_patterns.txt` (665 lines after deduplication), are what Section 8.5.5’s dataset benchmark actually runs against.

Unanchored matching. Real rules are *written* as search patterns: a Suricata rule fires on seeing a match anywhere in a packet payload, not on the payload being exactly that pattern. They are therefore parsed through the frontend’s two search entry points, `parse_dataset_pattern` and `parse_pcre_rule`, instead of the full-string-equality `parse_pattern` used everywhere else in this thesis’s own examples. That difference is not cosmetic: real dataset patterns, run unanchored the way they have to be, are measurably bigger than the pattern text suggests and expose scaling behaviour the small synthetic patterns of Section 8.5.4 do not. The mechanism is worth setting out before the measurements that depend on it.

8.3.1 Substring Search: How the Corpus Is Run

No parser in Chapters 5 and 6 answers any question but "does this entire input derive to nullable". There is no "found somewhere inside a longer string" mode anywhere below the frontend, and neither [Sulzmann and Lu \[2014\]](#) nor Sulzmann's teaching-page material defines one: POSIX and Antimirov derivatives are inherently full-match operations. Real rules, though, are *written* as search patterns. A Suricata rule fires on seeing its pattern anywhere in a packet payload, not on the payload being exactly that pattern.

`frontend/mod.rs` therefore implements substring search entirely at this layer, before a `Regex` ever reaches a parser, by padding the translated core with `wildcard_run()`, a `Star` over a 98-way `Alt` of the full alphabet meaning "any run of characters at all", on whichever side lacks an explicit anchor. Figure 8.1 gives the four cases, with the size of each lowered result alongside.

<code>^abc\$</code>	<table><tr><td>abc</td></tr></table>	abc	9 nodes, no padding		
abc					
<code>^abc</code>	<table><tr><td>abc</td><td>Σ^*</td></tr></table>	abc	Σ^*	204 nodes, end padded	
abc	Σ^*				
<code>abc\$</code>	<table><tr><td>Σ^*</td><td>abc</td></tr></table>	Σ^*	abc	204 nodes, start padded	
Σ^*	abc				
<code>abc</code>	<table><tr><td>Σ^*</td><td>abc</td><td>Σ^*</td></tr></table>	Σ^*	abc	Σ^*	399 nodes, both sides padded
Σ^*	abc	Σ^*			

Figure 8.1: Anchor-aware padding under the search entry points. Each Σ^* is one `wildcard_run()`, 196 nodes on its own, so the padding dominates the lowered expression whenever the pattern is short. The fully anchored case still costs 9 rather than 5 nodes, because the erased `^` and `$` leave an `Eps` apiece behind in the concatenation.

`starts_with_carat` and `ends_with_dollar` (`ext_pattern.rs`) decide which sides need padding by walking into `Concat` and `Group` to find a leading `^` or trailing `$` even when nested, before `Carat` and `Dollar` are themselves translated away to `Eps`. Under the search-oriented entry points `parse_dataset_pattern` and `parse_pcre_rule`, as against `parse_pattern`'s plain full-string equality used everywhere else in this thesis's own examples, deciding which side of this padding is skipped is the anchors' only real effect.

Because the anchors are recognized structurally rather than positionally, a `^` that is not at the start, as in `a^b`, is not an anchor at all. It still parses to `Carat` and still translates to `Eps`, so it contributes a node to the lowered expression but nothing to the language:

```
$ cargo run -- parse "a^b" "ab" --diag 1
Regex:  a^b
Input:  "ab"
Match:  true
Tree:   ((a, ()), b)
```

The `()` in the middle of that tree is the erased anchor. PCRE would treat the same pattern as unmatchable outside multiline mode. This is a silent difference rather than an error, and it is the one place in the frontend where a mis-written pattern is neither rejected nor faithfully represented.

Edge case. An unanchored pattern already nullable on empty input, with `"a*"` the simplest example, gets padded on both sides regardless of its own content. The padding's

own empty match, combined with either `Star`, is then enough by itself to match *any* string at all, not just runs of `a`. This is correct substring-search semantics, since "some substring matches" is satisfied by the empty substring that every string contains, and `mod.rs`'s own test `unanchored_nullable_pattern_matches_anything` asserts exactly this behaviour rather than treating it as a defect. It is worth flagging nonetheless, because such a pattern carries no information in a corpus: it matches every input and so contributes nothing to Section 8.3's measurements beyond its own parse cost.

anchors are not a language feature here. Recognising `^` and `$` structurally, and letting them do nothing but suppress one side of the padding, is a decision about the corpus pipeline and not an account of what anchors mean. This thesis's core algebra has six constructors and no notion of a position within an input, so there is nowhere in it for an anchor to live: a derivative consumes a character and knows nothing about where that character sits. Extended matchers that do support anchors get them from a richer algebra. [Varatalu et al. \[2023\]](#) handle intersection, complement and lookarounds on top of derivatives, and treat anchors as expressible through lookarounds rather than as primitives of their own. Adopting that route would mean extending the core algebra and every derivative rule defined over it, which is well beyond the scope of this implementation, but it is the direction in which the gap described above closes, and it is the reason the gap is a scope boundary rather than an oversight.

What this dataset does not (yet) give. The corpus was triaged for "does it parse at all," not for "does POSIX visibly diverge from the partial-derivative answer on this pattern the way `paper_r1`/`paper_r2` do", and unlike Mamouras et al.'s own disambiguation-robustness analysis [[Mamouras et al., 2024](#)], this thesis does not (yet) identify which, if any, of the 685 accepted real-world patterns are themselves disambiguation-non-robust in their sense. Combining this dataset with their static-analysis technique to find such patterns automatically, or constructing inputs that provoke visible divergence between the two constructions on the existing corpus directly, is a natural extension of this evaluation left to future work (Section 8.7).

8.4 Correctness Verification

8.4.1 Paper Examples Reproduced

Both worked examples from Sulzmann and Lu's paper are reproduced as executable tests rather than only as the manual derivations of Sections 4.3 and 5.1: `parse_recursive_paper_r1_on_ab` and `parse_recursive_paper_r2_on_ab` (`tests/test_deriv_std.rs`) assert the exact POSIX tree shape derived by hand in Section 5.1; `paper_r1_computes_greedy_not_posix` and `paper_r2_computes_greedy_not_posix` (`src/parsers/pderiv_bc/parse.rs`) assert both the expected tree shape of Section ?? *and* that it disagrees with the POSIX answer on the identical input – encoding the divergence itself, not merely each answer separately, as the property under test. Every one of these four assertions currently passes, confirming this implementation reproduces both documented policies exactly on the paper's own examples.

8.4.2 Cross-Implementation Equivalence

Following the agreement-testing strategy of Section 8.2, the full test suite checks, and at the time of writing passes, agreement at every layer this thesis introduces more than one implementation of the same specification:

- `match_naive = match_deriv = match_pderiv` on membership, across the hand-picked cases of `tests/test_matchers.rs`.
- `parse_recursive` and `parse_loop` produce identical parse trees on every shared test case (Section 5.2), including `recursive_traced_and_loop_traced_agree_on_all_paper_examples`, comparing full derivation traces, not just final answers.
- `deriv_bc` agrees with `deriv_std` (via `parse_recursive`) on every shared test case (`bitcoded_agrees_with_recursive`, `tests/test_deriv_bc.rs`), the empirical counterpart of Section 5’s claim that both compute the identical POSIX tree by different means.
- `pderiv_std` and `pderiv_bc` agree exactly, not just on membership, on every hand-picked and fuzzed case (Section 8.2).
- All four parser combinations agree on membership wherever the two constructions are not known to diverge, and each pair agrees exactly on tree shape within itself: `deriv_std` with `deriv_bc`, and `pderiv_std` with `pderiv_bc`. This is exercised directly by the CLI’s `--parser all` comparison mode (`src/cli/parser.rs`), which reports all three checks side by side for a given regex/input pair on demand.

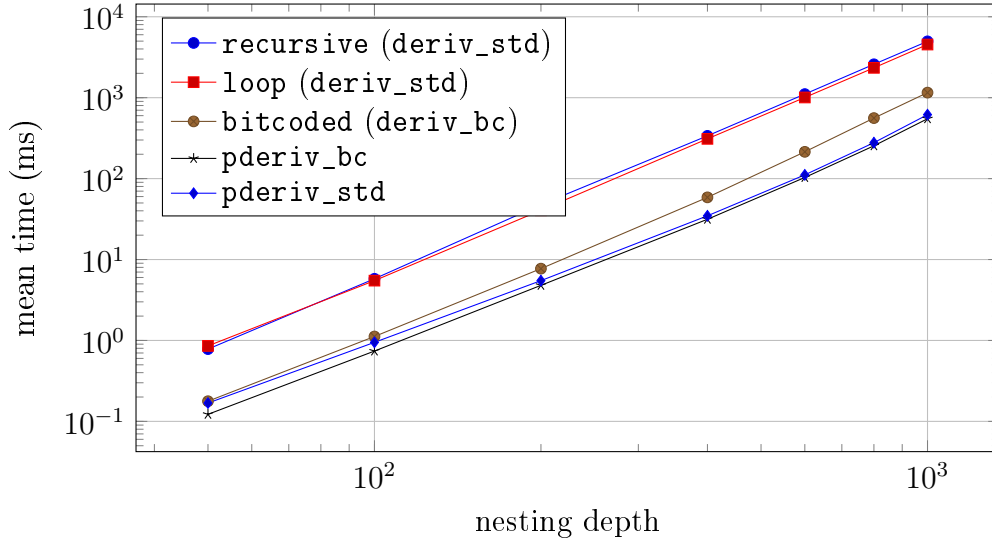
No disagreement has been observed in any of these checks against the current implementation; a regression in any one parser would show up first as a failure here, in the specific pairing that parser participates in, before it could show up as an incorrect answer with no diagnosis of which implementation is at fault.

8.5 Performance Analysis

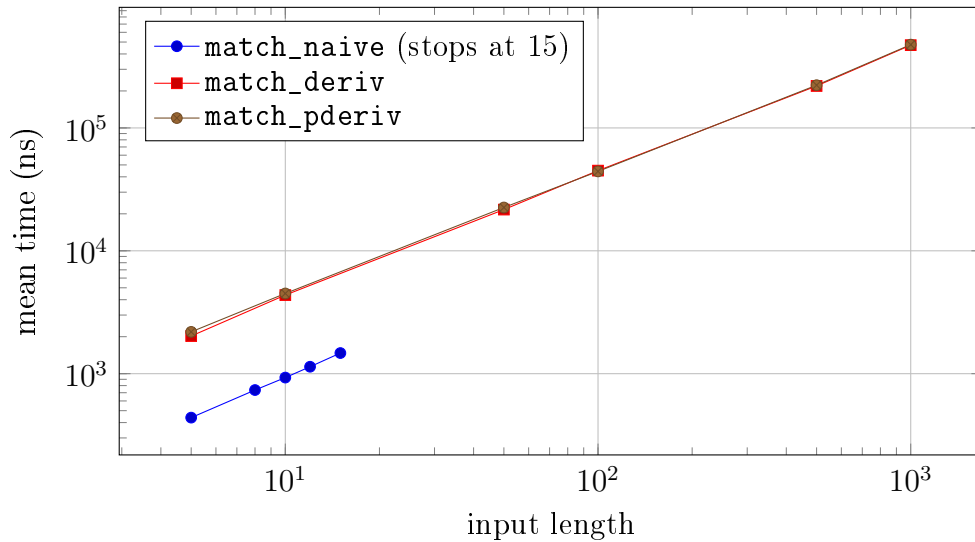
8.5.1 Stack Depth: Recursive vs. Iterative Construction

8.5.2 Heap Allocation: deriv_std vs. deriv_bc

8.5.3 Derivative-Based vs. Partial-Derivative-Based



8.5.4 Pathological Inputs



8.5.5 Benchmark Results

Parser	$n=5$	$n=10$	$n=15$	$n=18$
recursive (deriv_std)	8.0 μ s	110.7 μ s	1.427 ms	6.052 ms
bitcoded (deriv_bc)	11.4 μ s	183.8 μ s	4.175 ms	49.491 ms
pderiv_bc	13.4 μ s	210.6 μ s	2.207 ms	9.236 ms
pderiv_std	22.4 μ s	323.9 μ s	3.374 ms	14.263 ms

Parser	Mean time / sweep
<code>deriv_std_rec</code>	1.311 s
<code>deriv_std_loop</code>	1.091 s
<code>deriv_bc</code>	643.5 ms
<code>pderiv_bc</code>	159.2 ms
<code>pderiv_std</code>	158.6 ms

Parser	Mean time
<code>deriv_std_rec</code>	1.000 s
<code>deriv_bc</code>	81.1 ms

8.6 Comparison with Existing Engines

8.6.1 Performance

Engine	Mean time / sweep	vs. <code>regex</code>
<code>deriv_std_rec</code> (POSIX, recursive)	1.503 s	×1,415,254
<code>deriv_std_loop</code> (POSIX, loop)	1.218 s	×1,146,893
<code>deriv_bc</code> (POSIX, bit-coded)	690.74 ms	×650,417
<code>pderiv_bc</code> (partial derivative, bit-coded)	171.85 ms	×161,815
<code>pderiv_std</code> (partial derivative, tree)	173.28 ms	×163,167
<code>regex</code> (Rust crate)	1.06 μs	×1
<code>re2_perl</code> (leftmost-first)	6.61 μs	×6.2
<code>re2_posix</code> (leftmost-longest)	2.75 μs	×2.6

Engine	Mean time / sweep	vs. <code>regex</code>
<code>match_deriv</code>	273.07 ms	×259,815
<code>match_pderiv</code>	134.95 ms	×128,397
<code>regex</code> (Rust crate)	1.05 μs	×1
<code>re2_perl</code> (leftmost-first)	6.63 μs	×6.3
<code>re2_posix</code> (leftmost-longest)	2.72 μs	×2.6

8.6.2 Standardized Benchmarking via rebar

8.7 Optimization Challenges and Insights

Two optimizations were identified while implementing the four parsers. Neither is applied in the shipped code, and in both cases the reason is the same: the change is worth more than it costs only if correctness is measured against something other than the reference construction. They are recorded here with the measurements that establish their value, since those measurements are the product of this evaluation rather than of the implementation.

Character classes in place of literal alternation. The core algebra of Chapter 2 has six constructors and no character class among them, so the frontend lowers every class to an explicit alternation (Section 7.3). The cost of that decision is visible in the lowered node counts of Section 7.4: a `\d` becomes ten nodes, a `\w` becomes 125, and a wildcard becomes 195. The padding a search pattern receives is one `wildcard_run()` of 196 nodes

per unanchored side, so an unanchored three-letter literal reaches 399 nodes against the 5 its own content needs.

Every one of those nodes is structure the parser walks once per input character. A dedicated class constructor carrying a set of characters would collapse each of them to a single node whose derivative is one membership test, which changes the per-character work from linear in the size of the class to constant. Section 8.5’s dataset measurements are dominated by exactly this effect, since essentially every real-world pattern in the corpus is an unanchored search containing at least one class shorthand.

The optimization is not applied because it is not local. A seventh constructor has to be understood by `nullable`, `deriv`, `simp`, `mkEps`, `inj`, the annotated `ARegex` and its refined derivative, `pDerivBC`, and `decode`, which is every definition in Chapters 5 and 6. The four parsers would then no longer be transcriptions of the constructions they are transcriptions of, which is the property the correctness argument of Section 8.4 rests on. There is a second benefit worth recording alongside the performance one: because the expected set reported on a failure is read off the first-literal set of the expression, a mismatch against a lowered wildcard currently reports all 98 alternatives individually, where a native class could report `expected: any character`.

Frontier-wide deduplication. The partial-derivative parsers apply `nub2` inside a single expansion and never across the flattened frontier, which is what lets the frontier double on the expressions of Section 6.4 while the number of distinct residuals stays bounded. Moving that deduplication to the frontier as a whole is a change of one call site. Measured over the same 11700 cases as Section 8.4.2, it changes no answer, on membership or on the tree returned, and takes the peak frontier from 170 strands to 4.

It is not applied for the reason given in Section 6.2: this parser’s correctness criterion is literal agreement with the reference `pDerivBC`, and a change verified on finitely many cases without a proof is a change to the thing being verified against. The measurement establishes that the optimization is available and what it is worth; adopting it would require replacing agreement with an argument.

8.8 Discussion of Results

9 Related Work and Conclusion

9.1 Related Work

9.2 Conclusion

Bibliography

- Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.
- Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4): 481–494, 1964. doi: 10.1145/321239.321249.
- Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). <https://swtch.com/~rsc/regexp/regexp1.html>, 2007. Accessed: 2026-08-05.
- Alain Frisch and Luca Cardelli. Greedy regular expression matching. In *Automata, Languages and Programming (ICALP 2004)*, volume 3142, pages 618–629. Springer, 2004. doi: 10.1007/978-3-540-27836-8_53.
- Google LLC. RE2: A fast, safe, POSIX-compatible regular expression library. <https://github.com/google/re2>, 2024. Accessed: 2026-08-05.
- IEEE and The Open Group. Posix.1-2017: Base definitions, chapter 9 (regular expressions), 2017. IEEE Std 1003.1-2017.
- Konstantinos Mamouras, Alexis Le Glaunec, Wu Angela Li, and Agnishom Chattopadhyay. Static analysis for checking the disambiguation robustness of regular expressions. *Proceedings of the ACM on Programming Languages*, 8(PLDI):231, 2024. doi: 10.1145/3656461.
- Martin Sulzmann. Programming paradigms: Regular expressions. <https://sulzmann.github.io/ProgrammingParadigms/pp-regular-expressions.html>, 2026. Accessed: 2026-08-08.
- Martin Sulzmann and Kenny Zhuo Ming Lu. Regular expression sub-matching using partial derivatives. In *PPDP*, pages 79–90, 2012. doi: 10.1145/2370776.2370788.
- Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.
- The Rust Project Developers. Using `Box<T>` to point to data on the heap. The Rust Programming Language, 2026a. URL <https://doc.rust-lang.org/book/ch15-01-box.html>. Accessed: 2026-09-11.

The Rust Project Developers. Types. The Rust Reference, 2026b. URL <https://doc.rust-lang.org/reference/types.html>. Accessed: 2026-09-11.

Ian Erik Varatalu, Margus Veanes, and Juhan-Peep Ernits. Derivative based extended regular expression matching supporting intersection, complement and lookarounds. <https://arxiv.org/abs/2309.14401>, 2023. Accessed: 2026-09-18.

A Code

A.1 Core Types

Regex

src/types/regex.rs

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    /// Empty language:  $L(\Phi) = \{\}$ 
    Phi,

    /// Empty word:  $L(\epsilon) = \{\epsilon\}$ 
    Eps,

    /// Single character:  $L(\text{Lit}(c)) = \{c\}$ 
    Lit(char),

    /// Sequence:  $L(\text{Seq}(r,s)) = \{v++w \mid v \in L(r), w \in L(s)\}$ 
    Seq(Box<Regex>, Box<Regex>),

    /// Alternative:  $L(\text{Alt}(r,s)) = L(r) \cup L(s)$ 
    Alt(Box<Regex>, Box<Regex>),

    /// Kleene star:  $L(\text{Star}(r)) = \{\epsilon\} \cup L(r).L(\text{Star}(r))$ 
    Star(Box<Regex>),
}
```

ARegex

src/types/aregex.rs

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ARegex {
    /// Empty language
    Phi,
    ///  $bs @ \epsilon$ 
    Eps(Vec<bool>),
    ///  $bs @ 1$ 
    Lit(Vec<bool>, char),
    ///  $bs @ (ri1 (+) ri2)$ 
    Alt(Vec<bool>, Box<ARegex>, Box<ARegex>),
    ///  $bs @ (ri1 ri2)$ 
    Seq(Vec<bool>, Box<ARegex>, Box<ARegex>),
    ///  $bs @ ri^*$ 
    Star(Vec<bool>, Box<ARegex>),
}
```

ParseTree

src/types/tree.rs

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseTree {
    /// Empty parse tree (for epsilon)
    Empty,

    /// Literal character
    Char(char),
}
```

```

    /// Pair for concatenation (v1, v2)
    Pair(Box<ParseTree>, Box<ParseTree>),

    /// Left injection for alternative (Left v)
    Left(Box<ParseTree>),

    /// Right injection for alternative (Right v)
    Right(Box<ParseTree>),

    /// List for Kleene star iterations [v1, v2, ..., vn]
    Star(Vec<ParseTree>),
}

```

flatten
src/types/tree.rs

```

/// Flattens parse tree to underlying word (string)
pub fn flatten(v: &ParseTree) -> String {
    match v {
        ParseTree::Empty => String::new(),
        ParseTree::Char(c) => c.to_string(),
        ParseTree::Pair(v1, v2) => format!("{}", flatten(v1), flatten(v2)),
        ParseTree::Left(v) => flatten(v),
        ParseTree::Right(v) => flatten(v),
        ParseTree::Star(vs) => vs.iter().map(flatten).collect(),
    }
}

```

A.2 Shared Algorithms

nullable
src/regex/nullable.rs

```

/// Decides whether epsilon is in L(r)
pub fn nullable(r: &Regex) -> bool {
    match r {
        Regex::Phi => false,
        Regex::Eps => true,
        Regex::Lit(_) => false,
        Regex::Alt(r, s) => nullable(r) || nullable(s),
        Regex::Seq(r, s) => nullable(r) && nullable(s),
        Regex::Star(_) => true,
    }
}

```

deriv
src/regex/deriv.rs

```

/// Computes derivative of r based on character x
pub fn deriv(r: &Regex, x: char) -> Regex {
    match r {
        Regex::Phi => Regex::Phi,
        Regex::Eps => Regex::Phi,
        Regex::Lit(c) => {
            if *c == x { Regex::Eps } else { Regex::Phi }
        }
        Regex::Alt(r1, r2) => {
            Regex::alt(deriv(r1, x), deriv(r2, x))
        }
        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let dr1 = Regex::seq(deriv(r1, x), *r2.clone());
                Regex::alt(dr1, deriv(r2, x))
            } else {
                Regex::seq(deriv(r1, x), *r2.clone())
            }
        }
        Regex::Star(r1) => {
            Regex::seq(deriv(r1, x), Regex::star(*r1.clone()))
        }
    }
}

```

```
}
```

mk_eps
src/regex/mk_eps.rs

```
pub fn mk_eps(r: &Regex) -> ParseTree {
    match r {
        Regex::Eps => ParseTree::Empty,

        Regex::Star(_) => ParseTree::Star(Vec::new()),

        Regex::Seq(r1, r2) => {
            let v1 = mk_eps(r1);
            let v2 = mk_eps(r2);
            ParseTree::Pair(Box::new(v1), Box::new(v2))
        }

        Regex::Alt(r1, r2) => {
            if nullable(r1) {
                ParseTree::Left(Box::new(mk_eps(r1)))
            } else {
                ParseTree::Right(Box::new(mk_eps(r2)))
            }
        }

        Regex::Lit(c) => panic!("mk_eps called on Lit('{ }')", c),
        Regex::Phi => panic!("mk_eps called on Phi"),
    }
}
```

decode
src/regex/decode.rs

```
pub fn decode(r: &Regex, bs: &[bool]) -> ParseTree {
    let (v, rest) = decode_inner(r, bs);
    assert!(
        rest.is_empty(),
        "decode: {} leftover bit(s) after decoding {:?}",
        rest.len(),
        r
    );
    v
}
```

simplify
src/regex/simplify.rs

```
/// Simplify regular expression using algebraic laws
pub fn simplify(r: Regex) -> Regex {
    match r {
        Regex::Seq(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) => Regex::Phi,
                (_, Regex::Phi) => Regex::Phi,
                (Regex::Eps, _) => s,
                (_, Regex::Eps) => r,
                _ => Regex::seq(r, s),
            }
        }
        Regex::Alt(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) => s,
                (_, Regex::Phi) => r,
                _ if r == s => r,
                _ => Regex::alt(r, s),
            }
        }
        Regex::Star(r) => {
            let r = simplify(*r);

```

```

        match r {
            Regex::Eps => Regex::Eps,
            Regex::Phi => Regex::Eps,
            r => Regex::star(r),
        }
    }
    other => other,
}
}

```

A.3 Matchers

match_naive
src/matchers/match_naive.rs

```

pub fn match_naive(input: &str, r: &Regex) -> bool {
    let chars: Vec<char> = input.chars().collect();
    let len = chars.len();
    match_naive_range(&chars, 0, len, r)
}

```

match_deriv
src/matchers/match_deriv.rs

```

pub fn match_deriv(input: &str, r: &Regex) -> bool {
    let mut current = r.clone();
    for c in input.chars() {
        current = simplify(deriv(&current, c));
    }
    nullable(&current)
}

```

match_pderiv
src/matchers/match_pderiv.rs

```

pub fn match_pderiv(input: &str, r: &Regex) -> bool {
    let mut states: HashSet<Regex> = HashSet::new();
    states.insert(r.clone());

    for c in input.chars() {
        let mut next_states: HashSet<Regex> = HashSet::with_capacity(states.len());
        for state in &states {
            next_states.extend(pderiv(state, c));
        }
        if next_states.is_empty() {
            return false;
        }
        states = next_states;
    }

    states.iter().any(nullable)
}

```

A.4 The deriv_std Parser

inject
src/parsers/deriv_std/inject.rs

```

pub fn inject(r: &Regex, l: char, v: ParseTree) -> ParseTree {
    match r {
        Regex::Lit(c) => {
            assert!(*c == l);
            assert!(matches!(v, ParseTree::Empty));
            ParseTree::Char(l)
        }

        Regex::Star(r1) => {
            match v {

```

```

    ParseTree::Pair(v1, vs) => {
        let v1_inj = inject(r1, 1, *v1);
        let mut iterations = vec![v1_inj];
        if let ParseTree::Star(rest) = *vs {
            iterations.extend(rest);
        }
        ParseTree::Star(iterations)
    }
    _ => panic!("inject on Star: expected Pair, got {:?}", v),
}

Regex::Seq(r1, r2) => {
    let r1_nullable = nullable(r1);

    if !r1_nullable {
        match v {
            ParseTree::Pair(v1, v2) => {
                let v1_inj = inject(r1, 1, *v1);
                ParseTree::Pair(Box::new(v1_inj), v2)
            }
            _ => panic!("inject on Seq (non-nullable r1): expected Pair, got {:?}", v),
        }
    } else {
        match v {
            ParseTree::Left(v_pair) => {
                match *v_pair {
                    ParseTree::Pair(v1, v2) => {
                        let v1_inj = inject(r1, 1, *v1);
                        ParseTree::Pair(Box::new(v1_inj), v2)
                    }
                    _ => panic!(
                        "inject on Seq (nullable r1, Left): expected Pair, got {:?}",
                        v_pair
                    ),
                }
            }
            ParseTree::Right(v2) => {
                let v2_inj = inject(r2, 1, *v2);
                let v1_eps = mk_eps(r1);
                ParseTree::Pair(Box::new(v1_eps), Box::new(v2_inj))
            }
            _ => panic!("inject on Seq (nullable r1): expected Left or Right, got {:?}", v),
        }
    }
}

Regex::Alt(r1, r2) => {
    match v {
        ParseTree::Left(v1) => {
            ParseTree::Left(Box::new(inject(r1, 1, *v1)))
        }
        ParseTree::Right(v2) => {
            ParseTree::Right(Box::new(inject(r2, 1, *v2)))
        }
        _ => panic!("inject on Alt: expected Left or Right, got {:?}", v),
    }
}

Regex::Eps => panic!("inject called on Eps"),
Regex::Phi => panic!("inject called on Phi"),
}
}

```

A.5 The deriv_bc Parser

`src/parsers/deriv_bc/internalize.rs`

```

pub fn fuse(bs: &[bool], ri: ARegex) -> ARegex {
    if bs.is_empty() { return ri; }
}

```

```

match ri {
  ARegex::Phi => ARegex::Phi,
  ARegex::Eps(p) => ARegex::Eps(combine(bs, p)),
  ARegex::Lit(p, c) => ARegex::Lit(combine(bs, p), c),
  ARegex::Alt(p, r1, r2) => ARegex::Alt(combine(bs, p), r1, r2),
  ARegex::Seq(p, r1, r2) => ARegex::Seq(combine(bs, p), r1, r2),
  ARegex::Star(p, r) => ARegex::Star(combine(bs, p), r),
}
}

```

deriv_bc
src/parsers/deriv_bc/deriv.rs

```

/// Bit-coded derivative (paper Figure 5)
pub fn deriv_bc(ri: ARegex, l: char) -> ARegex {
  match ri {
    ARegex::Phi => ARegex::Phi,
    ARegex::Eps(_) => ARegex::Phi,
    ARegex::Lit(bs, c) => {
      if c == 1 {
        ARegex::Eps(bs)
      } else {
        ARegex::Phi
      }
    }
    ARegex::Alt(bs, r1, r2) => {
      let d1 = deriv_bc(*r1, l);
      let d2 = deriv_bc(*r2, l);
      ARegex::Alt(bs, Box::new(d1), Box::new(d2))
    }
    ARegex::Seq(bs, r1, r2) => {
      if nullable_bc(&r1) {
        let eps_bits = mk_eps_bc(&r1);
        let d1 = deriv_bc(*r1, l);
        let d2 = deriv_bc(*r2.clone(), l);
        let left_branch = ARegex::Seq(vec![], Box::new(d1), r2);
        let right_branch = fuse(&eps_bits, d2);
        ARegex::Alt(bs, Box::new(left_branch), Box::new(right_branch))
      } else {
        let d1 = deriv_bc(*r1, l);
        ARegex::Seq(bs, Box::new(d1), r2)
      }
    }
    // Empty star prefix: bs = []; Inner exp: r = Box<Lit([], 'a')>
    ARegex::Star(bs, r) => {
      let d = deriv_bc(*r.clone(), l); // ri\'a\' = Lit([], \'a\')\'a\' = Eps([])
      let fused = fuse(&[false], d); // fuse [0] Eps([]) = Eps([false])
      let new_star = ARegex::Star(vec![], r); // []@ri* = []@([]@\'a\')*
      ARegex::Seq(bs, Box::new(fused), Box::new(new_star))
      // = []@Eps([false]) . []@([]@\'a\')*
    }
  }
}

```

simp
src/parsers/deriv_bc/simplify.rs

```

/// Simplify an annotated expression to a fixpoint.
pub fn simp(ri: ARegex) -> ARegex {
  simp_once(ri)
}

```

A.6 The pderiv_bc Parser

pderiv_bc
src/parsers/pderiv_bc/pderiv.rs

```

/// Bit-coded partial derivative (pDerivBC): one '(residual, bits)' pair per surviving strand.
pub fn pderiv_bc(r: &Regex, x: char) -> Vec<(Regex, Vec<bool>)> {
  match r {

```

```

Regex::Eps => Vec::new(),
Regex::Phi => Vec::new(),

Regex::Lit(y) => {
  if *y == x {
    vec![(Regex::Eps, Vec::new())]
  } else {
    Vec::new()
  }
}

Regex::Alt(r1, r2) => {
  let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
    .into_iter()
    .map(|(r1_, mut bs)| {
      let mut b = vec![false];
      b.append(&mut bs);
      (r1_, b)
    })
    .collect();
  out.extend(
    pderiv_bc(r2, x)
      .into_iter()
      .map(|(r2_, mut bs)| {
        let mut b = vec![true];
        b.append(&mut bs);
        (r2_, b)
      })
  );
  nub2(out)
}

Regex::Seq(r1, r2) => {
  if nullable(r1) {
    // Strand 1: continue r1, r2 untouched (not smart-constructed,
    // matching the reference literally)
    let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
      .into_iter()
      .map(|(r1_, bs)| (Regex::seq(r1_, *r2.clone()), bs))
      .collect();

    // Strand 2: r1 matched empty, jump straight into r2.
    let eps_bits = mk_eps_bits(r1);
    out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, bs)| {
      let mut b = eps_bits.clone();
      b.extend(bs);
      (r2_, b)
    }));

    nub2(out)
  } else {
    pderiv_bc(r1, x)
      .into_iter()
      .map(|(r1_, bs)| (smart_seq(r1_, r2), bs))
      .collect()
  }
}

Regex::Star(r1) => {
  let out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
    .into_iter()
    .map(|(r1_, mut bs)| {
      let mut b = vec![false];
      b.append(&mut bs);
      (smart_seq(r1_, &Regex::star(*r1.clone())), b)
    })
    .collect();
  nub2(out)
}
}

```

nub2

src/parsers/pderiv_bc/deriv.rs

```
/// Stable dedup on the residual, keeping the first (highest-priority)
fn nub2(pairs: Vec<Regex, Vec<bool>>>) -> Vec<Regex, Vec<bool>> {
    let mut seen: HashSet<Regex> = HashSet::with_capacity(pairs.len());
    let mut out = Vec::with_capacity(pairs.len());
    for (r, bs) in pairs {
        if seen.insert(r.clone()) {
            out.push((r, bs));
        }
    }
    out
}
```

step_frontier
src/parsers/pderiv_bc/parse.rs

```
/// One step of 'parsePDerivBC2': replace the frontier with union of 'pderiv_bc(x, .)'
pub(super) fn step_frontier(frontier: &[(Regex, Vec<bool>)], x: char) -> Vec<Regex, Vec<bool>> {
    let mut out = Vec::new();
    for (r, bits) in frontier {
        for (r_next, extra) in pderiv_bc(r, x) {
            let mut combined = bits.clone();
            combined.extend(extra);
            out.push((r_next, combined));
        }
    }
    out
}
```

select_bits
src/parsers/pderiv_bc/parse.rs

```
/// 'filter ((r,_) -> nullable r) rs', take head, and append 'mkEpsBC'.
pub(super) fn select_bits(frontier: &[(Regex, Vec<bool>)]) -> Option<Vec<bool>> {
    for (r, bits) in frontier {
        if nullable(r) {
            let mut result = bits.clone();
            result.extend(mk_eps_bits(r));
            return Some(result);
        }
    }
    None
}
```

parse_pderiv_bc
src/parsers/pderiv_bc/parse.rs

```
pub fn parse_pderiv_bc(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier = vec![(r.clone(), Vec::new())];
    for c in input.chars() {
        frontier = step_frontier(&frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    let bits = select_bits(&frontier)?;
    Some(decode(r, &bits))
}
```

A.7 The pderiv_std Parser

Inj
src/parsers/pderiv_std/inject.rs

```
/// Reconstructs a strand's 'ParseTree'; 'Rc' since a fan-out step (e.g. 'Alt') shares one parent injection
pub(crate) type Inj = Rc<dyn Fn(ParseTree) -> ParseTree>;
```

```
pub(crate) fn identity_inj() -> Inj {
    Rc::new(|v| v)
}
```

nub_tree
src/parsers/pderiv_std/inject.rs

```
/// Dedup on the residual alone, keeping the first (highest-priority, GREEDY-ordered)
pub(crate) fn nub_tree(strands: Vec<(Regex, Inj)> -> Vec<(Regex, Inj)> {
    let mut seen: HashSet<Regex> = HashSet::with_capacity(strands.len());
    let mut out = Vec::with_capacity(strands.len());
    for (r, inj) in strands {
        if seen.insert(r.clone()) {
            out.push((r, inj));
        }
    }
    out
}
```

pderiv_tree
src/parsers/pderiv_std/pderiv_tree.rs

```
pub(crate) fn pderiv_tree(r: &Regex, x: char) -> Vec<(Regex, Inj)> {
    match r {
        Regex::Eps | Regex::Phi => vec![],

        Regex::Lit(c) => {
            if *c == x {
                vec![(Regex::Eps, Rc::new(move |_| ParseTree::Char(x)) as Inj)]
            } else {
                vec![]
            }
        }

        Regex::Alt(r1, r2) => {
            let mut out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
                .into_iter()
                .map(|(r1_, inj1)| {
                    let f: Inj = Rc::new(move |v| ParseTree::Left(Box::new(inj1(v))));
                    (r1_, f)
                })
                .collect();
            out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                let f: Inj = Rc::new(move |v| ParseTree::Right(Box::new(inj2(v))));
                (r2_, f)
            }));
            nub_tree(out)
        }

        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let mut out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
                    .into_iter()
                    .map(|(r1_, inj1)| {
                        let r2c = (**r2).clone();
                        let f: Inj = Rc::new(move |v: ParseTree| match v {
                            ParseTree::Pair(v1, v2) => ParseTree::Pair(Box::new(inj1(*v1)), v2),
                            other => panic!(
                                "pderiv_tree: Seq-continue strand expected Pair, got {:?}",
                                other
                            ),
                        },
                    ),
                    (Regex::seq(r1_, r2c), f)
                })
                    .collect();

                let v1_eps = mk_eps(r1);
                out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                    let v1c = v1_eps.clone();
                    let f: Inj = Rc::new(move |v2: ParseTree| {
                        ParseTree::Pair(Box::new(v1c.clone()), Box::new(inj2(v2)))
                    });
                    (r2_, f)
                }));
                nub_tree(out)
            } else {
                pderiv_tree(r1, x)
            }
        }
    }
}
```

```

        .into_iter()
        .map(|(r1_, inj1)| {
            let r2c = (**r2).clone();
            let f: Inj = Rc::new(move |v: ParseTree| match v {
                ParseTree::Pair(v1, v2) => ParseTree::Pair(Box::new(inj1(*v1)), v2),
                other => panic!(
                    "pderiv_tree: Seq(non-nullable) strand expected Pair, got {:?}",
                    other
                ),
            });
            (Regex::seq(r1_, r2c), f)
        })
        .collect()
    }
}

Regex::Star(r1) => {
    let out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
        .into_iter()
        .map(|(r1_, inj1)| {
            let r1c = (**r1).clone();
            let f: Inj = Rc::new(move |v: ParseTree| match v {
                ParseTree::Pair(v1, vs) => match *vs {
                    ParseTree::Star(mut items) => {
                        items.insert(0, inj1(*v1));
                        ParseTree::Star(items)
                    }
                    other => panic!(
                        "pderiv_tree: Star strand expected Star tail, got {:?}",
                        other
                    ),
                },
                other => panic!(
                    "pderiv_tree: Star strand expected Pair, got {:?}",
                    other
                ),
            });
            (Regex::seq(r1_, Regex::star(r1c)), f)
        })
        .collect();
    nub_tree(out)
}
}
}
}

```

parse_pderiv_std
src/parsers/pderiv_std/parse.rs

```

pub fn parse_pderiv_std(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier: Vec<(Regex, Inj)> = vec![(r.clone(), identity_inj())];
    for c in input.chars() {
        frontier = step_frontier(frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    select(&frontier)
}

```

A.8 Frontend

ExtPat
src/frontend/ext_pattern.rs

```

#[derive(Debug, Clone, PartialEq)]
pub enum ExtPat {
    /// the empty string, 'translate's to 'Regex::Eps'.
    Empty,
    Eps,
    /// Literal 'phi' token 'translate's to 'Regex::Phi'.
    Never,
}

```

```

    /// A marking group '( ... )'.
    Group(Box<ExtPat>),
    /// A non-marking group '(?: ... )'.
    GroupNonMarking(Box<ExtPat>),
    /// Alternation 'a|b|c'.
    Or(Vec<ExtPat>),
    /// Concatenation 'ab c'.
    Concat(Vec<ExtPat>),
    /// 'r?' / 'r??' (bool: greedy).
    Opt(Box<ExtPat>, bool),
    /// 'r+' / 'r+?' (bool: greedy).
    Plus(Box<ExtPat>, bool),
    /// 'r*' / 'r*?' (bool: greedy).
    Star(Box<ExtPat>, bool),
    /// 'r{lo,hi}' / 'r{lo,}' / 'r{lo}' (bool: greedy).
    Bound(Box<ExtPat>, u32, Option<u32>, bool),
    /// '^'.
    Carat,
    /// '$'.
    Dollar,
    /// '.'.
    Dot,
    /// '[ ... ]' listed characters (ranges and '\d'/'\w'/'\s'
    /// shorthands already expanded by the parser).
    Any(Vec<char>),
    /// '[^ ... ]'.
    NoneOf(Vec<char>),
    /// '\c' for a 'c' that carries regex meaning to 'translate'
    /// ('d'/'D'/'w'/'W'/'s'/'S') or, for any other 'c', an escaped literal
    /// ('\.', '\(', '\\', ...).
    Escape(char),
    /// An ordinary, unescaped literal character.
    Char(char),
}

```

translate

src/frontend/translate.rs

```

/// Lower an 'ExtPat' into this crate's core 'Regex'.
pub fn translate(ep: &ExtPat) -> Result<Regex, String> {
    match ep {
        ExtPat::Empty => Ok(Regex::Eps),
        ExtPat::Eps => Ok(Regex::Eps),
        ExtPat::Never => Ok(Regex::Phi),
        ExtPat::Group(inner) | ExtPat::GroupNonMarking(inner) => translate(inner),
        ExtPat::Or(alts) => {
            let mut iter = alts.iter();
            let mut acc = translate(iter.next().ok_or("empty alternation")??);
            for a in iter {
                acc = Regex::alt(acc, translate(a)?);
            }
            Ok(acc)
        }
        ExtPat::Concat(parts) => {
            let mut iter = parts.iter();
            let mut acc = match iter.next() {
                Some(p) => translate(p)?,
                None => return Ok(Regex::Eps),
            };
            for p in iter {
                acc = Regex::seq(acc, translate(p)?);
            }
            Ok(acc)
        }
        ExtPat::Opt(inner, _greedy) => Ok(Regex::alt(translate(inner)?, Regex::Eps)),
        ExtPat::Plus(inner, _greedy) => {
            let r = translate(inner)?;
            Ok(Regex::seq(r.clone(), Regex::star(r)))
        }
        ExtPat::Star(inner, _greedy) => Ok(Regex::star(translate(inner)?)),
        ExtPat::Bound(inner, lo, hi, _greedy) => desugar_bound(translate(inner)?, *lo, *hi),
        ExtPat::Carat | ExtPat::Dollar => Ok(Regex::Eps),
        ExtPat::Dot => alt_of_chars(alphabet::alphabet()),
    }
}

```

```
ExtPat::Any(cs) => alt_of_chars(cs.clone()),
ExtPat::NoneOf(cs) => alt_of_chars(alphabet::complement(cs)),
ExtPat::Escape(c) => translate_escape(*c),
ExtPat::Char(c) => Ok(Regex::lit(*c)),
    }
}
```

B Tests

B.1 Shared Fixtures

paper_r1
tests/common/mod.rs

```
pub fn paper_r1() -> Regex {
    Regex::seq(
        Regex::alt(Regex::lit('a'), Regex::seq(Regex::lit('a'), Regex::lit('b'))),
        Regex::alt(Regex::lit('b'), Regex::Eps),
    )
}
```

paper_r2
tests/common/mod.rs

```
/// (a + (b + ab))* - POSIX star-preference example from flops14 p.1
/// Structure: Star(Alt(a, Alt(b, ab))) - nested Alt, ab is Right(Right(...))
pub fn paper_r2() -> Regex {
    Regex::star(Regex::alt(
        Regex::lit('a'),
        Regex::alt(
            Regex::lit('b'),
            Regex::seq(Regex::lit('a'), Regex::lit('b')),
        ),
    ))
}
```

assert_round_trip
tests/common/mod.rs

```
#[allow(dead_code)]
pub fn assert_round_trip(tree: &ParseTree, expected_word: &str) {
    assert_eq!(
        flatten(tree), expected_word,
        "flatten(tree) should equal the matched word"
    );
}
```

assert_parsers_agree
tests/common/mod.rs

```
#[allow(dead_code)]
pub fn assert_parsers_agree(
    label_a: &str,
    tree_a: &Option<ParseTree>,
    label_b: &str,
    tree_b: &Option<ParseTree>,
) {
    assert_eq!(
        tree_a, tree_b,
        "{} and {} disagree:\n {}: {:?}\n {}: {:?}",
        label_a, label_b, label_a, tree_a, label_b, tree_b
    );
}
```

B.2 Paper Examples Reproduced

`parse_recursive_paper_r1_on_ab`
tests/test_deriv_std.rs

```
/// (a + ab)(b + eps) on "ab": A1 picks the longer "ab" branch on the left factor.
#[test]
fn parse_recursive_paper_r1_on_ab() {
    let r = paper_r1();
    let tree = parse_deriv_std_rec("ab", &r).expect("should match");
    assert_round_trip(&tree, "ab");

    assert!(
        matches!(&tree, ParseTree::Pair(_, _)),
        "outer should be Pair (regex is Seq), got {:?}", tree
    );

    if let ParseTree::Pair(left, right) = &tree {
        assert!(
            matches!(left.as_ref(), ParseTree::Right(_)),
            "left factor should be Right (ab branch chosen by A1), got {:?}", left
        );
        assert!(
            matches!(right.as_ref(), ParseTree::Right(_)),
            "right factor should be Right (eps branch, only valid choice), got {:?}", right
        );
    }
}
```

`parse_recursive_paper_r2_on_ab`
tests/test_deriv_std.rs

```
/// (a + b + ab)* on "ab": A1 picks one "ab" iteration over Greedy's two-iteration answer.
#[test]
fn parse_recursive_paper_r2_on_ab() {
    let r = paper_r2();
    let tree = parse_deriv_std_rec("ab", &r).expect("should match");
    assert_round_trip(&tree, "ab");

    if let ParseTree::Star(ref iters) = tree {
        assert_eq!(iters.len(), 1,
            "POSIX should pick 'ab' as one star iteration (A1), got {:?}", iters);

        assert!(
            matches!(&iters[0], ParseTree::Right(inner)
                if matches!(inner.as_ref(), ParseTree::Right(innermost)
                    if matches!(innermost.as_ref(), ParseTree::Pair(_, _)))),
            "iteration should be Right(Right(Pair(a,b))), got {:?}", iters[0]
        );
    } else {
        panic!("expected Star, got {:?}", tree);
    }
}
```

`paper_r1_computes_greedy_not_posix`
src/parsers/pderiv_bc/parse.rs

```
#[test]
fn paper_r1_computes_greedy_not_posix() {
    // (a + ab)(b + eps) on "ab"
    let r = Regex::seq(
        Regex::alt(Regex::lit('a'), Regex::seq(Regex::lit('a'), Regex::lit('b'))),
        Regex::alt(Regex::lit('b'), Regex::Eps),
    );

    let posix = parse_deriv_std_rec("ab", &r);
    let greedy = parse_pderiv_bc("ab", &r);

    assert!(posix.is_some() && greedy.is_some(), "both should match");
    assert_eq!(flatten(posix.as_ref().unwrap()), "ab");
    assert_eq!(flatten(greedy.as_ref().unwrap()), "ab");
}
```

```

// POSIX: the longer "ab" alternative wins (rule A1)
assert_eq!(
  posix,
  Some(ParseTree::Pair(
    Box::new(ParseTree::Right(Box::new(ParseTree::Pair(
      Box::new(ParseTree::Char('a')),
      Box::new(ParseTree::Char('b')),
    )))),
    Box::new(ParseTree::Right(Box::new(ParseTree::Empty))),
  ))
);
// Greedy: the leftmost alternative 'a' wins, committed to immediately
assert_eq!(
  greedy,
  Some(ParseTree::Pair(
    Box::new(ParseTree::Left(Box::new(ParseTree::Char('a')))),
    Box::new(ParseTree::Left(Box::new(ParseTree::Char('b')))),
  ))
);
assert_ne!(posix, greedy, "this is exactly the documented divergence");
}

```

paper_r2_computes_greedy_not_posix

src/parsers/pderiv_bc/parse.rs

```

#[test]
fn paper_r2_computes_greedy_not_posix() {
  // (a + (b + ab))* on "ab"
  let r = Regex::star(Regex::alt(
    Regex::lit('a'),
    Regex::alt(Regex::lit('b'), Regex::seq(Regex::lit('a'), Regex::lit('b')))),
  ));

  let posix = parse_deriv_std_rec("ab", &r);
  let greedy = parse_pderiv_bc("ab", &r);

  assert!(posix.is_some() && greedy.is_some(), "both should match");
  assert_eq!(flatten(posix.as_ref().unwrap()), "ab");
  assert_eq!(flatten(greedy.as_ref().unwrap()), "ab");

  // POSIX: one iteration, the longer "ab" alternative wins (rule A1)
  assert_eq!(
    posix,
    Some(ParseTree::Star(vec! [ParseTree::Right(Box::new(ParseTree::Right(
      Box::new(ParseTree::Pair(
        Box::new(ParseTree::Char('a')),
        Box::new(ParseTree::Char('b')),
      ))
    ))]))
  );
  // Greedy: two iterations, leftmost alternative wins at each step
  assert_eq!(
    greedy,
    Some(ParseTree::Star(vec! [
      ParseTree::Left(Box::new(ParseTree::Char('a'))),
      ParseTree::Right(Box::new(ParseTree::Left(Box::new(ParseTree::Char('b'))))),
    ]))
  );
  assert_ne!(posix, greedy, "this is exactly the documented divergence");
}

```

B.3 Cross-Implementation Equivalence

recursive_traced_and_loop_traced_agree_on_all_paper_examples

tests/test_deriv_std.rs

```

#[test]
fn recursive_traced_and_loop_traced_agree_on_all_paper_examples() {
  use regex_engine::parsers::{parse_recursive_traced, parse_loop_traced};

```

```

let cases: Vec<(&str, Regex)> = vec![
    ("aaa", Regex::star(Regex::lit('a'))),
    ("aab", Regex::star(Regex::lit('a'))),
    ("ab", paper_r1()),
    ("a", paper_r1()),
    ("ab", paper_r2()),
    ("", Regex::star(Regex::lit('a'))),
];

for (input, r) in &cases {
    let (rec_tree, _) = parse_recursive_traced(input, r);
    let (loop_tree, _) = parse_loop_traced(input, r);
    assert_eq!(
        rec_tree, loop_tree,
        "recursive_traced and loop_traced disagree on {:?}" , input
    );
}

}

bitcoded_agrees_with_recursive
tests/test_deriv_bc.rs

fn bitcoded_agrees_with_recursive(input: &str, r: &Regex) {
    let rec = parse_deriv_std_rec(input, r);
    let bc = parse_deriv_bc(input, r);
    assert_parsers_agree("recursive", &rec, "bitcoded", &bc);
}

```

B.4 Frontend

unanchored_nullable_pattern_matches_anything
src/frontend/mod.rs

```

#[test]
fn unanchored_nullable_pattern_matches_anything() {
    let r = parse_dataset_pattern("a*").expect("should parse");
    assert!(parse_deriv_std_rec("", &r).is_some());
    assert!(parse_deriv_std_rec("xyz", &r).is_some());
    assert!(parse_deriv_std_rec("zzz999", &r).is_some());
}

```

C Diagnostics

```
cargo run -- parse "(a|ab)(b|)" <INPUT> --parser <PARSER> --diag 3
```

Field	Value
Expression	"(a ab)(b)", that is $(a + ab)(b + \varepsilon)$
Inputs	"ab" (match), "b" (no match)
Parsers	deriv_std_rec, deriv_bc, pderiv_std, pderiv_bc
Output file	reports/report.txt
Omitted	the Timestamp line
Wrapped	long lines
∂	d
.	.
$\oplus$	(+)
ε	epsilon
ϕ	phi
✓	[OK]
Policy	the implementation's internal label, not a claim about disambiguation policy

C.1 deriv_std_rec, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \  
--parser deriv_std_rec --diag 3
```

```
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Standard (recursive parser)
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
=====
```

```
TIMING
=====
Parse time:          0.012ms
Expressions stored:  3 (r0 -> r1 -> r2)
=====
```

```
FORWARD PASS (Derivatives)
=====
```

```
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
```

```
Step 1: d(r0, 'a')
```

```
rule: d(r1.r2, 'a') [nullable(r1)=false]
      = d(r1, 'a').r2
```

```
r1 = Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps))
```

```
nullable(r1) = true [OK] position 1 matched
```

```
Step 2: d(r1, 'b')
```

```
rule: d(r1.r2, 'b') [nullable(r1)=true]
      = d(r1, 'b').r2 + d(r2, 'b')
```

```
r2 = Alt(Seq(Alt(Phi, Alt(Seq(Phi, Lit('b'))), Eps)), Alt(Lit('b'),
Eps)), Alt(Eps, Phi))
```

```
nullable(r2) = true [OK] position 2 matched
```

```
-----=====
NULLABILITY CHECK
```

```
-----=====
nullable(r2) = true -> proceed to backward pass
```

```
-----=====
BACKWARD PASS (mkEps + inject)
```

```
-----=====
mkEps(r2):
```

```
mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))
```

```
v2 = Left (Right Right (), Right ())
```

```
inject(r1, 'b', v2):
```

```
inject(Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps)), 'b',
Left (Right Right (), Right ()))
```

```
-> (Right ((), b), Right ())
```

```
v1 = (Right ((), b), Right ())
```

```
inject(r0, 'a', v1):
```

```
inject(Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
Eps)), 'a', (Right ((), b), Right ()))
```

```
-> (Right (a, b), Right ())
```

```
v0 = (Right (a, b), Right ())
```

```
-----=====
RESULT
```

```
-----=====
Parse tree:          (Right (a, b), Right ())
```

```
Flattened:           "ab"
```

```
-----=====
END OF REPORT
```

C.2 deriv_bc, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser deriv_bc --diag 3
-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
-----=====
TIMING
-----=====
Parse time:          0.015ms
Derivative steps:    2
-----=====
INTERNALIZE
-----=====
internalize((a|ab)(b|))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([]@[false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([]@[false]@'b' (+) [true]@epsilon))
-----=====
FORWARD PASS (deriv_bc + simp)
-----=====
Step 1 (char: 'a', position 1)
ri0 = []@([]@([]@[false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([]@[false]@'b' (+) [true]@epsilon))
deriv_bc(ri0, 'a'):
  rule: (bs@ri1.ri2) \ 'a' [nullable_bc(ri1)=false]
    = bs@(ri1\'a'.ri2)
simp -> []@([]@([]@[false]@epsilon (+) [true]@'b') . []@([]@[false]@'b' (+)
  [true]@epsilon))
ri1 = []@([]@([]@[false]@epsilon (+) [true]@'b') . []@([]@[false]@'b' (+)
  [true]@epsilon))
  nullable_bc(ri1) = true [OK] bits accumulating

Step 2 (char: 'b', position 2)
ri1 = []@([]@([]@[false]@epsilon (+) [true]@'b') . []@([]@[false]@'b' (+)
  [true]@epsilon))
deriv_bc(ri1, 'b'):
  rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=true]
    = bs@((ri1\'b'.ri2) (+) (fuse (mkEpsBC ri1) (ri2\'b')))
simp -> []@([true]@([]@[false]@'b' (+) [true]@epsilon) (+) [false,
  false]@epsilon)
ri2 = []@([true]@([]@[false]@'b' (+) [true]@epsilon) (+) [false,
```

```

    false]@epsilon)
nullable_bc(ri2) = true   [OK]   bits accumulating

-----
NULLABILITY CHECK
-----
nullable_bc(ri2) = true -> proceed to mkEpsBC + decode
-----
MKEEPSBC + DECODE
-----
mkEpsBC(ri2) = [11]

decode((a|ab)(b|), [11]):
  = (Right (a, b), Right ())
-----
RESULT
-----
Bits:                [11]
Parse tree:          (Right (a, b), Right ())
Flattened:           "ab"
-----
END OF REPORT
-----

```

C.3 pderiv_std, match

```

$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser pderiv_std --diag 3
-----
REGEX ENGINE DEBUG REPORT
-----
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              GREEDY
-----
TIMING
-----
Parse time:          0.017ms
Steps:               2
-----
INITIAL FRONTIER
-----
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
-----
FORWARD PASS (pderiv_tree per residual, frontier flattened)

```

```

-----=====
Step 1 (char: 'a', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  pderiv_tree('a', .) applied to each:
    from [0]:
      rule: pderiv_tree('a', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not printable)
-> Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps)) (+ injection closure,
      not printable)
  frontier after (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  a residual in this frontier is nullable [OK] parse tree available

Step 2 (char: 'b', position 2)
  frontier before (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=true]
= nub_tree $ [(r1'.r2, inj: continue r1) | (r1',_) <-
      pderiv_tree(c,r1)]
++ [(r2', inj: mk_eps(r1) paired with r2's result) | (r2',_) <-
      pderiv_tree(c,r2)]
      -> Eps (+ injection closure, not printable)
    from [1]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not printable)
  frontier after (2 residual(s)):
    [0] Eps
    [1] Seq(Eps, Alt(Lit('b'), Eps))
  a residual in this frontier is nullable [OK] parse tree available

-----=====
SELECTION (first nullable residual, priority/list order)
-----=====
any nullable residual in final frontier = true -> proceed to injection
(already applied by select)
-----=====
RESULT
-----=====
Parse tree:          (Left a, Left b)
Flattened:           "ab"
-----=====

```

END OF REPORT

C.4 pderiv_bc, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser pderiv_bc --diag 3
```

REGEX ENGINE DEBUG REPORT

```
Mode:                Bitcoded Partial-Derivative (pDerivBC)
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              GREEDY
```

TIMING

```
Parse time:          0.012ms
Steps:               2
```

INITIAL FRONTIER

```
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
```

FORWARD PASS (pDerivBC per residual, frontier flattened)

Step 1 (char: 'a', position 1)

frontier before (1 residual(s)):

```
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  bits-so-far = []
```

pDerivBC('a', .) applied to each:

from [0]:

```
rule: pDerivBC('a', r1.r2) [nullable(r1)=false]
      = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
-> (Alt(Lit('b'), Eps), [0])
-> (Seq(Lit('b'), Alt(Lit('b'), Eps)), [1])
```

frontier after (2 residual(s)):

```
[0] Alt(Lit('b'), Eps)  bits = [0]
[1] Seq(Lit('b'), Alt(Lit('b'), Eps))  bits = [1]
```

a residual in this frontier is nullable [OK] bits accumulating

Step 2 (char: 'b', position 2)

frontier before (2 residual(s)):

```
[0] Alt(Lit('b'), Eps)  bits-so-far = [0]
[1] Seq(Lit('b'), Alt(Lit('b'), Eps))  bits-so-far = [1]
```

pDerivBC('b', .) applied to each:

```

    from [0]:
rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++ tag(1,
    pDerivBC('b',r2)), nub2
    -> (Eps, [0])
    from [1]:
    rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
        = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
    -> (Alt(Lit('b'), Eps), [])
frontier after (2 residual(s)):
[0] Eps    bits = [00]
[1] Alt(Lit('b'), Eps)    bits = [1]
a residual in this frontier is nullable [OK] bits accumulating

=====
SELECTION (first nullable residual, priority/list order)
=====
any nullable residual in final frontier = true -> proceed to mkEpsBC +
    decode
=====
MKEEPSBC + DECODE
=====
selected residual's accumulated bits ++ mkEpsBC(residual) = [00]

decode((a|ab)(b|), [00]):
    = (Left a, Left b)
=====
RESULT
=====
Bits:                [00]
Parse tree:          (Left a, Left b)
Flattened:           "ab"
=====
END OF REPORT
=====

```

C.5 deriv_std_rec, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
    --parser deriv_std_rec --diag 3
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Standard (recursive parser)
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              POSIX
=====

```

TIMING

```
=====
Parse time:          0.005ms
Expressions stored:  2 (r0 -> r1)
=====
FORWARD PASS (Derivatives)
=====
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))

Step 1: d(r0, 'b')
  rule: d(r1.r2, 'b') [nullable(r1)=false]
        = d(r1, 'b').r2
r1 = Seq(Alt(Phi, Seq(Phi, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = false [FAIL] position 1

=====
NULLABILITY CHECK
=====
nullable(r1) = false -> no parse tree exists for full input
=====
PARTIAL RECOVERY
=====
No prefix matched.
=====
ERROR SUMMARY
=====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
  b
  ^
=====
END OF REPORT
=====
```

C.6 deriv_bc, no match

```
$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser deriv_bc --diag 3
```

```
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              POSIX
=====
```

```

TIMING
=====
Parse time:          0.008ms
Derivative steps:    1
=====
INTERNALIZE
=====
internalize((a|ab)(b|))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
=====
FORWARD PASS (deriv_bc + simp)
=====
Step 1 (char: 'b', position 1)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
deriv_bc(ri0, 'b'):
  rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=false]
    = bs@(ri1\'b'.ri2)
simp -> phi
ri1 = phi
nullable_bc(ri1) = false [FAIL]

=====
NULLABILITY CHECK
=====
nullable_bc(ri1) = false -> no full parse tree exists
=====
PARTIAL RECOVERY
=====
No prefix matched.
=====
ERROR SUMMARY
=====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
  b
  ^
=====
END OF REPORT
=====

```

C.7 pderiv_std, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser pderiv_std --diag 3

```

```

-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              GREEDY
-----=====
TIMING
-----=====
Parse time:          0.007ms
Steps:               1
-----=====
INITIAL FRONTIER
-----=====
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
    Alt(Lit('b'), Eps))
-----=====
FORWARD PASS (pderiv_tree per residual, frontier flattened)
-----=====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
  -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier [FAIL]

-----=====
SELECTION (first nullable residual, priority/list order)
-----=====
any nullable residual in final frontier = false -> no full parse tree
exists
-----=====
PARTIAL RECOVERY
-----=====
No prefix matched.
-----=====
ERROR SUMMARY
-----=====
Failure at position: 1 (1-indexed)
Found:               'b'
Expected:             'a'
  b

```

```

~
-----=====
END OF REPORT
-----=====

```

C.8 pderiv_bc, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser pderiv_bc --diag 3

```

```

-----=====
REGEX ENGINE DEBUG REPORT

```

```

-----=====
Mode:                Bitcoded Partial-Derivative (pDerivBC)
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              GREEDY

```

```

-----=====
TIMING

```

```

-----=====
Parse time:          0.006ms
Steps:              1

```

```

-----=====
INITIAL FRONTIER

```

```

-----=====
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))

```

```

-----=====
FORWARD PASS (pDerivBC per residual, frontier flattened)

```

```

-----=====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
    bits-so-far = []
  pDerivBC('b', .) applied to each:
    from [0]:
      rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
          = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
      -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier [FAIL]

```

```

-----=====
SELECTION (first nullable residual, priority/list order)

```

```

-----=====

```

```

any nullable residual in final frontier = false -> no full parse tree
exists

```

```

-----=====

```

PARTIAL RECOVERY

No prefix matched.

ERROR SUMMARY

Failure at position: 1 (1-indexed)
Found: 'b'
Expected: 'a'
b
^

END OF REPORT
